

ENTITY FRAMEWORK İLE OTOMASYON PROJESİ

MSSQL İLE VERİ TABANI VE TABLOLARIN OLUŞTURULMASI

Bu otomasyon projesinde, bir müşterinin bir ürün almasını sağlayan proje yapılmaktadır. Bu nedenle, SQL tarafında tabloları oluştururken;

Ürün bilgilerini içeren Table_Product,

Müşteri bilgilerini içeren Table_Customer,

Ürünün hangi kategoriye ait olduğunu içeren Table_Category,

Hangi müşterinin hangi ürünü aldığı bilgisini tutan Table_CustomerProduct isimli 4 adet tablomuz vardır.

Burada amaç, bir ürün satın alınırken, gerekli olan asgari bilgilerin saklanması ve işlenmesidir.

SQL tarafındaki Tabloların sütunları ve veri tipleri sonraki sayfalarda yer almaktadır.

MSSQL İLE VERİ TABANI VE TABLOLARIN OLUŞTURULMASI

	Column Name	Data Type	Allow Nulls
🔑	Product_Id	int	☐
	Product_Name	nvarchar(50)	☐
	Product_Price	decimal(18, 0)	☐
	Category_Id	int	☐

(a)

	Column Name	Data Type	Allow Nulls
🔑	Customer_Id	int	☐
	Customer_Name	nvarchar(50)	☐
	Customer_Surname	nvarchar(50)	☐
	Customer_Email	nvarchar(50)	☐
	Customer_Telephone	nvarchar(50)	☐

(b)

	Column Name	Data Type	Allow Nulls
🔑	Category_Id	int	☐
	Category_Name	nvarchar(50)	☐

(c)

	Column Name	Data Type	Allow Nulls
🔑	CusPro_Id	int	☐
	Customer_Id	int	☐
	Product_Id	int	☐

(d)

a: Table_Product, b: Table_Customer, c: Table_Category, d: Table_CustomerProduct

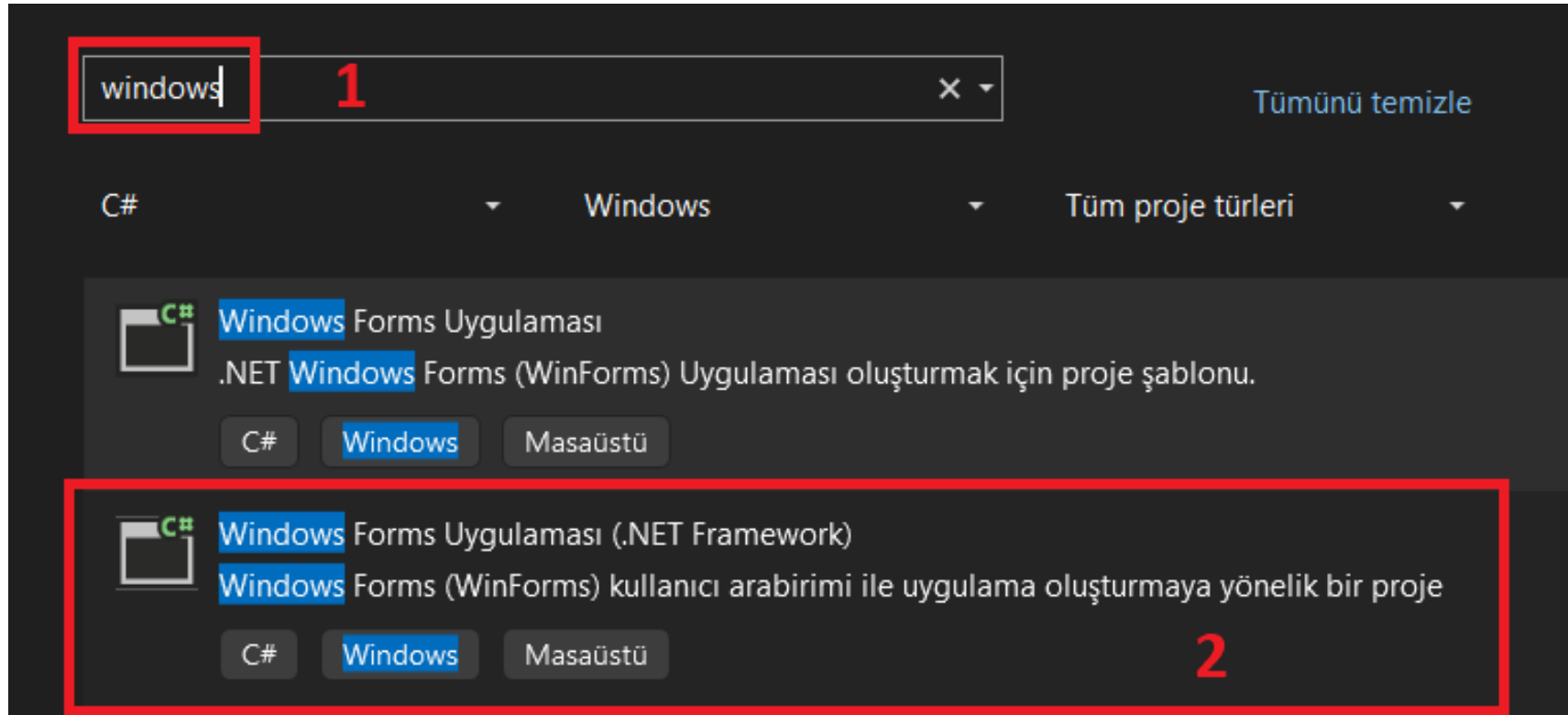
VİSUAL STUDIO İLE WINDOWS FORM PROJESİ OLUŞTURULMASI

Şimdi de, Visual Studio'yu açıp .Net framework ortamında Windows Form Uygulaması oluşturacağız. Bunu, otomasyon projemize arayüz sağlamak için gerçekleştiriyoruz.

Visual Studio'yu açtıktan sonra sağ taraftaki 4. seçenek olan Yeni bir proje oluştur butonuna tıklayınız.

VİSUAL STUDIO İLE WINDOWS FORM PROJESİ OLUŞTURULMASI

Karşınıza gelen menüden arama çubuğuna «Windows» yazınız ve çıkan sonuçlardan Windows Form Uygulaması (.Net Framework) seçeneğini seçiniz.



VİSUAL STUDIO İLE WINDOWS FORM PROJESİ OLUŞTURULMASI

Projemizin türünü seçtikten sonra projemize isim olarak Otomasyon_Projesi adını veriyoruz ve Oluştur butonuna tıklayarak projemizi oluşturuyoruz.

Yeni projenizi yapılandırın

Windows Forms Uygulaması (.NET Framework) C# Windows Masaüstü

Proje adı

Otomasyon_Projesi 1

Konum

C:\Users\MSI\source\repos

Çözüm adı ⓘ

Otomasyon_Projesi

☐ Çözümü ve projeyi aynı dizine yerleştirin

Altyapı

.NET Framework 4.7.2

Proje "C:\Users\MSI\source\repos\Otomasyon_Projesi\Otomasyon_Projesi\' içinde oluşturulacak

Geri Oluştur 2

FORM BİLEŞENLERİNİN OLUŞTURULMASI

Karşınıza gelen boş Windows form içerisine aşağıdaki bileşenleri ekliyoruz:

1 adet datagridview,

4 adet buton (ekleme, silme, güncelleme ve listemele işlemleri için)

1 adet groupBox ve groupBox içerisine (4 adet label, 4 adet textbox) ekliyoruz.

groupBox içerisindeki bileşenleri veri ekleme ve veri güncelleme işlemleri için ekliyoruz.

FORM BİLEŞENLERİNİN OLUŞTURULMASI

Eklediğimiz bileşenler arasında butonların ve textbox'ların id bilgisini değiştirmeliyiz. Bunun nedeni, işlemler esnasında bu bileşenler kullanılırken hangi bileşenin hangi görev için ayarlandığını unutmamaktır. Örneğin, yeni bir müşteri eklerken 1. textbox bileşenine kullanıcı adı bilgisini gireceğimiz için bu bileşene `txt_isim` diyeceğiz ya da güncelleme işlemi için eklediğimiz butonun id bilgisine `btn_guncelle` diyeceğiz.

Visual studio'da bir bileşenin (toolbox penceresinden eklenen bir form öğesinin) id ya da text bilgisini değiştirmek için o bileşene bir defa sol tıklayıp seçtikten sonra sağ tıklayıp Özellikler (Properties) seçeneğini seçerseniz sağ alt tarafa Özellikler penceresi eklenecektir.

Oradan, ilgili bileşenin text bilgisini (Örneğin bir butonun üzerinde EKLE yazması için), ya da id bilgisini (bir butonun tıklama işlevi için kod yazarken hangi buton olduğunu belirtmek için) düzenleyebilirsiniz.

FORM BİLEŞENLERİNİN OLUŞTURULMASI

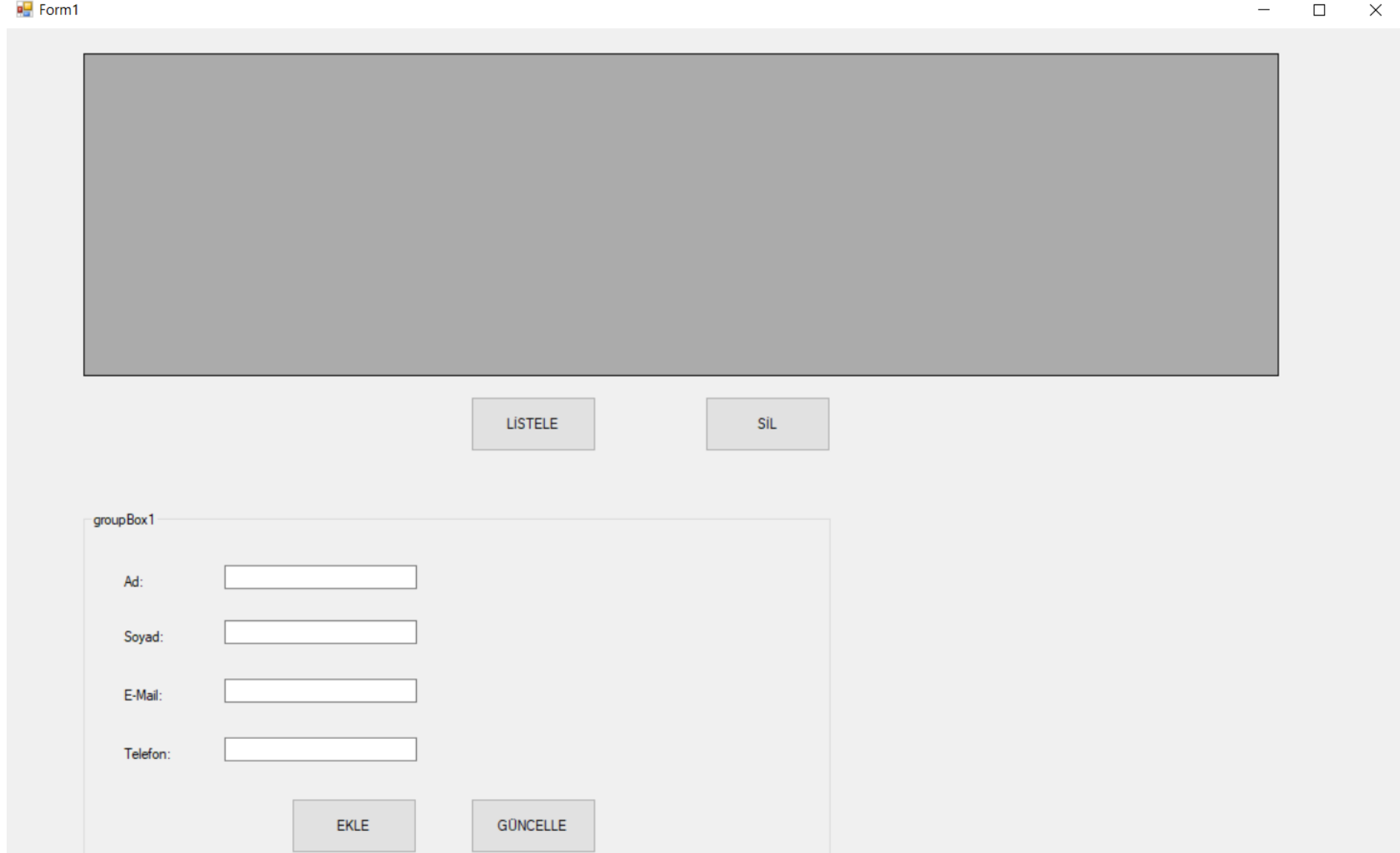
Form'a eklediğimiz bileşenler arasında butonların ve textbox'ların id bilgileri aşağıdaki gibidir:

btn_ekle, btn_guncelle, btn_sil, btn_listele

txt_isim, txt_soyisim, txt_email, txt_telefon

FORM BİLEŞENLERİNİN OLUŞTURULMASI

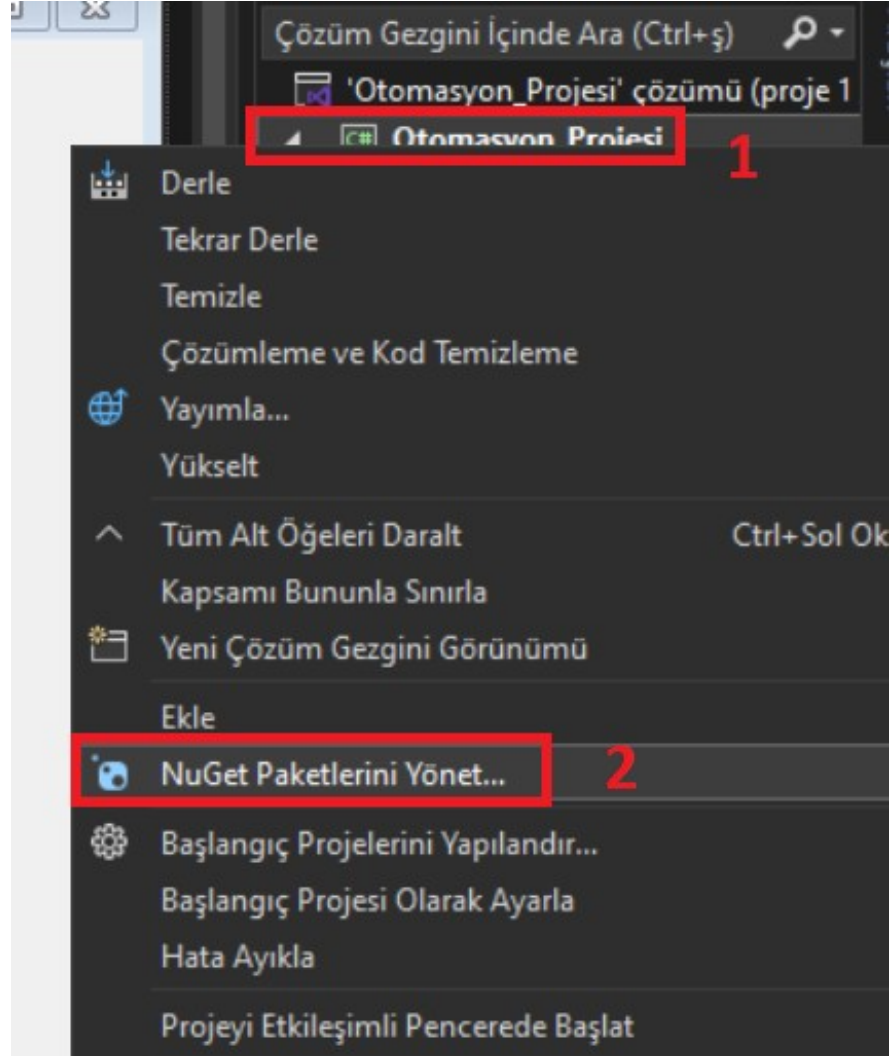
Form'u oluşturup, özelliklerini tamamladıktan sonra görünümü aşağıdaki gibidir.



The screenshot displays a Windows application window titled "Form1". The window contains a large, empty gray rectangular area at the top, likely intended for a list or data display. Below this area are two buttons: "LISTELE" (List) and "SİL" (Delete). At the bottom of the window is a group box labeled "groupBox1". Inside this group box, there are four text labels: "Ad:", "Soyad:", "E-Mail:", and "Telefon:", each followed by a text input field. Below these input fields are two buttons: "EKLE" (Add) and "GÜNCELLE" (Update).

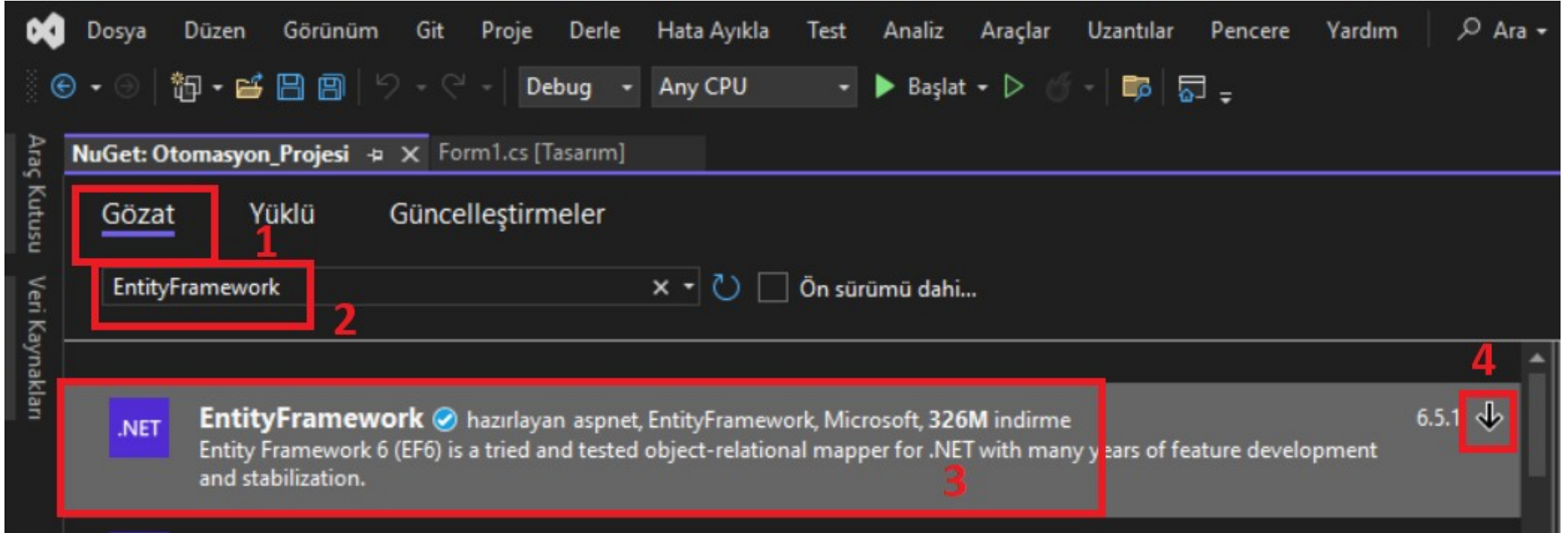
PROJEYE ENTITY FRAMEWORK KURULUMU

Şimdi, projemize Entity Framework'ü eklememiz gerekli. Bunun için projemize sağ tıklayıp NuGet paketlerini yönet (Manage NuGet Packages) seçeneğini tıklıyoruz.



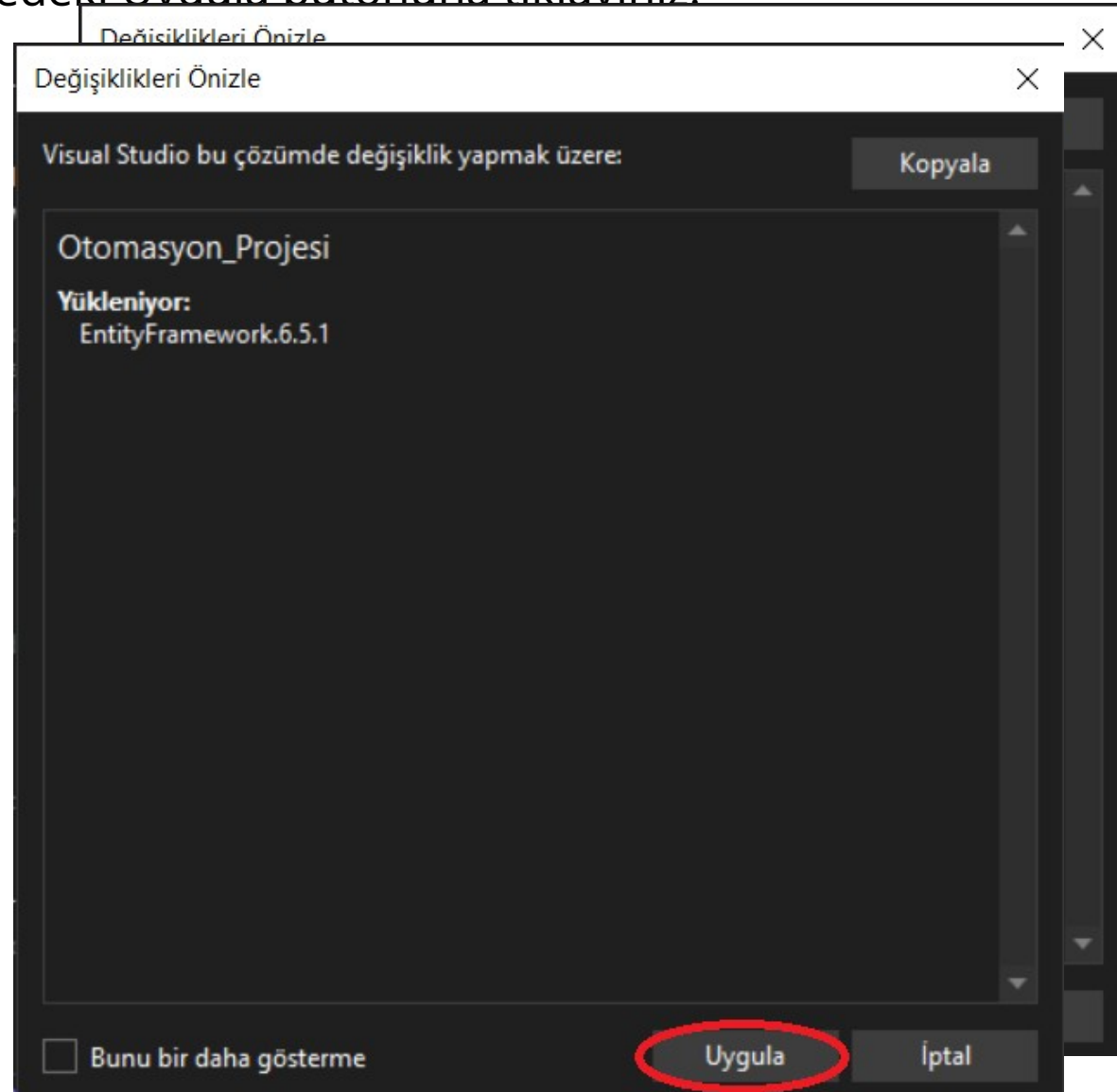
PROJEYE ENTITY FRAMEWORK KURULUMU

Karşıımıza gelen pencereden Gözet (Browse) sekmesine tıklayınız. Ara kısmına EntityFramework yazınız ve çıkan sonuçlardan EntityFramework'ü tıklayınız ve sağında yer alan indirmek simgesi olan aşağı ok simgesine tıklayınız. NOT: Bunun için internete bağlı olmanız gerekmektedir.



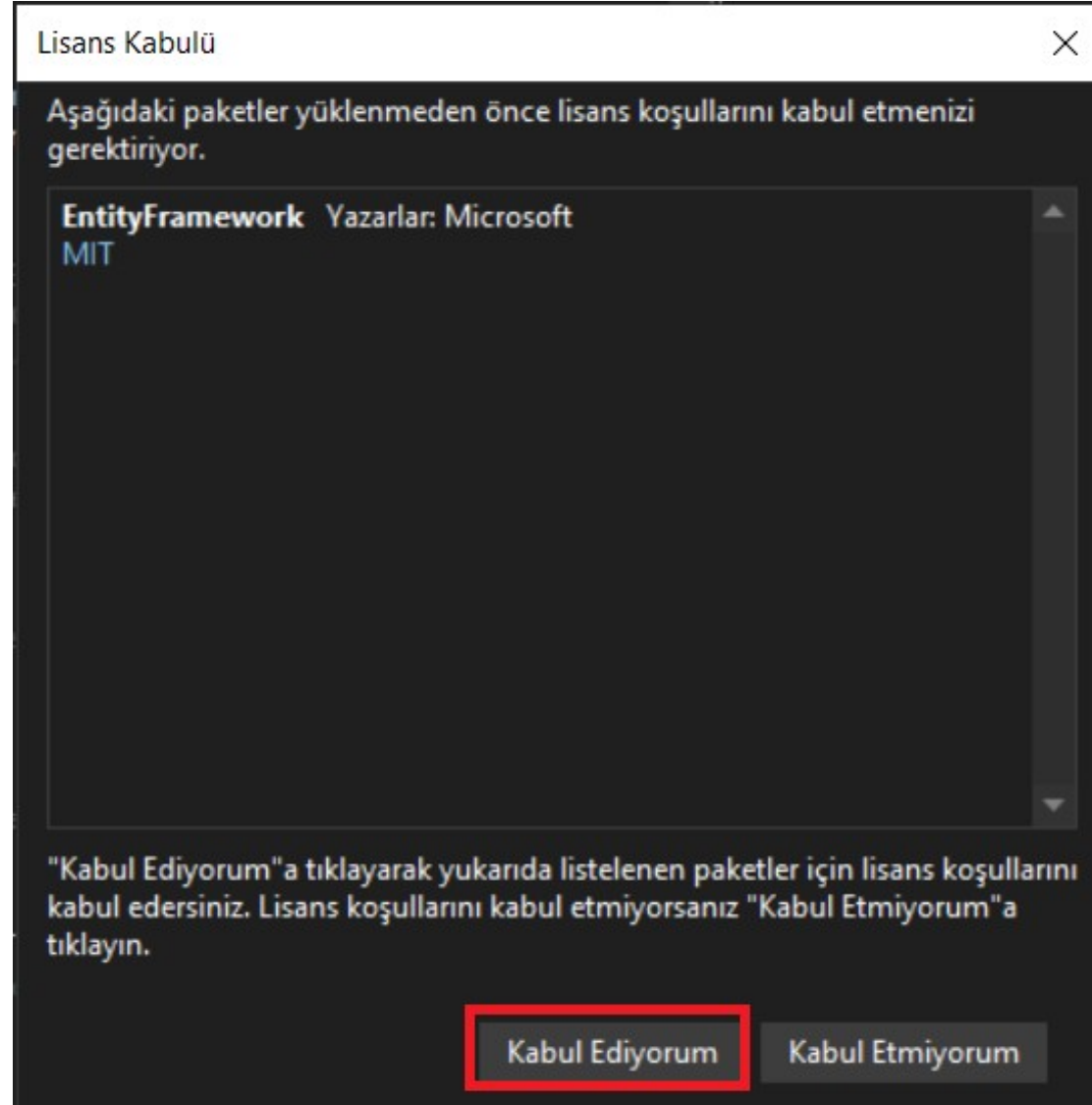
PROJEYE ENTITY FRAMEWORK KURULUMU

Karşınıza gelen penceredeki Uygula butonuna tıklavınız.



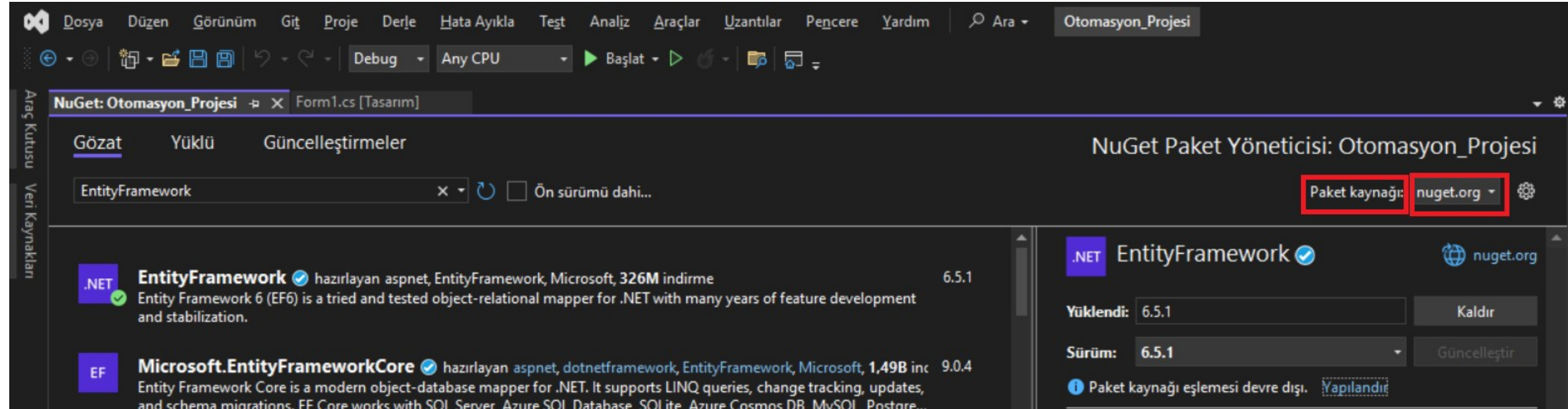
PROJEYE ENTITY FRAMEWORK KURULUMU

Karşınıza gelen penceredeki Kabul Ediyorum butonuna tıklayınız.



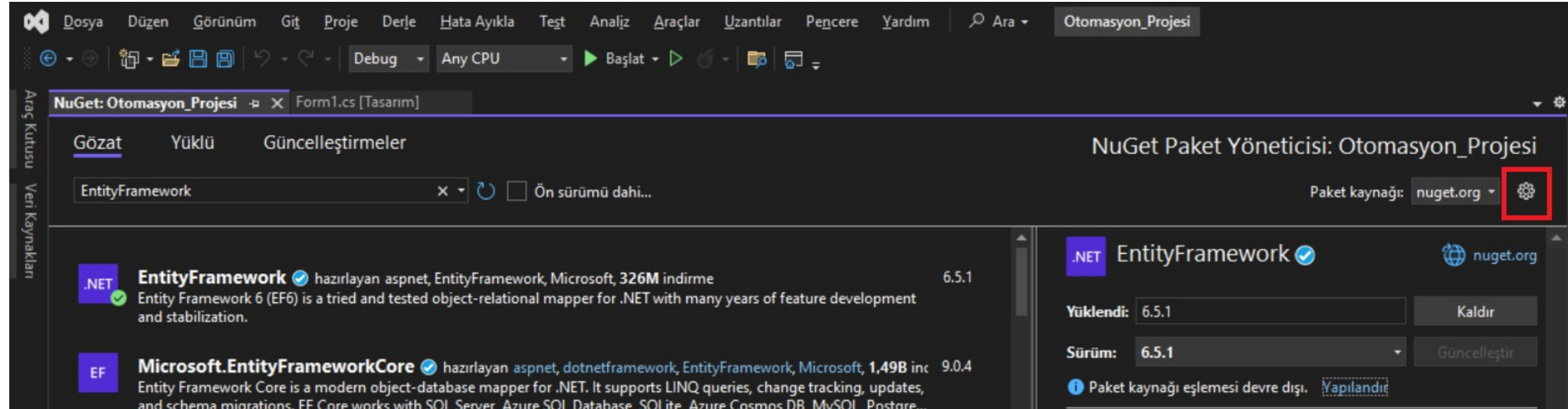
PROJEYE ENTITY FRAMEWORK KURULUMU

Bu şekilde EntityFramework'ü projemize kurmuş olduk. Eğer kurulumda bir sorun yaşarsanız. Aşağıdaki gibi Paket Kaynağı: kısmında nuget.org yazdığından emin olun.



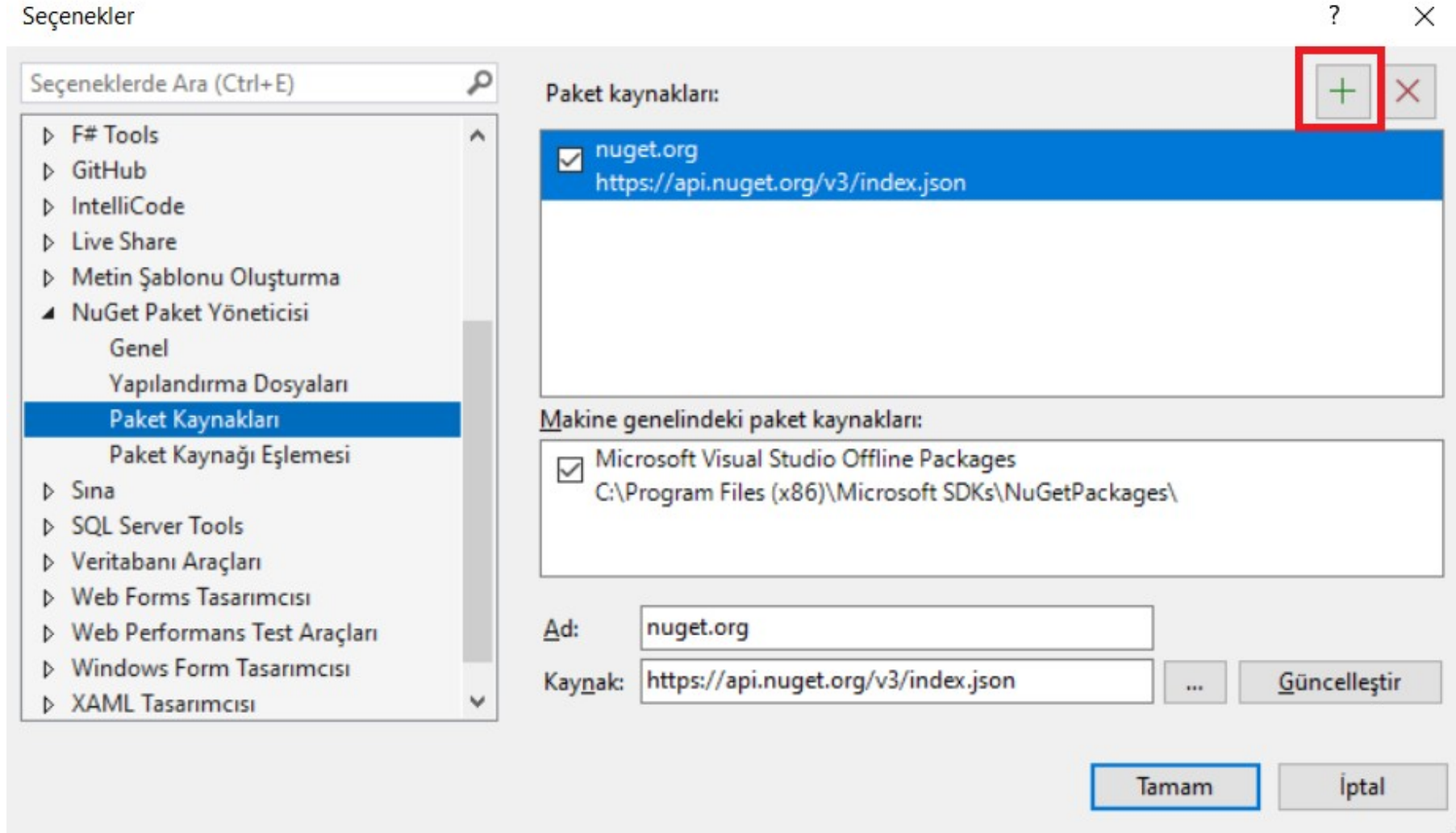
PROJEYE ENTITY FRAMEWORK KURULUMU

Eğer paket kaynağı olarak nuget.org seçili değilse Ayarlar simgesine tıklayınız.



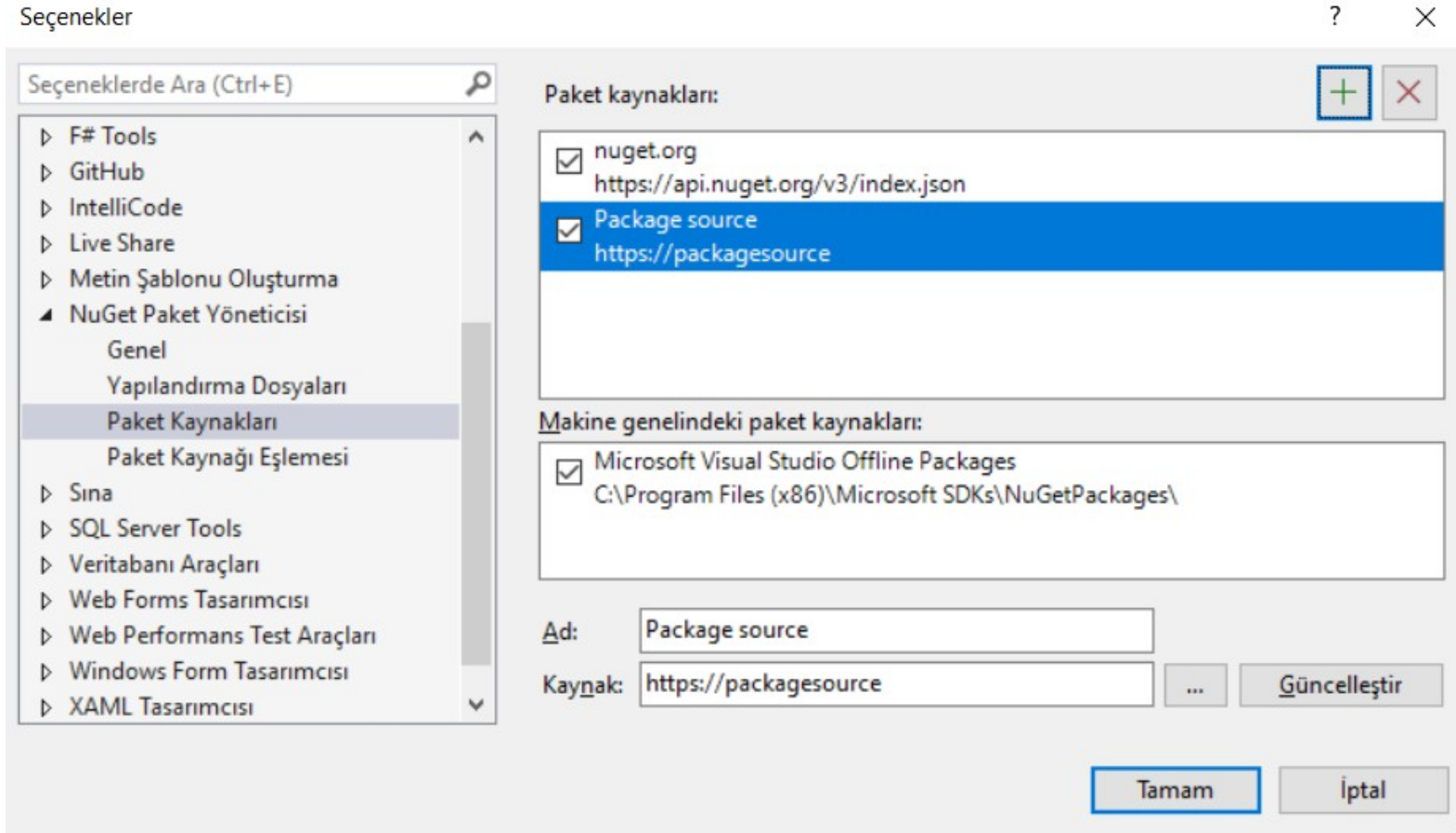
PROJEYE ENTITY FRAMEWORK KURULUMU

Karşınıza gelen pencereden + simgesine tıklayınız.



PROJEYE ENTITY FRAMEWORK KURULUMU

Karşınıza aşağıdaki gibi bir menü gelecek. Bu menüde Ad ve Kaynak kısımlarına aşağıdaki kodları yapıştırıp Tamam butonuna tıklayınız ve önceki işlemleri tekrarlayınız (EntityFramework yazıp kurun

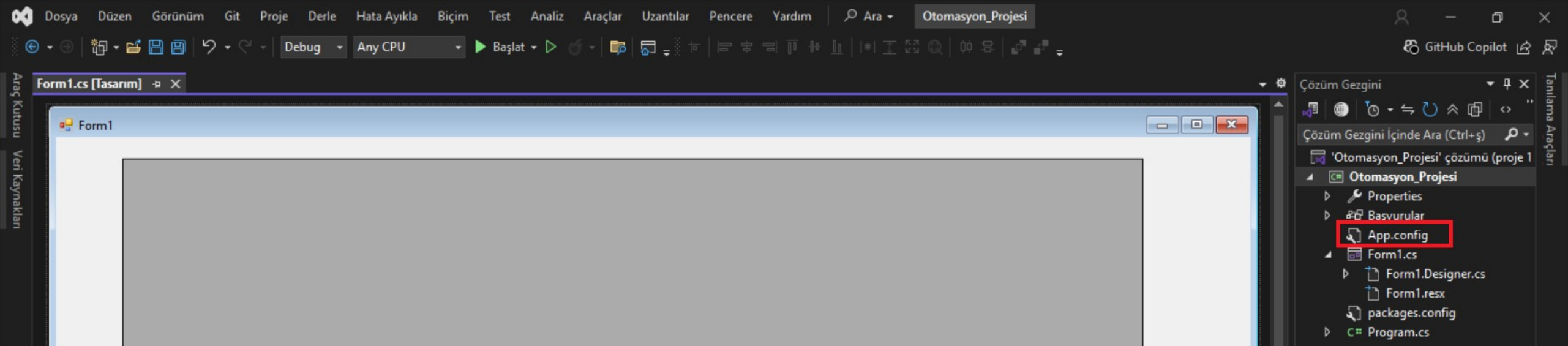


Ad: nuget.org Kaynak: https://api.nuget.org/v3/index.json

ConnectionString Bilgisini Eklemek

EntityFramework'ü projemize kurduktan sonra şimdi de veri tabanı bağlantısını tanıtmak için Proje menüsünde yer alan App.config dosyası içerisine connectionstring bilgisini eklemeliyiz.

App.config dosyasına çift tıklayarak açınız.



ConnectionString Bilgisini Eklemek

Aşağıda bir connectionstring dizgesi yer almaktadır.

```
<connectionStrings>
```

```
  <add name="CustomerDbContext"
```

```
    connectionString="Data Source= server_adını_yazınız;Initial Catalog= veri_tabanı_adını_yazınız;Integrated Security=True"
```

```
    providerName="System.Data.SqlClient" />
```

```
</connectionStrings>
```

Buradaki name=, server_adını_yazınız ve veri_tabanı_adını_yazınız yazan yerlere kendi bilgilerinizi girmelisiniz:

Özellikle name="CustomerDbContext" kısmı çok önemlidir. Siz, kendi connectionstring bilginizde bu alanı da değiştirip kendi projenize uygun bir isim vermelisiniz.

ConnectionString Bilgisini Ekleme

Server name bilgisini öğrenmek için SQL SERVER MANAGEMENT STUDIO'yu açınız:

Görseldeki kırmızı alan ile seçili yerdeki bilgi, sizin server name bilginizdir. Bunu kopyalayıp, sizinle paylaşılan connectionstring kodlarındaki server_adını_yazınız yazan yere yapıştırın

Connect to Server

SQL Server

Login | Connection Properties | Always Encrypted | Additional Connection Parameters

Server

Server type: Database Engine

Server name: **DESKTOP-7DAR20\SQLEXPRESS**

Authentication: Windows Authentication

User name: DESKTOP-7DAR20\MSI

Password:

☐ Remember password

Connection Security

Encryption: Mandatory

☒ Trust server certificate

Host name in certificate:

Connect Cancel Help Options <<

ConnectionString Bilgisini Eklemek

Veri tabanı adı da zaten SQL'e bağlandıktan sonra kendi oluşturduğunuz veri tabanınızın adıdır. Dolayısıyla, bu proje için connectionstring bilgisi aşağıdaki gibi olmalıdır.

NOT: Siz de kendi bilgilerinize göre connectionstring bilginizi oluşturun!!!!

```
<connectionStrings>
```

```
  <add name="CustomerDbContext"
```

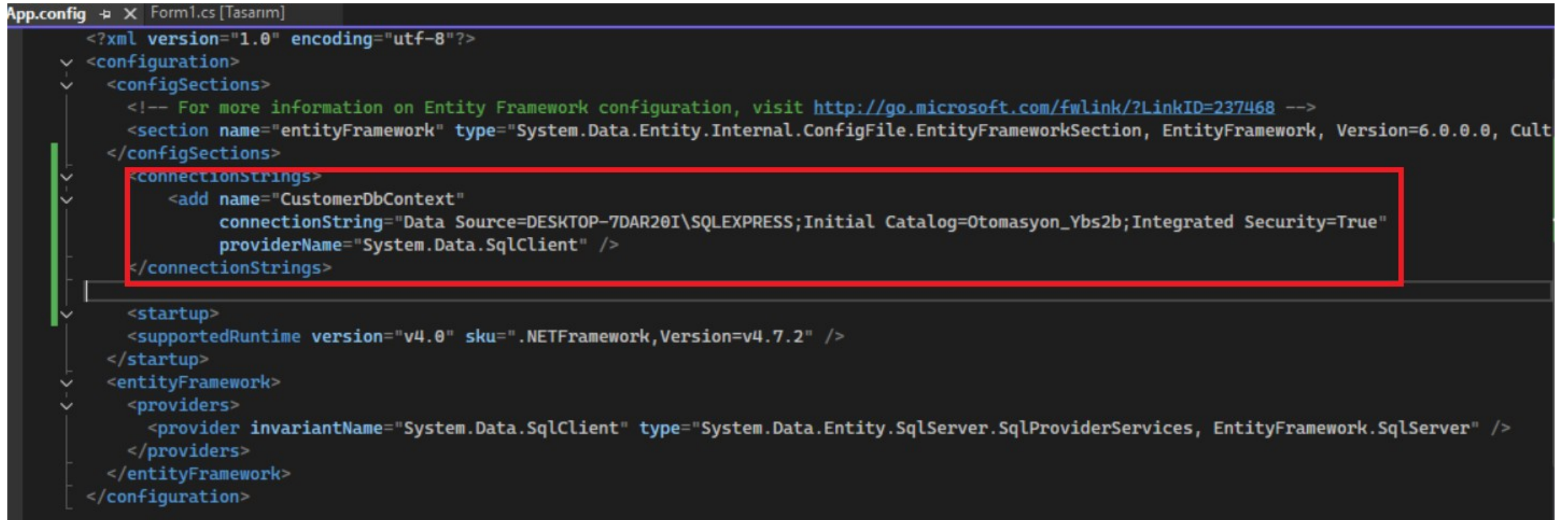
```
    connectionString="Data Source=DESKTOP-7DAR20I\SQLEXPRESS;Initial  
Catalog=Otomasyon_Ybs2b;Integrated Security=True"
```

```
    providerName="System.Data.SqlClient" />
```

```
</connectionStrings>
```

ConnectionString Bilgisini Eklemek

Şimdi, connectionstring bilgimizi oluşturduğumuza göre bunu App.config içerisindeki uygun etiketler arasına kopyala-yapıştır yapıyoruz.



```
App.config  Form1.cs [Tasarım]
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <configSections>
    <!-- For more information on Entity Framework configuration, visit http://go.microsoft.com/fwlink/?LinkID=237468 -->
    <section name="entityFramework" type="System.Data.Entity.Internal.ConfigFile.EntityFrameworkSection, EntityFramework, Version=6.0.0.0, Cult
  </configSections>
  <connectionStrings>
    <add name="CustomerDbContext"
      connectionString="Data Source=DESKTOP-7DAR20I\SQLEXPRESS;Initial Catalog=Otomasyon_Ybs2b;Integrated Security=True"
      providerName="System.Data.SqlClient" />
  </connectionStrings>
  <startup>
    <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.7.2" />
  </startup>
  <entityFramework>
    <providers>
      <provider invariantName="System.Data.SqlClient" type="System.Data.Entity.SqlServer.SqlProviderServices, EntityFramework.SqlServer" />
    </providers>
  </entityFramework>
</configuration>
```

Configurations etiketi kapandıktan sonra (/configurations) enter tuşuna basıp yapıştırıyoruz.
CTRL tuşuna

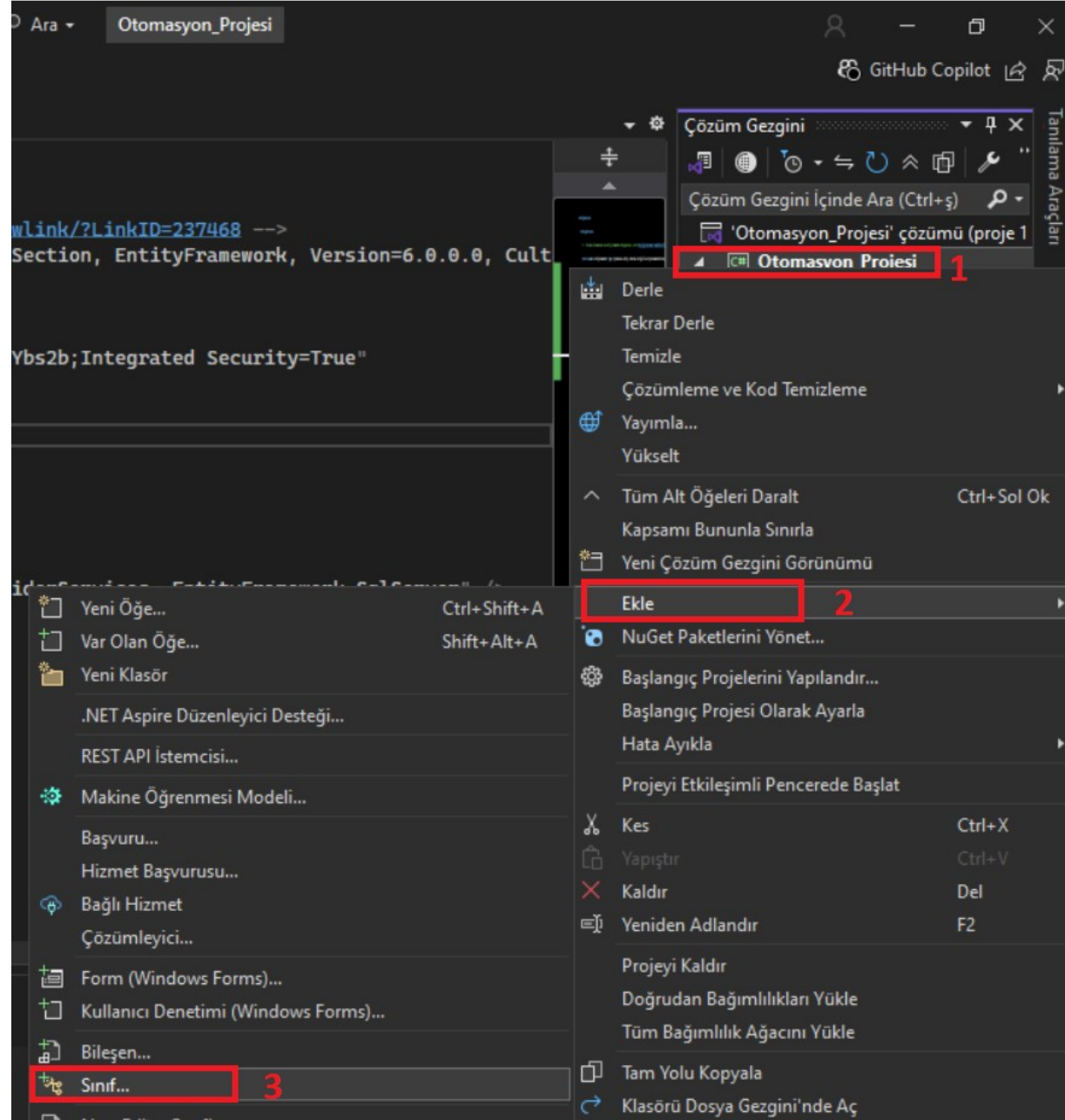
Basılı tutarken s tuşuna basıp kaydediyoruz.

Model Sınıfını Ekleme

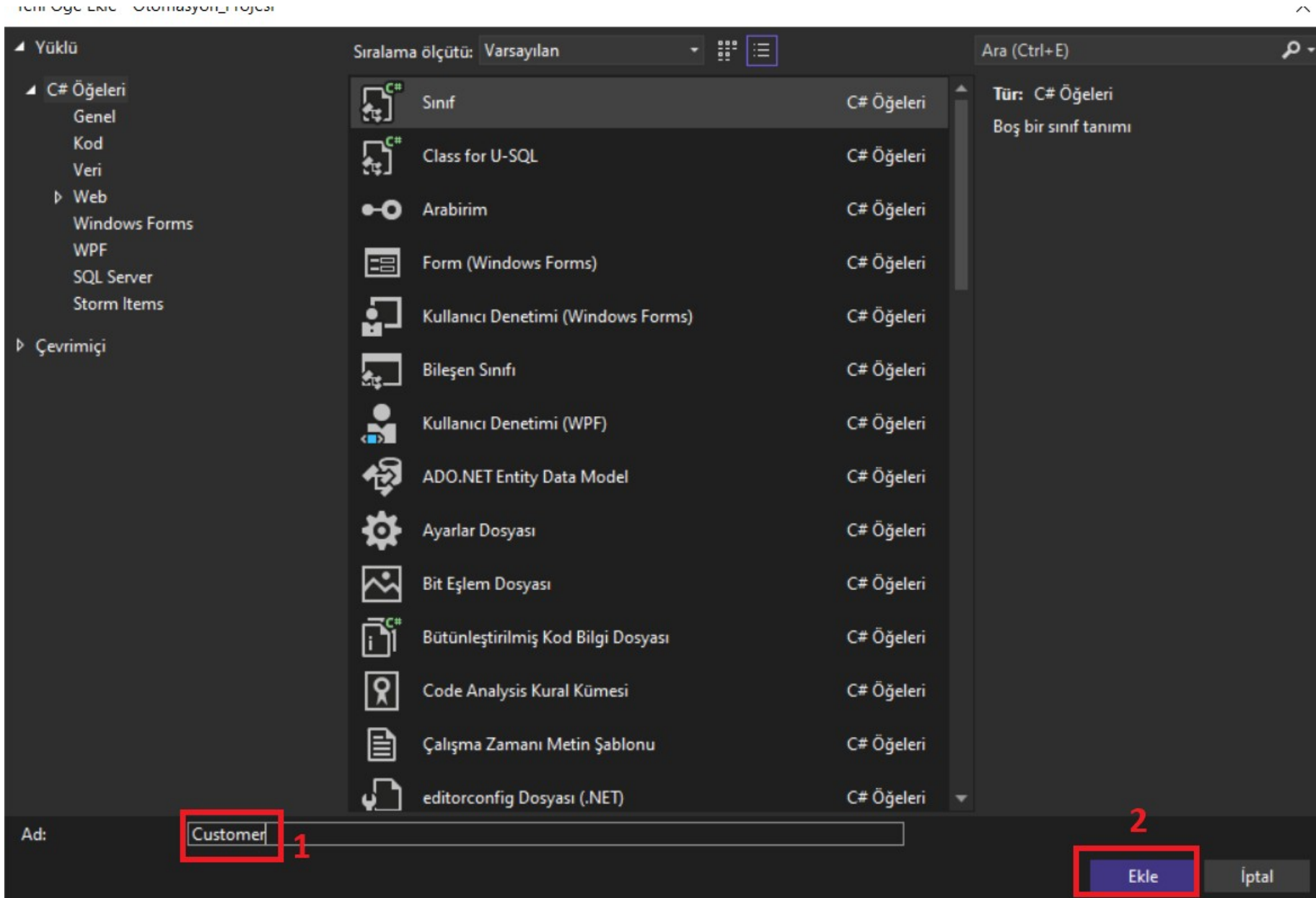
Hatırlarsanız, bu proje için oluşturulan 4 tablo vardı. Şimdi, ilk tablo olan Table_Customer için bir sınıf ekliyoruz. Yani, SQL tarafındaki tablomuzu Visual Studio tarafına bir class (sınıf) olarak tanıtıyoruz. Tablodaki sütun adları ve bu sütunların veri tiplerini olduğu gibi bir class olarak tanımlayacağız.

Şimdi projemize sağ tıklayıp Ekle->Sınıf seçeneğini seçiniz (Add->Class) daha sonra sınıfınıza bir isim veriniz. Ben, Table_Customer tablosu için bu sınıfa Customer ismini veriyorum.

Model Sınıfını Eklemek



Model Sınıfını Ekleme

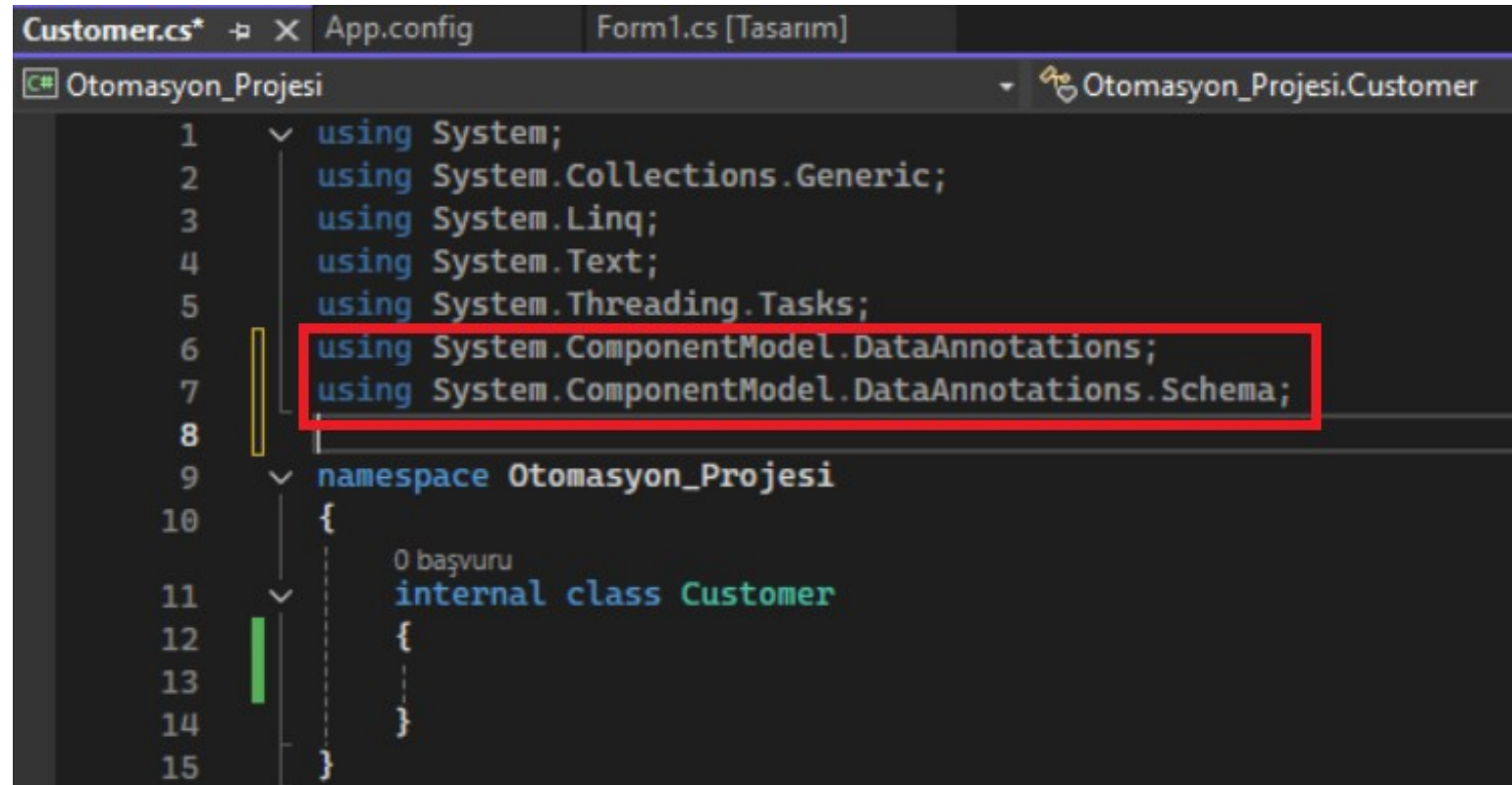


Model Sınıfını Eklemek

Sınıfınızı oluşturduktan sonra ilk olarak using System blokları arasına aşağıdaki kodları yapıştırın:

```
using System.ComponentModel.DataAnnotations;
```

```
using System.ComponentModel.DataAnnotations.Schema;
```



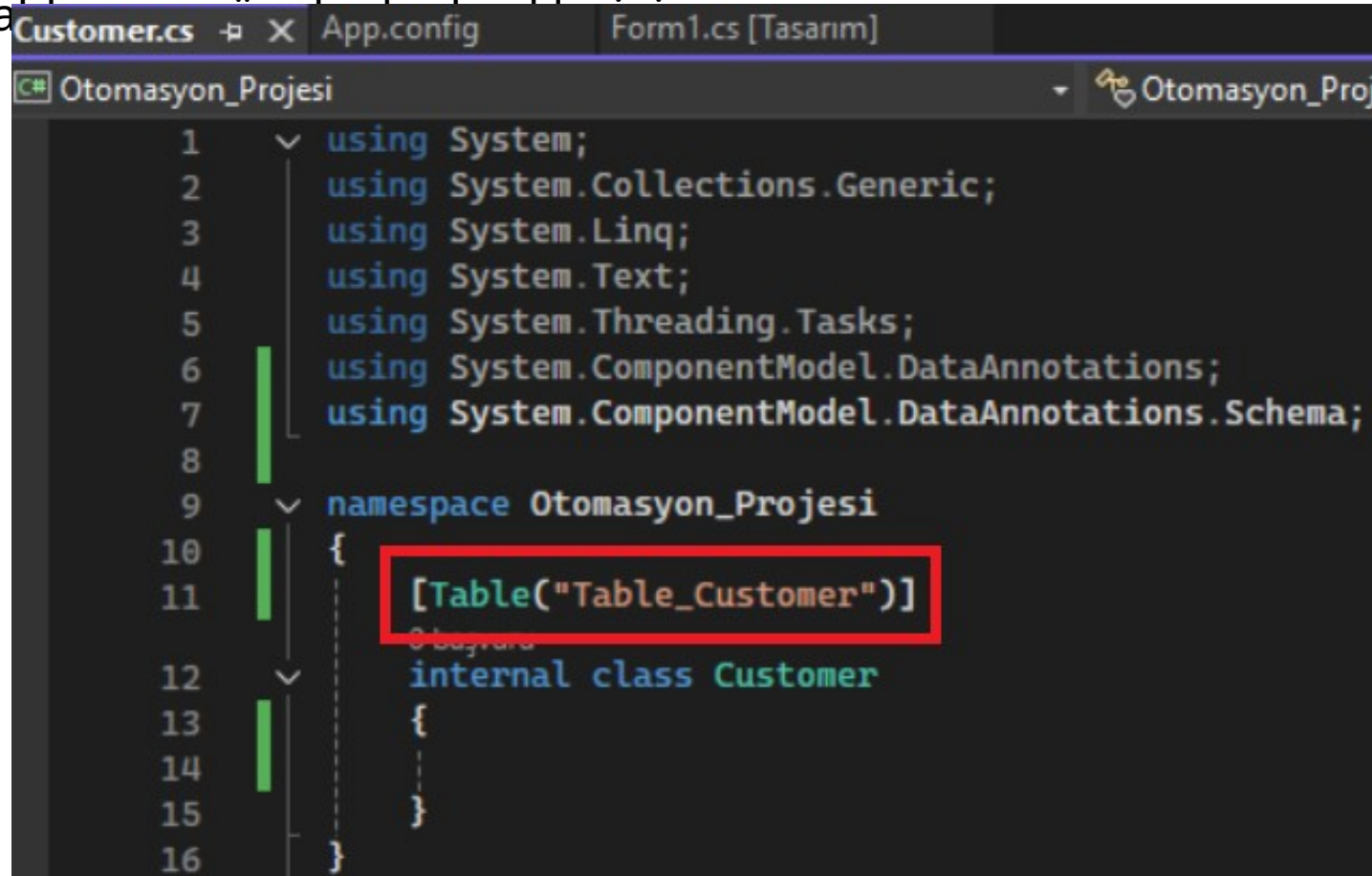
```
Customer.cs* X App.config Form1.cs [Tasarım]
[C#] Otomasyon_Projesi Otomasyon_Projesi.Customer
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6  using System.ComponentModel.DataAnnotations;
7  using System.ComponentModel.DataAnnotations.Schema;
8
9  namespace Otomasyon_Projesi
10 {
11     0 başvuru
12     internal class Customer
13     {
14     }
15 }
```

Model Sınıfını Eklemek

Şimdi de internal class Customer yazan yerin hemen üstüne (public alana) aşağıdaki kodu yapıştırıyoruz:

[Table("Table_Customer")] böylelikle sql ve c# bağlantısını kurmaya çalışıyoruz (entity framework konsepti dahilinde)

NOT: Sizler de kendi tablo



The screenshot shows a C# code editor with a file named `Customer.cs` open. The code is part of a project named `Otomasyon_Projesi`. The code includes several using statements for System, System.Collections.Generic, System.Linq, System.Text, System.Threading.Tasks, System.ComponentModel.DataAnnotations, and System.ComponentModel.DataAnnotations.Schema. It then defines a namespace `Otomasyon_Projesi`. Inside this namespace, there is an `internal class Customer`. A red rectangle highlights the attribute `[Table("Table_Customer")]` being added to the class definition. The code is as follows:

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6 using System.ComponentModel.DataAnnotations;
7 using System.ComponentModel.DataAnnotations.Schema;
8
9 namespace Otomasyon_Projesi
10 {
11     [Table("Table_Customer")]
12     internal class Customer
13     {
14     }
15 }
16
```

Model Sınıfını Ekleme

Şimdi, Bu class'ın (Customer) Table_Customer tablosu olduğunu tanıttık. Sıra geldi, Table_Customer tablosunun sütunlarını burada birer property (özellik) olarak tanıtmaya. Bunun için, Table_Customer tablosunun sütun isimlerine ve bu sütunları veri tipleri bilgisine ihtiyacımız var. Aşağıda, bu projedeki Table_Customer tablosunun sütun isimleri ve bu sütunların veri tipleri verilmektedir.

	Column Name	Data Type	Allow Nulls
▶🔑	Customer_Id	int	☐
	Customer_Name	nvarchar(50)	☐
	Customer_Surname	nvarchar(50)	☐
	Customer_Email	nvarchar(50)	☐
	Customer_Telephone	nvarchar(50)	☐

Model Sınıfını Ekleme

C# tarafında bir sütunun property olarak eklenmesinde eğer bu sütun SQL tarafında bir primary key'e karşılık geliyorsa bunun için onu [Key] olarak belirtiriz.

Eğer bu sütunun boş geçilmemesi gerekirse (Allow Nulls kısmı işaretlenmemişse boş geçilemez demektir)

[Required] olarak belirtiriz. Eğer SQL tarafındaki bu sütunun tipi string tipteyse (nvarchar, varchar, char) ve bir uzunluğu varsa (en fazla 10 karakter ya da 20 karakter ya da x karakter) bunu da [MaxLength(karakter_sayisi)] olarak belirtiriz.

Örneğin nvarchar(10) için [MaxLength(10)] olarak belirtiriz.

Tüm bunlara göre bu projedeki Table_Customer tablosunun sütunlarının, veri tipleriyle birlikte bir class olarak tanımlanması sonraki sayfadaki kodlarda verilmektedir.

Model Sınıfını Ekleme

```
[Table("Table_Customer")]
internal class Customer
{
    [Key]
    public int Customer_Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string Customer_Name { get; set; }

    [Required]
    [MaxLength(50)]
    public string Customer_Surname { get; set; }

    [Required]
    [MaxLength(50)]
    public string Customer_email { get; set; }

    [Required]
    [MaxLength(50)]
    public string Customer_telephone { get; set; }
}
```


Model Sınıfını Ekleme

```
namespace Otomasyon_Projesi
{
    [Table("Table_Customer")]
    0 başvuru
    internal class Customer
    {
        [Key]
        0 başvuru
        public int Customer_Id { get; set; }

        [Required]
        [MaxLength(50)]
        0 başvuru
        public string Customer_Name { get; set; }

        [Required]
        [MaxLength(50)]
        0 başvuru
        public string Customer_Surname { get; set; }

        [Required]
        [MaxLength(50)]
        0 başvuru
        public string Customer_email { get; set; }

        [Required]
        [MaxLength(50)]
        0 başvuru
        public string Customer_telephone { get; set; }
    }
}
```


Model Sınıfını Ekleme

Böylelikle SQL tarafındaki ilk tablomuz için bir class tanımladık. Şimdi, veri tabanı işlemleri için (ekleme, silme, güncelleme, listeleme) yapacağımız işlemleri EntityFramework konseptinde tanımlanmış bir sistem class'ı olan DbContext sınıfına ihtiyacımız var.

Customer sınıfı içerisinde sadece Table_Customer tablosunun içerdiği bilgileri (sütun isimleri ve veri tipleri) oluşturduk. Ancak, bu tablo için yapacağımız insert into, delete, update ve select sorguları için C# tarafında başka bir class tanımlayacağız. Bu class içerisinde sadece ekleme, silme, güncelleme ve listeleme işlemlerine dair kodlar yer alacak. Bu sayede, her bir iş için bir class tanımlamış olacağız ve mantıksal kod bütünlüğünü sağlayacağız. Yani, her iş için o işe uygun bir class yazacağız.

Veri Tabanı İşlemleri için Sınıf Tanımlamak

EntityFramework konseptinde veri tabanı işlemleri için hazırlanmış bir DbContext sınıfı vardır. Biz de kendi tablomuz için bir sınıf tanımlayıp, bu sınıftan miras alacağız. Bundan dolayı .Net geliştiricileri böyle bir class tanımlarken şu formülü izler: Tabloya karşılık gelen class adı (Örneğin bu örnekteki Customer) ve DbContext son eki (DbContext class'ını miras alacağımız için).

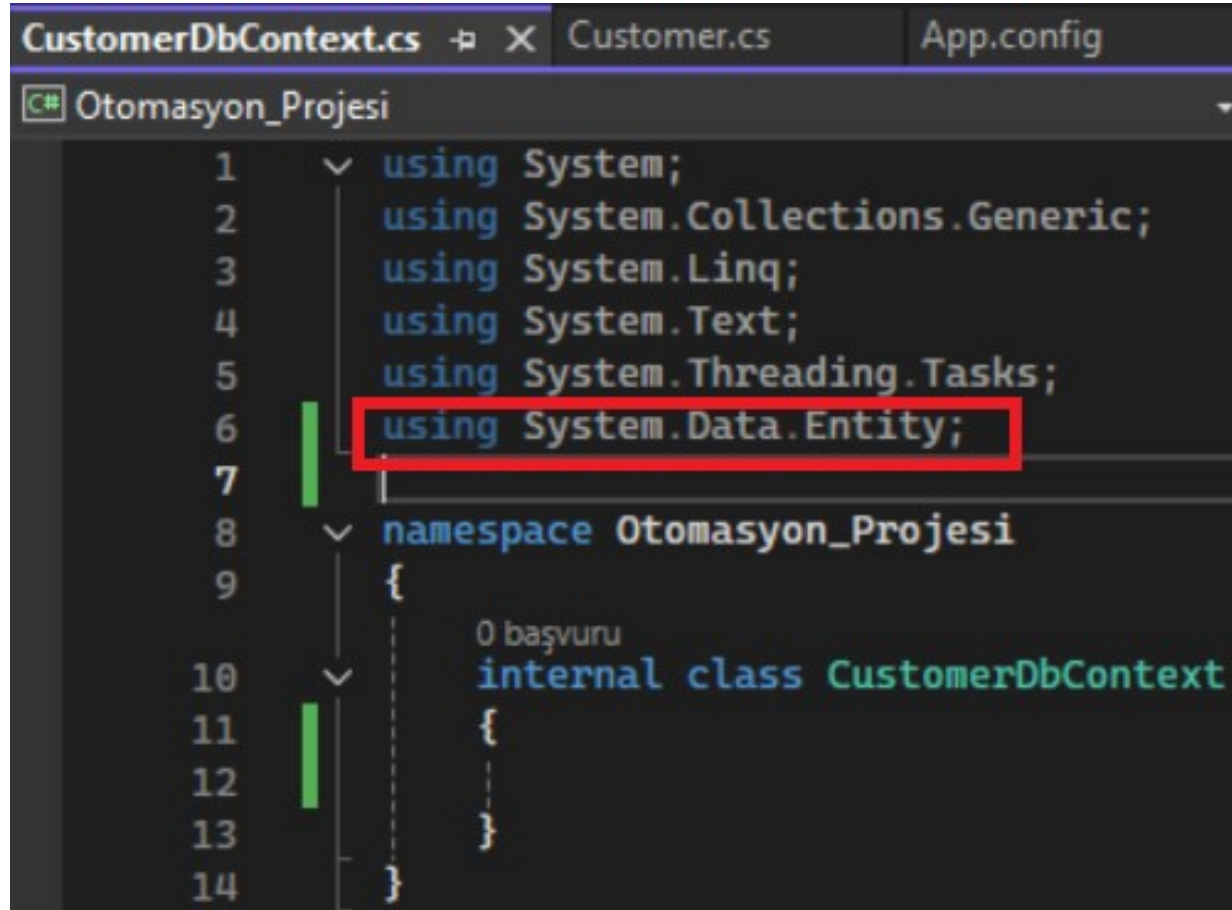
Şimdi biz de bu tanımlamaya sadık kalıp CustomerDbContext isimli bir class tanımlıyoruz.

Bunun için az önceki gibi Projemize sağ tıklayıp Ekle->Sınıf seçeneğini seçip CustomerDbContext adını veriyoruz.

Veri Tabanı İşlemleri için Sınıf Tanımlamak

CustomerDbContext sınıfımızı oluşturduktan sonra using System blokları arasına aşağıdaki kodu ekliyoruz:

```
using System.Data.Entity;
```



```
CustomerDbContext.cs X Customer.cs App.config
C# Otomasyon_Projesi
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6  using System.Data.Entity;
7
8  namespace Otomasyon_Projesi
9  {
10     0 başvuru
11     internal class CustomerDbContext
12     {
13     }
14 }
```

Veri Tabanı İşlemleri için Sınıf Tanımlamak

Şimdi, ilk olarak kendi CustomerDbContext sınıfımızın , EntityFramework konsepti içerisinde tanımlanmış ve veri tabanı işlemleri için hazır metotlar barındıran DbContext sınıfından miras almasını sağlayacağız.

Bunun için, sınıfımızı ilk oluşturduğumuzda hazır gelen internal class CustomerDbContext kısmında miras alma (inheritance) işlemini gerçekleştiriyoruz.

```
public class CustomerDbContext : DbContext
```

```
{
```

```
}
```

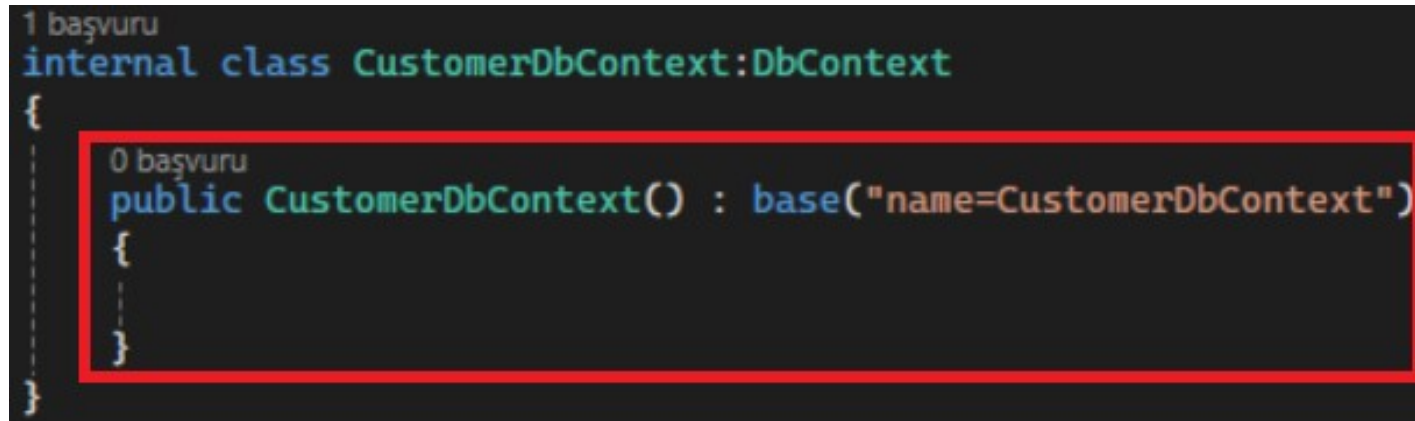
```
namespace Otomasyon_Projesi
{
    0 başvuru
    internal class CustomerDbContext : DbContext
    {
        ...
    }
}
```

Veri Tabanı İşlemleri için Sınıf Tanımlamak

Daha sonra CustomerDbContext sınıfımız için bir constructor (yapıcı) metot tanımlıyoruz ve bu metodun her çalıştığında yani bu sınıftan her instance (örnek) türetildiğinde veri tabanı bağlantısını hatırlaması için `base(name="CustomerDbContext")` kodunu ekliyoruz. Sınıfımızın son hali aşağıdaki gibidir:

```
internal class CustomerDbContext:DbContext
```

```
{  
    public CustomerDbContext() : base("name=CustomerDbContext")  
    {  
  
    }  
}
```



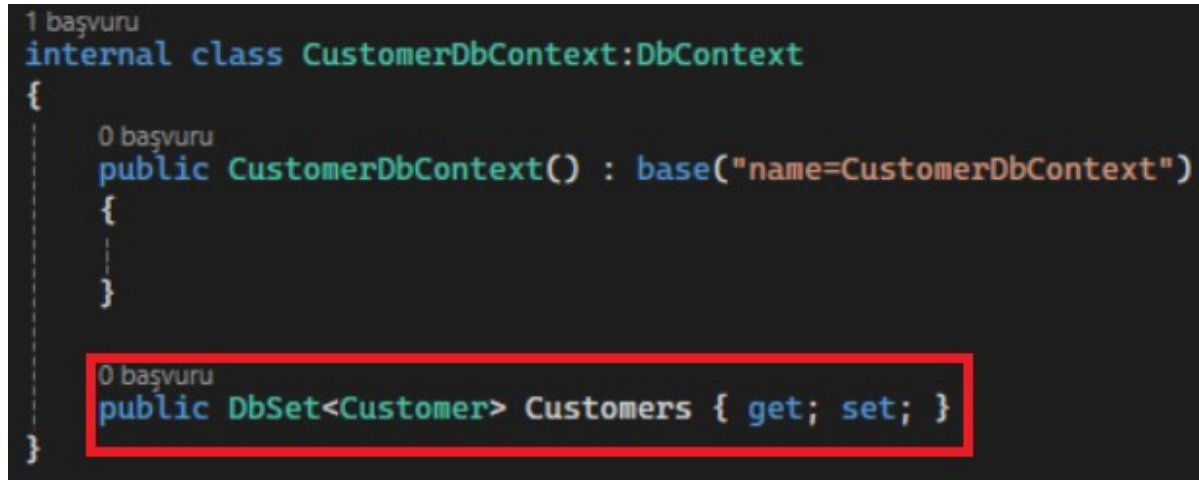
The screenshot shows the same C# code as the previous block, but with a red rectangular box highlighting the constructor method. The code is as follows:

```
1 başvuru  
internal class CustomerDbContext:DbContext  
{  
    0 başvuru  
    public CustomerDbContext() : base("name=CustomerDbContext")  
    {  
    }  
}
```

Veri Tabanı İşlemleri için Sınıf Tanımlamak

Şimdi de EntityFramework, bizim sınıfımızı bir tablo olarak algılayıp veri tabanı işlemleri için onu bir tablo olarak görsün diye `public DbSet<Customer> Customers { get; set; }` kodunu ekliyoruz. CustomerDbContext sınıfının son kodları aşağıdaki gibidir:

```
internal class CustomerDbContext:DbContext
{
    public CustomerDbContext() : base("name=CustomerDbContext")
    {
    }
    public DbSet<Customer> Customers { get; set; }
}
```



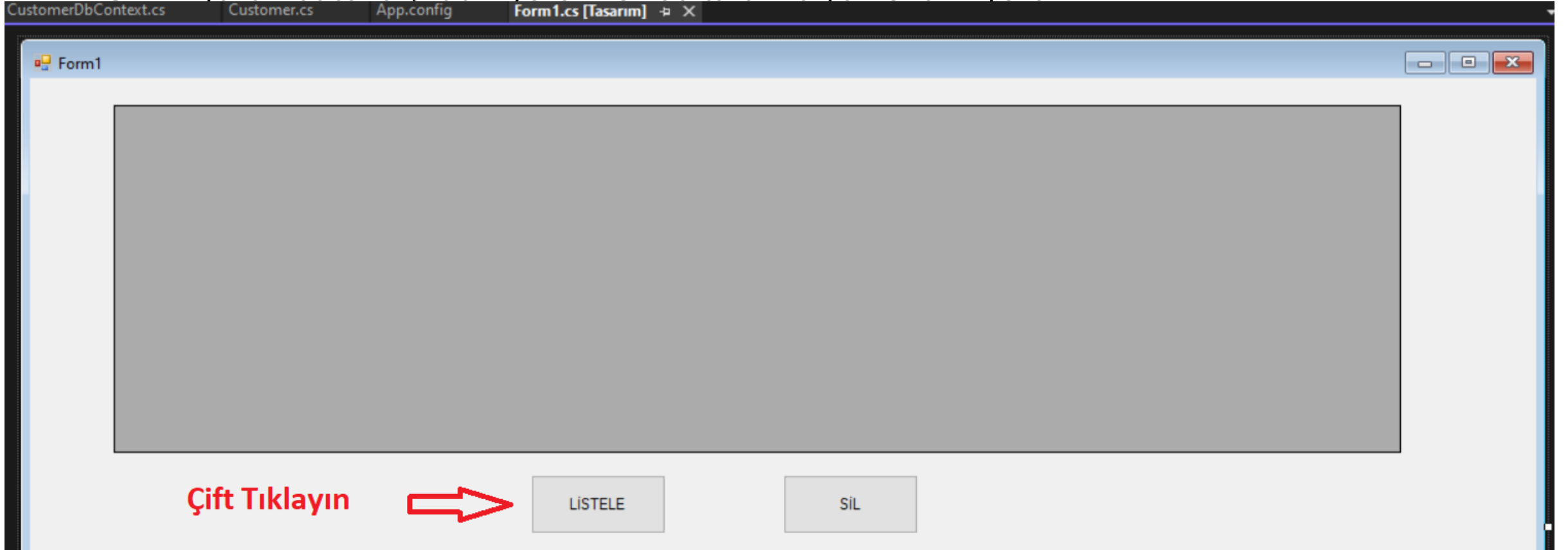
```
1 başvuru
internal class CustomerDbContext:DbContext
{
    0 başvuru
    public CustomerDbContext() : base("name=CustomerDbContext")
    {
    }
    0 başvuru
    public DbSet<Customer> Customers { get; set; }
}
```

The screenshot shows the same C# code as the previous block, but with line numbers (1 başvuru, 0 başvuru) and a red rectangular box highlighting the `public DbSet<Customer> Customers { get; set; }` line.

Veri Tabanı İşlemleri için Sınıf Tanımlamak

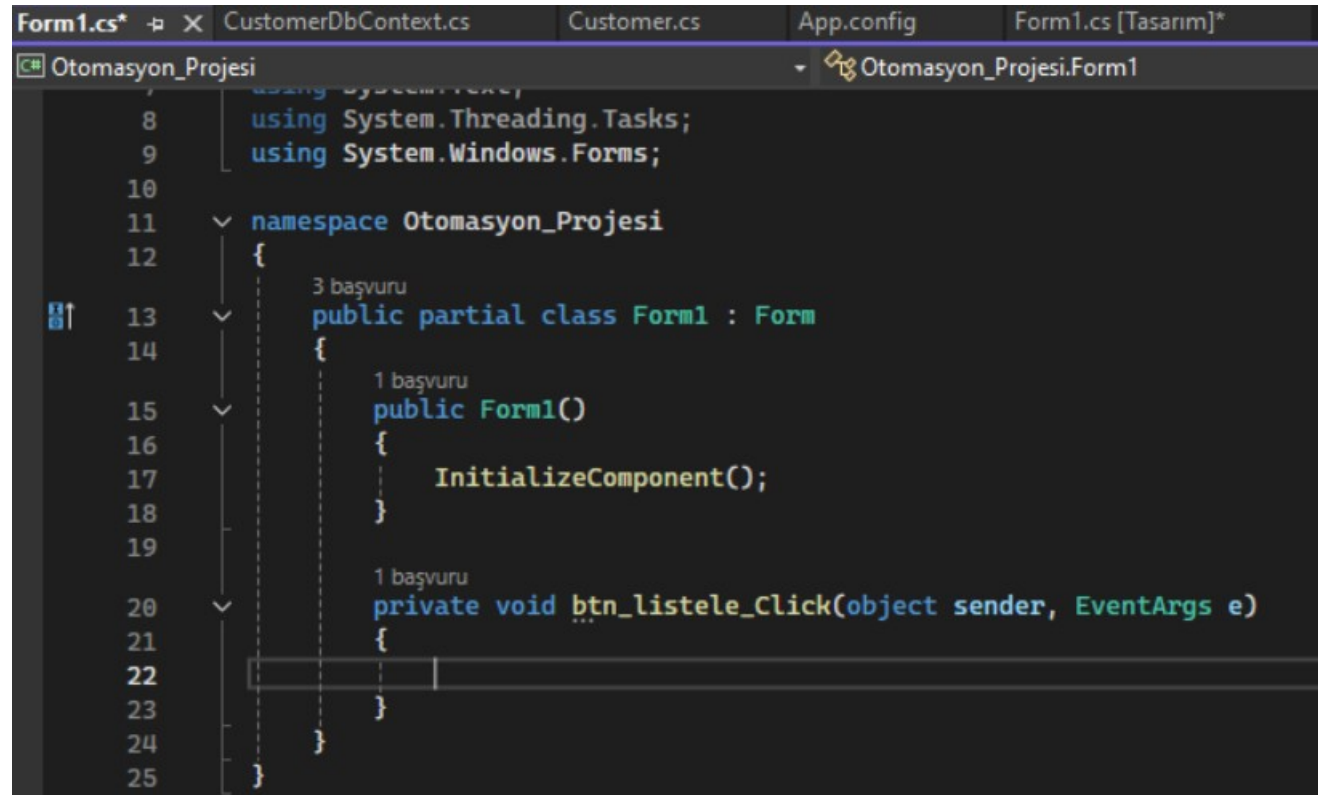
Artık veri tabanındaki tablomuz ve bu tabloyla yapacağımız işlemler için tanıma kısmını bitirdik. Şimdi artık listeleme, ekleme, silme ve güncelleme işlemleri için kodlarımızı ilgili butonlara yazabiliriz.

Yani, üzerinde EKLE yazan butona tıklandığında ilgili textbox'lardaki verileri ekleyen ya da diğer işlemleri yapan kodları ilgili butonlara tanıtacağız. Bunun için Windows form kısmına gelip LİSTELE yazan butona çift tıklıyoruz ve kod tarafına yönlendiriliyoruz



Listeleme İşlemi (Select operations)

Görüldüğü gibi butonumuzun id bilgisine btn_listele verdiğimiz için bunun click olayının kodları geldi. Şimdi, bu butona tıklandığında çalışmasını istediğimiz kodları bu click olayının içerisine (süslü parantezler içerisine) yazacağız.



```
Form1.cs*  X CustomerDbContext.cs Customer.cs App.config Form1.cs [Tasarım]*
[C#] Otomasyon_Projesi Otomasyon_Projesi.Form1
7  using System.Linq;
8  using System.Threading.Tasks;
9  using System.Windows.Forms;
10
11  namespace Otomasyon_Projesi
12  {
13      3 başvuru
14      public partial class Form1 : Form
15      {
16          1 başvuru
17          public Form1()
18          {
19              InitializeComponent();
20          }
21          1 başvuru
22          private void btn_listele_Click(object sender, EventArgs e)
23          {
24          }
25      }
26  }
```


Listeleme İşlemi (Select operations)

Öncelikle, CustomerDbContext sınıfımızdan bir instance(örnek) üretmemiz lazım. Her işlemde (listeleme, ekleme, silme ve güncelleme) bu türettiğimiz örneğe ihtiyacımız olacağı için public alanda bu tanımlamayı yapacağız. Aşağıdaki kodu public alana yapııştırıyoruz:

CustomerDbContext db = new CustomerDbContext();

```
namespace Otomasyon_Projesi
{
    3 başvuru
    public partial class Form1 : Form
    {
        1 başvuru
        public Form1()
        {
            InitializeComponent();
        }

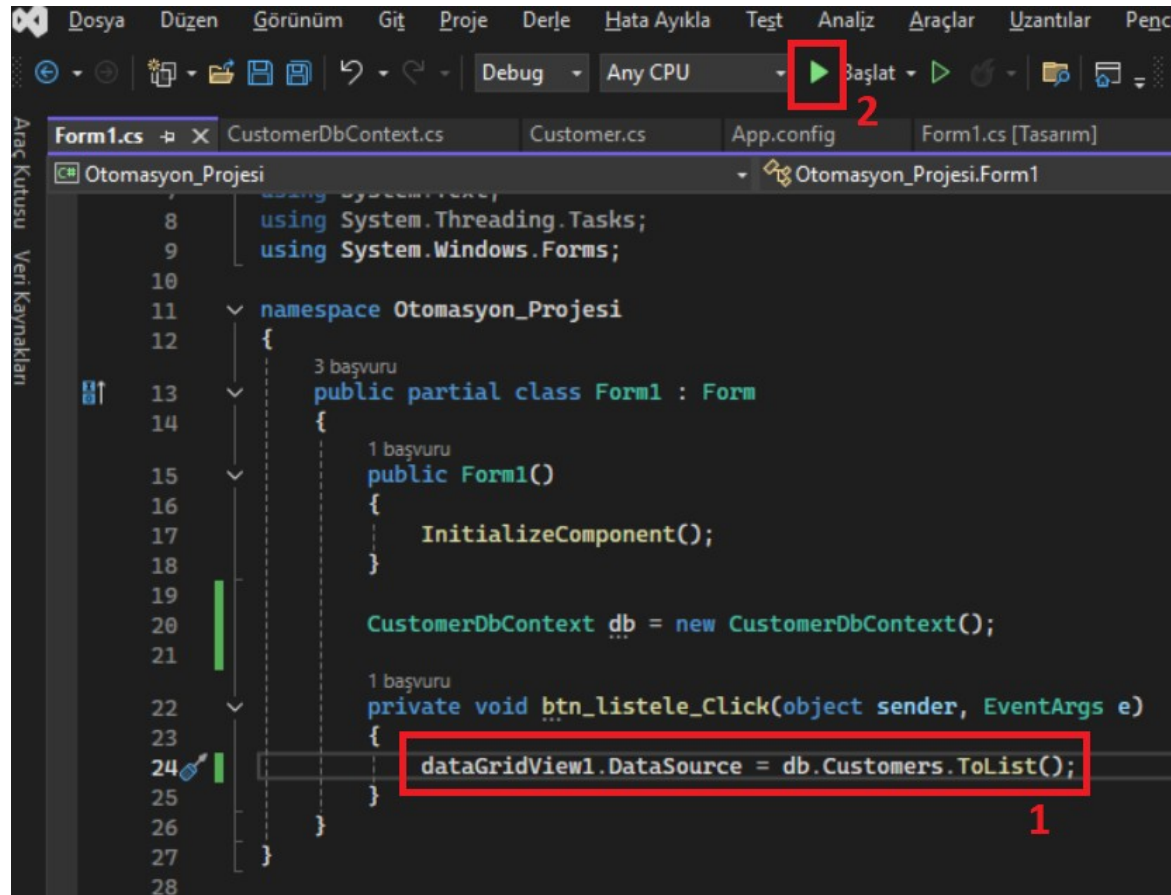
        CustomerDbContext db = new CustomerDbContext();

        1 başvuru
        private void btn_listele_Click(object sender, EventArgs e)
        {
        }
    }
}
```

Listeleme İşlemi (Select operations)

LİSTELE yazan butona tıklandığında, Table_Customer tablosundaki verilerin getirilmesini ve datagridview'da gösterilmesi için datagridview'ın DataSource özelliğinin neye göre değişeceğini belirtmemiz gerek. Aşağıdaki kodla listeleme işlemini yapıyoruz ve Start butonuna tıklayıp projemizi çalıştırıyoruz.

```
dataGridView1.DataSource = db.Customers.ToList();
```



Listeleme İşlemi (Select operations)

Şekilde görüldüğü gibi LİSTELE butonuna tıkladığımızda, Table_Customer tablosundaki verileri getirdi

Form1

	Customer_Id	Customer_Name	Customer_Surname	Customer_email	Customer_telephon
►	1	Furkan	ATLAN	furkanatlan@msn...	5329864512
	2	Ahmet	TEKOK	ahmetekok@yah...	5308536214
	3	Mehmet	VEYSEL	mveysel@gmail.c...	5391397241
	4	Enes	GÜMÜŞ	enesg@hotmail.c...	5063021086
	5	Leyla	SARDAL	lsardal@gmail.com	5024568239
	6	Tuğçe	HAYAT	tugcehayat@msn...	5389756432
	7	Elif	ŞEREFOĞLU	eserefoglu@gmai...	5063217847
	8	Tuğba	İZCİ	tubaizci@msn.com	5387962154

LİSTELE

SİL

DÜZELTMELER

Şu an için ilk işlemimizi (listeleme) gerçekleştirdik. Ancak, birkaç hususun düzeltilmesinde fayda var. Öncelikle listeleme işleminde kullandığımız kodu hatırlayalım:

```
dataGridView1.DataSource = db.Customers.ToList();
```

hatırlayalım: Buradaki db nesnesini CustomerDbContext sınıfından türetmiştik. CustomerDbContext sınıfı da bizim Customer sınıfımız (SQL'deki Table_Customer tablosuna karşılık gelen sınıfımız) için temel veri tabanı işlemlerini barındıran listeleme (select), ekleme (insert into), güncelleme (update) ve silme (delete) işlemlerini hazır bir şekilde içeren ve EntityFramework'ün kapsamında yer alan DbContext sınıfından türetilmişti.

Bu nedenle db nesnesinin altından kendi Customer sınıfımıza erişebildik. Customers sınıfımız bir nesne olarak değerlendirildiği için bu nesne altındaki hazır metotlarla listeleme, ekleme, güncelleme ve silme işlemlerini gerçekleştirebiliyoruz.

Örneğin, listeleme işlemi için db.Customers'in altındaki ToList() metodu, verilerimizi listelemeye yarıyor.

DÜZELTMELER

Düzeltilmesi gereken ilk husus, özellikle veri tabanı işlemleri içeren kodlar yazarken try-catch bloklarını kullanmamız gerekir. Eğer bu blokları kullanmazsak ve bir hata alırsak, programdan (otomasyondan) hemen çıkar. Bu durum hiç de istenilen bir durum değildir. Bir hatayla karşılaşıldığı anda yine programımızın açık kalmasını istiyorsak, çalışmasını istediğimiz kodları try blokları içerisine yazıp, catch blokları içerisine de bir hata tanımlarız.

Bir hata ile karşılaşıldığında program kapanmaz ve sadece hata mesajımızı yazdırır.

Şimdi, btn_listele kodlarına gidip, kodumuzu try-catch blokları arasına alalım.

DÜZELTMELER (try-catch)

btn_listele kodlarımızın son hali aşağıdaki gibi olmalıdır:

```
private void btn_listele_Click(object sender, EventArgs e)
{
    try
    {
        dataGridView1.DataSource = db.Customers.ToList();
    }
    catch (Exception ex)
    {

        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

NOT: try yazıp iki defa TAB tuşuna üst üste basarsanız. Boş bir try-catch bloku açılır ve siz de onu kendi ihtiyacınıza göre doldurursunuz.

DÜZELTMELER (try-catch)

NOT: Bundan sonra, her işlemimizi try-catch blokları içerisinde gerçekleştireceğiz

```
private void btn_listele_Click(object sender, EventArgs e)
{
    try
    {
        dataGridView1.DataSource = db.Customers.ToList();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

DÜZELTMELER (SÜTUN İSİMLERİ)

Şimdi kodumuzu tekrar çalıştıralım. Dikkat edecek olursanız, veriler datagridview'a yansıtıldığında 3 önemli sorun var:

Form1

	Customer_Id	Customer_Name	Customer_Surname	Customer_email	Customer_telephon
▶	1	Furkan	ATLAN	furkanatlan@msn...	5329864512
	2	Ahmet	TEKOK	ahmetekok@yah...	5308536214
	3	Mehmet	VEYSEL	mveysel@gmail.c...	5391397241
	4	Enes	GÜMÜŞ	enesg@hotmail.c...	5063021086
	5	Leyla	SARDAL	lsardal@gmail.com	5024568239
	6	Tuğçe	HAYAT	tugcehayat@msn...	5389756432
	7	Elif	ŞEREFOĞLU	eserefoglu@gmai...	5063217847
	8	Tuğba	İZCİ	tubaizci@msn.com	5387962154

LİSTELE

SİL

DÜZELTMELER (SÜTUN İSİMLERİ)

Böyle bir listeleme işlemi oldukça sıkıntılıdır. Çünkü, SQL tarafındaki orijinal sütun isimleri görülebiliyor. Bu durum bir güvenlik sorunudur. Şimdi ilk olarak, veriler listelendiğinde orijinal sütun isimleriyle gelmesin. Bunun yerine, bizim belirlediğimiz-istediğimiz sütun isimleriyle gelsin.

C#'ta datagridview'daki bir sütunun başlığını HeaderText özelliğiyle düzenleriz. Elimizdeki orijinal sütun isimlerine erişim, her birinin HeaderText özelliğine kendi istediğimiz ismi vereceğiz.

Buna göre, btn_listele'nin son kodlarının tamamı sonraki sayfada yer almaktadır.

DÜZELTMELER (SÜTUN İSİMLERİ)

```
private void btn_listele_Click(object sender, EventArgs e)
{
    try
    {
        var customers = db.Customers.ToList();
        dataGridView1.DataSource = customers;

        // Sütun başlıklarını değiştirme
        dataGridView1.Columns["Customer_Id"].HeaderText = "Müşteri ID";
        dataGridView1.Columns["Customer_Name"].HeaderText = "Müşteri Adı";
        dataGridView1.Columns["Customer_Surname"].HeaderText = "Müşteri Soyadı";
        dataGridView1.Columns["Customer_Email"].HeaderText = "E-Posta";
        dataGridView1.Columns["Customer_Telephone"].HeaderText = "Telefon";
    }
    catch (Exception ex)
    {

        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

DÜZELTMELER (SÜTUN İSİMLERİ)

```
private void btn_listele_Click(object sender, EventArgs e)
{
    try
    {
        var customers = db.Customers.ToList();
        dataGridView1.DataSource = customers;

        // Sütun başlıklarını değiştirme
        dataGridView1.Columns["Customer_Id"].HeaderText = "Müşteri ID";
        dataGridView1.Columns["Customer_Name"].HeaderText = "Müşteri Adı";
        dataGridView1.Columns["Customer_Surname"].HeaderText = "Müşteri Soyadı";
        dataGridView1.Columns["Customer_Email"].HeaderText = "E-Posta";
        dataGridView1.Columns["Customer_Telephone"].HeaderText = "Telefon";
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

DÜZELTMELER (SÜTUN İSİMLERİ)

Şimdi tekrar kodumuzu çalıştırıyoruz. Görüldüğü gibi sütun isimleri istediğimiz-bizim belirlediğimiz isimlerle geldi.

Form1

	Müşteri ID	Müşteri Adı	Müşteri Soyadı	E-Posta	Telefon
▶	1	Furkan	ATLAN	furkanatlan@msn...	5329864512
	2	Ahmet	TEKOK	ahmetekok@yah...	5308536214
	3	Mehmet	VEYSEL	mveysel@gmail.c...	5391397241
	4	Enes	GÜMÜŞ	enesg@hotmail.c...	5063021086
	5	Leyla	SARDAL	lsardal@gmail.com	5024568239
	6	Tuğçe	HAYAT	tugcehayat@msn...	5389756432
	7	Elif	ŞEREFOĞLU	eserefoglu@gmai...	5063217847
	8	Tuğba	İZCİ	tubaizci@msn.com	5387962154

LİSTELE

SİL

DÜZELTMELER (SÜTUN GİZLEME)

Bir sorunu hallettik. Şimdi de, sıra geldi başka bir sorunu halletmeye. Table_Customer tablosunun ilk sütunu ve primary key olan Customer_Id sütununun adını Müşteri_ID olarak değiştirdik ama bu sütunu kullanıcının görmesine gerek yok. Bu hem gereksiz hem de güvenlik sorunu içeren bir durumdur.

C# Windows form'da datagridview'daki bir sütunun görünür olup olmaması, o sütunun Visible özelliği tarafından belirlenir. Bu özellik true ve false olmak üzere iki değer alır.

Şimdi biz bu özelliği false yaparak gizlenmesini sağlayacağız.

Bunun için mevcut kodlarımıza aşağıdaki kodu ekliyoruz:

```
dataGridView1.Columns["Customer_Id"].Visible = false;
```

DÜZELTMELER (SÜTUN GİZLEME)

```
private void btn_listele_Click(object sender, EventArgs e)
{
    try
    {
        var customers = db.Customers.ToList();
        dataGridView1.DataSource = customers;

        // Sütun başlıklarını değiştirme
        dataGridView1.Columns["Customer_Id"].HeaderText = "Müşteri ID";
        dataGridView1.Columns["Customer_Name"].HeaderText = "Müşteri Adı";
        dataGridView1.Columns["Customer_Surname"].HeaderText = "Müşteri Soyadı";
        dataGridView1.Columns["Customer_Email"].HeaderText = "E-Posta";
        dataGridView1.Columns["Customer_Telephone"].HeaderText = "Telefon";

        // Customer_Id (Müşteri_ID) sütununu gizliyoruz
        dataGridView1.Columns["Customer_Id"].Visible = false;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

DÜZELTMELER (SÜTUN GİZLEME)

Şimdi, programımızı tekrar çalıştırıyoruz. Görüldüğü gibi Customer_Id (MÜŞTERİ_ID) sütunu artık görünmüyor.

Form1

Müşteri Adı	Müşteri Soyadı	E-Posta	Telefon
Furkan	ATLAN	furkanatlan@msn...	5329864512
Ahmet	TEKOK	ahmetekok@yah...	5308536214
Mehmet	VEYSEL	mveysel@gmail.c...	5391397241
Enes	GÜMÜŞ	enesg@hotmail.c...	5063021086
Leyla	SARDAL	lsardal@gmail.com	5024568239
Tuğçe	HAYAT	tugcehayat@msn...	5389756432
Elif	ŞEREFOĞLU	eserefoglu@gmai...	5063217847
Tuğba	İZCİ	tubaizci@msn.com	5387962154

LISTELE SİL

DÜZELTMELER (SÜTUNLARI GENİŞLETME)

Şimdilik son bir sorunumuz kaldı. Dikkat edilecek olursa datagridview'daki sütun isimleri ve onlara ait değerler geldiğinde datagridview'ın tamamını kaplamıyor ve bu da kötü bir görünüme neden oluyor.

Şimdi de bu sorunu düzelteceğiz. Datagridview'daki tüm sütunların tüm datagridview'ı kaplaması ve boş alan kalmaması için aşağıdaki kodu btn_listele'nin içine ekliyoruz:

```
dataGridView1.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
```

Son durumda btn_listele'nin tüm kodları sonraki sayfada yer almaktadır.

DÜZELTMELER (SÜTUNLARI GENİŞLETME)

```
private void btn_listele_Click(object sender, EventArgs e)
{
    try
    {
        var customers = db.Customers.ToList();
        dataGridView1.DataSource = customers;

        // Sütun başlıklarını değiştirme
        dataGridView1.Columns["Customer_Id"].HeaderText = "Müşteri ID";
        dataGridView1.Columns["Customer_Name"].HeaderText = "Müşteri Adı";
        dataGridView1.Columns["Customer_Surname"].HeaderText = "Müşteri Soyadı";
        dataGridView1.Columns["Customer_Email"].HeaderText = "E-Posta";
        dataGridView1.Columns["Customer_Telephone"].HeaderText = "Telefon";

        // Customer_Id (Müşteri_ID) sütununu gizliyoruz
        dataGridView1.Columns["Customer_Id"].Visible = false;

        // Sütun genişliklerini dolduracak şekilde ayarlama
        dataGridView1.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

DÜZELTMELER (SÜTUNLARI GENİŞLETME)

Programımızı tekrar çalıştırdığımızda sütunların tüm datagridview'i kapladığını görüyoruz.

Form1

Müşteri Adı	Müşteri Soyadı	E-Posta	Telefon
Furkan	ATLAN	furkanatlan@msn.com	5329864512
Ahmet	TEKOK	ahmetekok@yahoo.com	5308536214
Mehmet	VEYSEL	mveysel@gmail.com	5391397241
Enes	GÜMÜŞ	enesg@hotmail.com	5063021086
Leyla	SARDAL	lsardal@gmail.com	5024568239
Tuğçe	HAYAT	tugcehayat@msn.com	5389756432
Elif	ŞEREFOĞLU	eserefoglu@gmail.com	5063217847
Tuğba	İZCİ	tubaizci@msn.com	5387962154

LİSTELE

SİL

EKLEME İŞLEMİ (Insert Into-Add Operations)

Şimdi de form'da yer alan EKLE yazan butona çift tıklıyoruz. İlk olarak try-catch bloklarını açıp catch blokunu hata mesajını verecek şekilde düzenliyoruz. Buna göre btn_ekle'nin kodları aşağıdaki gibi olacaktır:

```
private void btn_ekle_Click(object sender, EventArgs e)
{
    try
    {

    }
    catch (Exception ex)
    {

        MessageBox.Show($"Hata = {ex.Message}");

    }
}
```

EKLEME İŞLEMİ (Insert Into-Add Operations)

İlk olarak, Customer sınıfından yeni bir instance üreteceğiz. Customer sınıfı, SQL'deki Table_Customer tablosuna karşılık geliyordu. Nasıl ki Table_Customer'ın 4 adet sütunu varsa, aynı şekilde C# tarafında bu 4 sütuna karşılık gelen 4 adet özellik (property) tanımlamıştık.

Dolayısıyla yeni bir müşteri ekleyeceğimiz zaman Customer sınıfından yeni bir instance türetmemiz lazım.

Bu örneği ilk türettiğimizde 4 adet veriyi de o anda dolduracağız. Yani bu müşterinin isim bilgisini txt_isim'den çekeceğiz ve diğer özelliklerini de diğer textbox'lardan çekeceğiz.

İlk olarak Customer sınıfından bir instance üretilip ilk aşamada (yapılandırıcı metot çalıştığında) ilgili textbox'lardaki değerleri bu yeni instance'ye atayacağız. Kodlar sonraki sayfada yer almaktadır.

EKLEME İŞLEMİ (Insert Into-Add Operations)

```
Customer newCustomer = new Customer()  
{  
    Customer_Name = txtName.Text,  
    Customer_Surname = txtSurname.Text,  
    Customer_email = txtEmail.Text,  
    Customer_telephone = txtPhone.Text  
};
```

```
private void btn_ekle_Click(object sender, EventArgs e)  
{  
    try  
    {  
        Customer newCustomer = new Customer()  
        {  
            Customer_Name = txt_isim.Text,  
            Customer_Surname = txt_soyisim.Text,  
            Customer_email = txt_email.Text,  
            Customer_telephone = txt_telefon.Text  
        };  
    }  
    catch (Exception ex)  
    {  
        MessageBox.Show($"Hata = {ex.Message}");  
    }  
}
```

EKLEME İŞLEMİ (Insert Into-Add Operations)

Şimdi de, değerleri atadıktan sonra bu değerlere sahip olan yeni müşteriyi ekliyoruz. Bunun için aşağıdaki kodları yazıyoruz:

```
db.Customers.Add(newCustomer);
```

```
db.SaveChanges();
```

Listeleme işlemi için db.Customers'in altındaki ToList() metodunu çalıştırmıştık. Veri eklemek için ise db.Customers'in altındaki Add metodunu çalıştırıyoruz. Bu metod, parametre aldığı için yeni eklenecek müşteriyi de az önce oluşturduğumuz ve ilgili textbox'lardaki değerleri atadığımız nesne olarak belirliyoruz.

db.SaveChanges(); metodu ile de değişiklikleri uyguluyoruz.

Buna göre btn_ekle'nin son kodları sonraki sayfadaki gibi olmalıdır.

EKLEME İŞLEMİ (Insert Into-Add Operations)

```
private void btn_ekle_Click(object sender, EventArgs e)
{
    try
    {
        Customer newCustomer = new Customer()
        {
            Customer_Name = txt_isim.Text,
            Customer_Surname = txt_soyisim.Text,
            Customer_email = txt_email.Text,
            Customer_telephone = txt_telefon.Text
        };

        db.Customers.Add(newCustomer);
        db.SaveChanges();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

EKLEME İŞLEMİ (Insert Into-Add Operations)

```
private void btn_ekle_Click(object sender, EventArgs e)
{
    try
    {
        Customer newCustomer = new Customer()
        {
            Customer_Name = txt_isim.Text,
            Customer_Surname = txt_soyisim.Text,
            Customer_email = txt_email.Text,
            Customer_telephone = txt_telefon.Text
        };

        db.Customers.Add(newCustomer);
        db.SaveChanges();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```


EKLEME İŞLEMİ (Insert Into-Add Operations)

Şimdi de eğer ekleme işlemi başarılı bir şekilde gerçekleşirse kullanıcıya bir uyarı versin diye MessageBox ile ekrana bir uyarı yazdırıyoruz. Bunun için de aşağıdaki kodu yazıyoruz:

```
MessageBox.Show("Yeni Müşteri Eklendi!");
```

Son olarak, ekleme işlemi gerçekleştikten sonra tablonun son halini versin diye, yani son eklenen müşteri de dahil tüm müşterileri listelesin diye aşağıdaki kodu yazıyoruz:

```
btn_listele.PerformClick();
```

Hatırlarsanız, listeleme işlemi için btn_listele'yi düzenlemiştik. Dolayısıyla, ekleme işleminden sonra tekrardan listeleme kodları yazmak yerine bu kodları içeren butonun PerformClick metodunu çağırıyoruz.

Son eklenen kodlardan sonra btn_ekle'nin tüm kodları sonraki sayfada yer almaktadır.

EKLEME İŞLEMİ (Insert Into-Add Operations)

```
private void btn_ekle_Click(object sender, EventArgs e)
{
    try
    {
        Customer newCustomer = new Customer()
        {
            Customer_Name = txt_isim.Text,
            Customer_Surname = txt_soyisim.Text,
            Customer_email = txt_email.Text,
            Customer_telephone = txt_telefon.Text
        };

        db.Customers.Add(newCustomer);
        db.SaveChanges();

        MessageBox.Show("Yeni Müşteri Eklendi!");
        btn_listele.PerformClick();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

EKLEME İŞLEMİ (Insert Into-Add Operations)

```
private void btn_ekle_Click(object sender, EventArgs e)
{
    try
    {
        Customer newCustomer = new Customer()
        {
            Customer_Name = txt_isim.Text,
            Customer_Surname = txt_soyisim.Text,
            Customer_email = txt_email.Text,
            Customer_telephone = txt_telefon.Text
        };

        db.Customers.Add(newCustomer);
        db.SaveChanges();

        MessageBox.Show("Yeni Müşteri Eklendi!");
        btn_listele.PerformClick();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

EKLEME İŞLEMİ (Insert Into-Add Operations)

Tüm bu işlemlerden sonra programımızı çalıştırıp önce LİSTELE butonuna tıklayarak mevcut müşterileri görüntülüyoruz. Görüldüğü gibi son müşterinin adı Tuğba.

Form1

Müşteri Adı	Müşteri Soyadı	E-Posta	Telefon
Furkan	ATLAN	furkanatlan@msn.com	5329864512
Ahmet	TEKOK	ahmetekok@yahoo.com	5308536214
Mehmet	VEYSEL	mveysel@gmail.com	5391397241
Enes	GÜMÜŞ	enesg@hotmail.com	5063021086
Leyla	SARDAL	lsardal@gmail.com	5024568239
Tuğçe	HAYAT	tugcehayat@msn.com	5389756432
Elif	ŞEREFOĞLU	eserefoglu@gmail.com	5063217847
Tuğba	İZCİ	tubaizci@msn.com	5387962154

LİSTELE

SİL

EKLEME İŞLEMİ (Insert Into-Add Operations)

Şimdi de, veri ekleme alanına görseldeki değerleri giriyoruz:

Elif	ŞEREFOĞLU	eserefoglu@gmail.com	5063217847
Tuğba	İZCİ	tubaizci@msn.com	5387962154

LİSTELE

SİL

groupBox1

Ad:

Süleyman

Soyad:

ASLAN

E-Mail:

saslan@msn.com

Telefon:

5458563214

EKLE

GÜNCELLE

EKLEME İŞLEMİ (Insert Into-Add Operations)

EKLE butonuna tıkladıktan sonra önce bize mesajla uyarı verecek.

Müşteri Adı	Müşteri Soyadı	E-Posta	Telefon
Furkan	ATLAN	furkanatlan@msn.com	5329864512
Ahmet	TEKOK	ahmetekok@yahoo.com	5308536214
Mehmet	VEYSEL	mveysel@gmail.com	5391397241
Enes	GÜMÜŞ	enesg@hotmail.com	5063021086
Leyla	SARDAL	lsardal@gmail.com	5024568239
Tuğçe	HAYAT	tugcehayat@msn.com	5389756432
Elif	ŞEREFOĞLU	eserefoglu@gmail.com	5063217847
Tuğba	İZCİ	tubaizci@msn.com	5387962154

LİSTELE

SİL

×

Yeni Müşteri Eklendi!

Tamam

groupBox1

Ad:

Süleyman

Soyad:

ASLAN

E-Mail:

saslan@msn.com

Telefon:

5458563214

EKLE

GÜNCELLE

EKLEME İŞLEMİ (Insert Into-Add Operations)

Uyarıyı kapattıktan sonra datagridview için listeleme işlemi tekrar yapılacak ve son müşteri de getirilecek.

	Müşteri Adı	Müşteri Soyadı	E-Posta	Telefon
▶	Furkan	ATLAN	furkanatlan@msn.com	5329864512
	Ahmet	TEKOK	ahmetekok@yahoo.com	5308536214
	Mehmet	VEYSEL	mveyssel@gmail.com	5391397241
	Enes	GÜMÜŞ	enesg@hotmail.com	5063021086
	Leyla	SARDAL	lsardal@gmail.com	5024568239
	Tuğçe	HAYAT	tugcehayat@msn.com	5389756432
	Elif	ŞEREFOĞLU	eserefoglu@gmail.com	5063217847
	Tuğba	İZCİ	tubaizci@msn.com	5387962154
	Süleyman	ASLAN	saslan@msn.com	5458563214

LİSTELE

SİL

groupBox1

Ad: Süleyman

Soyad: ASLAN

E-Mail: saslan@msn.com

Telefon: 5458563214

EKLE

GÜNCELLE

GÜNCELLEME İŞLEMİ (Update Operations)

4 temel işlemten ilk ikisini gerçekleştirdik. Şimdi de sıra geldi. Güncelleme işlemine.

Güncelleme işlemi için öncelikle güncellenecek müşteriyi seçmemiz lazım. Bunun için de datagridview'daki ilgili müşterinin en solundaki boş hücreye tıkladığımızda tüm satırı seçer. Dolayısıyla ilk olarak güncelleyeceğimiz müşteriyi seçeceğiz sonra da o hücredeki her bir değer ilgili textbox'a aktarılmasını sağlayacağız.

Örneğin, bir müşteriyi seçtikten sonra o müşterinin isim bilgisi txt_isim textbox'ına aktarılacak. Aynı şekilde diğer sütunlardaki bilgileri de ilgili textbox'lara aktarılacak. Dolayısıyla ilk olarak müşteri seçme işlemini gerçekleştireceğiz.

Bu işlemi datagridview'in event'ları (events) ile halledeceğiz. Yani, «datagridview'in bir hücreğine tıklandığında» olayını düzenlememiz lazım.

GÜNCELLEME İŞLEMİ (Update Operations)

Form ekranına gelip datagridview nesnesini bir defa sol tıklayarak onu seçiyoruz. Daha sonra sağ alt taraftaki Özellikler (Properties) menüsüne geliyoruz. Bu menüdeki şimşek simgesine tıklıyoruz. Şimşek simgesine tıkladıktan sonra «Fare» (Mouse) yazan yerdeki ilk seçenek olan CellClick yazan olayın hemen karşısındaki boşluğa tıklıyoruz.

GÜNCELLEME İŞLEMİ (Update Operations)

- 1) Datagridview'a bir defa sol tıklayınız.
- 2) sağ alt menüdeki şimşek simgesine tıklayınız
- 3) aşağı doğru inip Fare (Mouse) yazan yere geliniz
- 4) CellClick yazan yerin karşısındaki boş alana çift tıklayınız

LİSTELE

SİL

groupBox1

Ad:

Soyad:

Çözüm Gezgini

Çözüm Gezgini İçinde Ara (Ctrl+Ş)

'Otomasyon_Projesi' çözümü (proje 1)

- ▶ C# Otomasyon_Projesi
 - ▶ Properties
 - ▶ Başvurular
 - ▶ App.config
 - ▶ C# Customer.cs
 - ▶ C# CustomerDbContext.cs
 - ▶ Form1.cs
 - ▶ Form1.Designer.cs
 - ▶ Form1.resx
 - ▶ packages.config
 - ▶ C# Program.cs

Çözüm Gezgini Git Değişiklikleri

Özellikler

dataGridView1 System.Windows.Forms

UserDeletingRow

Fare

CellClick

CellContentClick

CellContentDoub

CellDoubleClick

Sütunları Düzenle... Sütun Ekle...

Çıktı

Şu çıktıyı göster: Hata Ayıkla

'Otomasyon_Projesi.exe' (CLR v4.0.30319: Otomasyon_Projesi.exe): 'EntityFrameworkDynamicProxies-EntityFramework' yüklendi.

GÜNCELLEME İŞLEMİ (Update Operations)

Karşınıza gelen CellClick olayını aşağıdaki gibi doldurunuz:

```
if (e.RowIndex >= 0)
{
    txt isim.Text =
dataGridView1.Rows[e.RowIndex].Cells["Customer_Name"].Value.ToString();
    txt sovisim.Text =
dataGridView1.Rows[e.RowIndex].Cells["Customer_Surname"].Value.ToString();
    txt email.Text =
dataGridView1.Rows[e.RowIndex].Cells["Customer_Email"].Value.ToString();
    txt telefon.Text =
dataGridView1.Rows[e.RowIndex].Cells["Customer_Telephone"].Value.ToString()
;
}
```

Burada, e harfi tıklanan hücreyi temsil eder. Buna göre tıklanma olayı gerçekleşmişse o satırdaki her bir sütunun değerini ilgili textbox bileşenine aktarır. Siz de kendi kullandığınız bileşenlere göre aktarma olayının kodlarını yazınız.

GÜNCELLEME İŞLEMİ (Update Operations)

```
private void dataGridView1_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        txt_isim.Text = dataGridView1.Rows[e.RowIndex].Cells["Customer_Name"].Value.ToString();
        txt_soyisim.Text = dataGridView1.Rows[e.RowIndex].Cells["Customer_Surname"].Value.ToString();
        txt_email.Text = dataGridView1.Rows[e.RowIndex].Cells["Customer_Email"].Value.ToString();
        txt_telefon.Text = dataGridView1.Rows[e.RowIndex].Cells["Customer_Telephone"].Value.ToString();
    }
}
```

GÜNCELLEME İŞLEMİ (Update Operations)

Görüldüğü gibi, kırmızı ile seçili alana tıkladığımızda her bir sütundaki bilgi, ilgili textbox'a aktarıldı-

	Müşteri Adı	Müşteri Soyadı	E-Posta	Telefon
	Furkan	ATLAN	furkanatlan@msn.com	5329864512
	Ahmet	TEKOK	ahmetekok@yahoo.com	5308536214
	Mehmet	VEYSEL	mveyssel@gmail.com	5391397241
	Enes	GÜMÜŞ	enesg@hotmail.com	5063021086
▶	Leyla	SARDAL	lsardal@gmail.com	5024568239
	Tuğçe	HAYAT	tugcehayat@msn.com	5389756432
	Elif	ŞEREFOĞLU	eserefoglu@gmail.com	5063217847
	Tuğba	İZCİ	tubaizci@msn.com	5387962154
	Süleyman	ASLAN	saslan@msn.com	5458563214

LISTELE

SİL

groupBox1

Ad:

Leyla

Soyad:

SARDAL

E-Mail:

lsardal@gmail.com

Telefon:

5024568239

EKLE

GÜNCELLE

GÜNCELLEME İŞLEMİ (Update Operations)

Böylelikle, güncelleme işleminin ilk ayağını tamamlamış olduk. Şimdi sıra geldi, textbox'lara aktarılan bilgileri istediğimiz gibi güncelledikten sonra GÜNCELLE butonuna tıkladığımızda bu güncellemenin gerçekleşmesine, değişikliklerin uygulanmasına.

Bunun için GÜNCELLE butonuna çift tıklayıp onun Click metodunu düzenleyeceğiz.

GÜNCELLEME İŞLEMİ (Update Operations)

Öncelikle, eğer güncelleme işlemi için datagridview'da bir satıra tıklanmışsa yani bir müşteri seçilmişse bunun kontrolünü yapmamız lazım ki güncelle butonu aktifleşsin. Eğer bir müşteri seçilmeden bu butona tıklanırsa bir olay gerçekleşmeyecek. Bunun için bir if kontrolü sağlıyoruz.

Tabi her şeyden önce try-catch blokları açıp tüm kodları try bloku içerisine yazıyoruz. Buna göre, kodlarımız aşağıdaki gibi olmalıdır:

```
try
{
    if (dataGridView1.CurrentRow != null)
    {

    }
    {

    }
}
catch (Exception ex)
{

    MessageBox.Show($"Hata = {ex.Message}");
}
```

GÜNCELLEME İŞLEMİ (Update Operations)

Eğer bir müşteri seçilmişse o zaman if bloku içerisine yazdığımız kodlar çalışacaktır. Hatırlarsanız, SQL'de update işlemi where şartıyla birlikte id sütununa göre yapılırdı.

Aksi halde aynı değere sahip olan tüm satırlar bu güncellemeden etkilenirdi. Bu nedenle biz de ilk olarak seçilen satırdaki müşterinin id bilgisini çekeceğiz. Bunun için aşağıdaki kodu yazıyoruz:

```
int selectedId = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Customer_Id"].Value);
```


GÜNCELLEME İŞLEMİ (Update Operations)

```
private void btn_guncelle_Click(object sender, EventArgs e)
{
    try
    {
        if (dataGridView1.CurrentRow != null)
        {
            int selectedId = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Customer_Id"].Value);
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

GÜNCELLEME İŞLEMİ (Update Operations)

Müşterinin id bilgisini çektikten sonra bu id bilgisine sahip müşteriye mevcut müşteriler arasında arıyoruz.

Bunun için de aşağıdaki kodu yazıyoruz. Hatırlarsanız bizim tüm müşteri bilgilerimiz CustomerDbContext sınıfından türettiğimiz ve global alanda tanımladığımız db nesnesinin içindeki Customers'in içerisindeydi.

Update işlemi için id araması yapan hazır bir metot var. Find metodu. Aşağıdaki kodla, id bilgisini çektiğimiz müşteriye mevcut müşteriler arasında arıyoruz:

```
Customer customer = db.Customers.Find(selectedId);
```

Güncelleme işlemi yapmamız için de Customer sınıfından bir instance üretiyoruz ki onun özelliklerini (Name, Surname, Email, Telephone) güncelleyebilelim.

Müşterinin özellikleri (nasıl bir müşteri olduğu bilgisi Customer sınıfında), Müşterinin kim olduğu ve neler yapabildiği ise CustomerDbContext sınıfında tanımlanmıştır. Bu iki sınıfı karıştırmayınız.

GÜNCELLEME İŞLEMİ (Update Operations)

```
private void btn_guncelle_Click(object sender, EventArgs e)
{
    try
    {
        if (dataGridView1.CurrentRow != null)
        {
            int selectedId = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Customer_Id"].Value);
            Customer customer = db.Customers.Find(selectedId);
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

GÜNCELLEME İŞLEMİ (Update Operations)

Şimdi, yeni bir if bloku açıyoruz. Eğer bulduğumuz-eşleştirdiğimiz kullanıcı doğruysa (mevcutsa) o zaman güncelleme işlemi için yeni değerlerin atamasını yapacağız. Öncelikle aşağıdaki if döngüsünü açmamız lazım:

```
if (customer != null)
{

}
```

GÜNCELLEME İŞLEMİ (Update Operations)

Seçilen müşteriye customer olarak tanımlamıştık. Şimdi, textbox'lardaki değerleri bu müşterinin ilgili özelliği olarak atayacağız. Örneğin, txt_name isimli textbox'ta müşterinin isim bilgisi (Customer_Name özelliği) yer alıyor. Dolayısıyla müşterinin adını değiştirmek istiyorsak, bu textbox'taki değeri değiştirmemiz yeterli olacak. Aşağıdaki kodları açtığımız ikinci if bloku içerisine yazıp her bir (yeni) değerin atamasını gerçekleştiriyoruz:

```
customer.Customer_Name = txt_isim.Text;  
customer.Customer_Surname = txt_soyisim.Text;  
customer.Customer_email = txt_email.Text;  
customer.Customer_telephone = txt_telefon.Text;
```

GÜNCELLEME İŞLEMİ (Update Operations)

```
private void btn_guncelle_Click(object sender, EventArgs e)
{
    try
    {
        if (dataGridView1.CurrentRow != null)
        {
            int selectedId = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Customer_Id"].Value);
            Customer customer = db.Customers.Find(selectedId);
            if (customer != null)
            {
                customer.Customer_Name = txt_isim.Text;
                customer.Customer_Surname = txt_soyisim.Text;
                customer.Customer_email = txt_email.Text;
                customer.Customer_telephone = txt_telefon.Text;
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

GÜNCELLEME İŞLEMİ (Update Operations)

Değer atama işleminden sonra, güncelleme işleminin gerçekleşmesi için aşağıdaki kodları yazıyoruz:

`db.SaveChanges();` // bu kodla güncelleme işlemi gerçekleştiriliyor.

`MessageBox.Show("Müşteri güncellendi!");` // bu kodla, güncelleme işlemi başarılıysa eğer mesaj verilir.

`btn_listele.PerformClick();` // bu kodla da güncelleme işleminden sonra yeni değerler listelenir.

GÜNCELLEME İŞLEMİ (Update Operations)

```
private void btn_guncelle_Click(object sender, EventArgs e)
{
    try
    {
        if (dataGridView1.CurrentRow != null)
        {
            int selectedId =
Convert.ToInt32(dataGridView1.CurrentRow.Cells["Customer_Id"].Value);
            Customer customer = db.Customers.Find(selectedId);
            if (customer != null)
            {
                customer.Customer_Name = txt_isim.Text;
                customer.Customer_Surname = txt_soyisim.Text;
                customer.Customer_email = txt_email.Text;
                customer.Customer_telephone = txt_telefon.Text;
                db.SaveChanges();
                MessageBox.Show("Müşteri güncellendi!");
                btn_listele.PerformClick();
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```


GÜNCELLEME İŞLEMİ (Update Operations)

```
try
{
    if (dataGridView1.CurrentRow != null)
    {
        int selectedId = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Customer_Id"].Value);
        Customer customer = db.Customers.Find(selectedId);
        if (customer != null)
        {
            customer.Customer_Name = txt_isim.Text;
            customer.Customer_Surname = txt_soyisim.Text;
            customer.Customer_email = txt_email.Text;
            customer.Customer_telephone = txt_telefon.Text;

            db.SaveChanges();

            MessageBox.Show("Müşteri güncellendi!");

            btn_listele.PerformClick();
        }
    }
}
catch (Exception ex)
{
    MessageBox.Show($"Hata = {ex.Message}");
}
```

GÜNCELLEME İŞLEMİ (Update Operations)

Şimdi programımızı çalıştırıp bir müşteriyi seçiyoruz ve onun mevcut bilgileri ilgili textbox'lara aktarılıyor.

	Müşteri Adı	Müşteri Soyadı	E-Posta	Telefon
	Furkan	ATLAN	furkanatlan@msn.com	5329864512
	Ahmet	TEKOK	ahmetekok@yahoo.com	5308536214
	Mehmet	VEYSEL	mveyssel@gmail.com	5391397241
	Enes	GÜMÜŞ	enesg@hotmail.com	5063021086
▶	Leyla	SARDAL	lsardal@gmail.com	5024568239
	Tuğçe	HAYAT	tugcehayat@msn.com	5389756432
	Elif	ŞEREFOĞLU	eserefoglu@gmail.com	5063217847
	Tuğba	İZCİ	tubaizci@msn.com	5387962154
	Süleyman	ASLAN	saslan@msn.com	5458563214

LİSTELE

SİL

groupBox1

Ad:

Leyla

Soyad:

SARDAL

E-Mail:

lsardal@gmail.com

Telefon:

5024568239

EKLE

GÜNCELLE

GÜNCELLEME İŞLEMİ (Update Operations)

Seçtiğimiz bu kullanıcının soyad, mail ve telefon bilgilerini değiştiriyoruz.

	Müşteri Adı	Müşteri Soyadı	E-Posta	Telefon
	Furkan	ATLAN	furkanatlan@msn.com	5329864512
	Ahmet	TEKOK	ahmetekok@yahoo.com	5308536214
	Mehmet	VEYSEL	mveyssel@gmail.com	5391397241
	Enes	GÜMÜŞ	enesg@hotmail.com	5063021086
►	Leyla	SARDAL	lsardal@gmail.com	5024568239
	Tuğçe	HAYAT	tugcehayat@msn.com	5389756432
	Elif	ŞEREFOĞLU	eserefoglu@gmail.com	5063217847
	Tuğba	İZCİ	tubaizci@msn.com	5387962154
	Süleyman	ASLAN	saslan@msn.com	5458563214

LİSTELE

SİL

groupBox1

Ad:

Leyla

Soyad:

ADA

E-Mail:

leylada@hotmail.com

Telefon:

55544443332

EKLE

GÜNCELLE

GÜNCELLEME İŞLEMİ (Update Operations)

GÜNCELLE butonuna tıkladıktan sonra, mesaj kutusunu da kapatınca, kullanıcı bilgilerinin değiştiğini görebiliyoruz.

	Müşteri Adı	Müşteri Soyadı	E-Posta	Telefon
►	Furkan	ATLAN	furkanatlan@msn.com	5329864512
	Ahmet	TEKOK	ahmetekok@yahoo.com	5308536214
	Mehmet	VEYSEL	mveysel@gmail.com	5391397241
	Enes	GÜMÜŞ	enesg@hotmail.com	5063021086
	Leyla	ADA	leylada@hotmail.com	55544443332
	Tuğçe	HAYAT	tugcehayat@msn.com	5389756432
	Elif	ŞEREFOĞLU	eserefoglu@gmail.com	5063217847
	Tuğba	İZCİ	tubaizci@msn.com	5387962154
	Süleyman	ASLAN	saslan@msn.com	5458563214

LİSTELE

SİL

groupBox1

Ad:

Leyla

Soyad:

ADA

E-Mail:

leylada@hotmail.com

Telefon:

55544443332

EKLE

GÜNCELLE

SİLME İŞLEMİ (Delete Operations)

Son olarak, Table_Customer tablosunun karşılık geldiği Customer sınıfı için Delete işlemini gerçekleştireceğiz. Delete işlemi geriye döndürülemez-ciddi bir işlem olduğu için, tıpkı güncelleme işleminde olduğu gibi silinmek istenen satır (müşteri) seçildikten sonra SİL butonuna tıklanınca hemen silmesini istemiyoruz. Öncesinde kullanıcıya bir soru sorulacak (Bu müşteriyi silmek istediğinizden emin misiniz? gibi). Eğer kullanıcı buna Evet derse o müşteriyi (kaydı) silecek, Hayır derse herhangi bir işlem yapmayacak.

Şimdi ilk olarak form'daki SİL butonuna çift tıklayıp click olayının içine try-catch bloklarını açıyoruz.

Bu defa önce kullanıcıya soru sorup sonra da bu sorunun cevabının Evet mi Hayır mı olduğunu kontrol etmemiz lazım. Eğer cevap Evet ise tıpkı güncelleme işleminde olduğu gibi iki if bloku açıyoruz. Yine, güncelleme işleminde olduğu gibi bu if bloklarında aynı durumları kontrol edeceğiz (Datagridview'da satır seçildi mi? Seçildiyse eğer bu müşteri var mı?). Silme işlemini de yine müşterinin id bilgisine göre yapacağız.

SİLME İŞLEMİ (Delete Operations)

```
private void btn_sil_Click(object sender, EventArgs e)
{
    try
    {
        DialogResult sonuc = MessageBox.Show(
            "Silmek istediğinize emin misiniz?",
            "Silme Onayı",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question);

    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

SİLME İŞLEMİ (Delete Operations)

Aşağıdaki kodlarla kullanıcının karşısına MessageBox olarak bir soru yöneltiyoruz. Sorunun cevabı Evet ya da Hayır olmak üzere 2 buton ile temsil ediliyor.

```
private void btn_sil_Click(object sender, EventArgs e)
{
    try
    {
        DialogResult sonuc = MessageBox.Show(
            "Silmek istediğinize emin misiniz?",
            "Silme Onayı",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

SİLME İŞLEMİ (Delete Operations)

Aşağıdaki kod ile eğer kullanıcının cevabı Evet ise, yapacağımız tüm işlemleri (silme) bu if blokları içerisine alacağız.

```
if (sonuc == DialogResult.Yes)
```

```
{
```

```
}
```


SİLME İŞLEMİ (Delete Operations)

```
private void btn_sil_Click(object sender, EventArgs e)
{
    try
    {
        DialogResult sonuc = MessageBox.Show(
            "Silmek istediğinize emin misiniz?",
            "Silme Onayı",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question);

        if ( sonuc == DialogResult.Yes)
        {
            // Silme işlemi buraya yazılabilir
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

SİLME İŞLEMİ (Delete Operations)

Silme işlemi, güncelleme işlemiyle çok benzerlik gösterir. Bu kodda da tıpkı güncelleme işlemindeki gibi yine iki adet if bloku açacağız ve aynı şartları kontrol edeceğiz. Bunlardan önce de yine seçilen satırdaki kullanıcının id bilgisini elde edip bu id bilgisini mevcut kullanıcılar içerisinde arayacağız (Böyle bir id'ye sahip kullanıcı var mı yok mu? diye).

SİLME İŞLEMİ (Delete Operations)

```
private void btn_sil_Click(object sender, EventArgs e)
{
    try
    {
        DialogResult sonuc = MessageBox.Show(
            "Silmek istediğimize emin misiniz?",
            "Silme Onayı",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question);

        if ( sonuc == DialogResult.Yes)
        {
            if (dataGridView1.CurrentRow != null)
            {
                int selectedId = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Customer_Id"].Value);
                Customer customer = db.Customers.Find(selectedId);
                if (customer != null)
                {
                    {
                    }
                }
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

SİLME İŞLEMİ (Delete Operations)

```
private void btn_sil_Click(object sender, EventArgs e)
{
    try
    {
        DialogResult sonuc = MessageBox.Show(
            "Silmek istediğinize emin misiniz?",
            "Silme Onayı",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question);

        if ( sonuc == DialogResult.Yes)
        {
            if (dataGridView1.CurrentRow != null)
            {
                int selectedId = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Customer_Id"].Value);
                Customer customer = db.Customers.Find(selectedId);
                if (customer != null)
                {
                    // Silme işlemi burada yapılabilir
                }
            }
        }
    }
    catch (Exception ex)
    {
        // Hata mesajı göster
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

SİLME İŞLEMİ (Delete Operations)

Şimdi direkt kullanıcıyı sileceğiz, bu değişikliği onaylayacağız, bunu MessageBox ile uyarı şeklinde göstereceğiz (Müşteri Silindi!) ve bu uyarı kapatıldığında mevcut müşterileri tekrar listeleyeceğiz.

SİLME İŞLEMİ (Delete Operations)

```
private void btn_sil_Click(object sender, EventArgs e)
{
    try
    {
        DialogResult sonuc = MessageBox.Show(
            "Silmek istediğimize emin misiniz?",
            "Silme Onayı",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question);
        if (sonuc == DialogResult.Yes)
        {
            if (dataGridView1.CurrentRow != null)
            {
                int selectedId = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Customer_Id"].Value);
                Customer customer = db.Customers.Find(selectedId);
                if (customer != null)
                {
                    db.Customers.Remove(customer);
                    db.SaveChanges();
                    MessageBox.Show("Müşteri Silindi!");
                    btn_listele.PerformClick();
                }
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

SİLME İŞLEMİ (Delete Operations)

```
private void btn_sil_Click(object sender, EventArgs e)
{
    try
    {
        DialogResult sonuc = MessageBox.Show(
            "Silmek istediğimize emin misiniz?",
            "Silme Onayı",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question);

        if (sonuc == DialogResult.Yes)
        {
            if (dataGridView1.CurrentRow != null)
            {
                int selectedId = Convert.ToInt32(dataGridView1.CurrentRow.Cells["Customer_Id"].Value);
                Customer customer = db.Customers.Find(selectedId);
                if (customer != null)
                {
                    db.Customers.Remove(customer);
                    db.SaveChanges();

                    MessageBox.Show("Müşteri Silindi!");
                    btn_listele.PerformClick();
                }
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

Müşteri ve Ürünlerin Birlikte Gösterilmesi ve İşlemler

Silme işleminden sonra bir tabloda yapılacak 4 temel veri tabanı işlemini bitirmiş olduk. Tüm bu işlemleri veri tabanı tarafında değil görsel tarafta programlama dili kullanarak EntityFramework konseptine uygun şekilde hallettik.

Şimdi sıra geldi müşteri ve ürün bilgilerini tutan Table_CustomerProduct tablosu ve onunla ilgili 4 temel işlemi C# tarafında yapmaya.

Table_CustomerProduct tablosu çok önemlidir bu proje için. Çünkü Table_Customer içerisinde sadece müşterilerle ilgili işlemler yapıyoruz, Table_Product içerisinde de sadece ürünlerle ilgili işlemler yapıyoruz. Ancak, Table_CustomerProduct tablosunda ise hangi müşterinin hangi ürünü aldığını, aldığı hangi ürünü güncellediğini ya da hangi üründen vazgeçtiğini (sildiğini) tanımlayabiliriz.

Müşteri ve Ürünlerin Birlikte Gösterilmesi ve İşlemler

İlk olarak SQL'deki diğer tablolar için de birer sınıf oluşturacağız, tıpkı Table_Customer için Customer class'ı oluşturduğumuz gibi.

Ayrıca, Table_Customer ve Table_Product tabloları Table_CustomerProduct tablosuyla ilişkili oldukları için bunlara ait sınıflara da bir kod ekleyeceğiz.

Nasıl ki SQL tarafında tablolar arasında ilişki kurarken primary key-foreign key üzerinden ilişki kuruyorsak, EntityFramework konseptinde de bu ilişkiyi bu konsept içerisinde betimlememiz gerekir.

MSSQL'deki Tüm Tabloların Sınıf Olarak Oluşturulması

Şu ana kadar yaptığımız tüm işlemler, SQL'deki 4 tablomuz arasından seçilen tek bir tablo içindi. Şimdi, diğer 3 tablomuz için de aynı şekilde birer sınıf oluşturup, EntityFramework kapsamında bu tabloları C# tarafında bir sınıfla ilişkilendirmeliyiz.

Table_Category için Category Sınıfı

Table_Category tablosu, ürünlerin (Table_Product) hangi kategoride yer aldığını belirtmek için kullanılıyordu ve sadece aşağıda görüldüğü gibi 2 sütunu vardı.

	Column Name	Data Type	Allow Nulls
	Category_Id	int	☐
	Category_Name	nvarchar(50)	☐

Table_Category için Category Sınıfı

Şimdi, visual studio tarafında projemize sağ tıklayıp Ekle->Sınıf (Add-Class) adımlarını takip edip Category adında bir sınıf ekliyoruz ve açılan Category.cs dosyasındaki kodlar aşağıdaki gibi olmalıdır.

```
[Table("Table_Category")]
internal class Category
{
    [Key]
    public int Category_Id { get; set; }

    [Required, MaxLength(50)]
    public string Category_Name { get; set; }

    public virtual ICollection<Product> Products { get; set; }
}
```

Table_Category için Category Sınıfı

Dikkat edilecek olursa, önceki sayfada yer alan bir kodun Table_Category içerisinde bir sütun olarak yeri yoktu: `public virtual ICollection<Product> Products { get; set; }`

Bu kod, Table_Category tablosunun Table_Product tablosuyla ilişkisi olduğu için eklendi. Nasıl ki SQL tarafında Diagramlar ile foreign key primary key ilişkisini kuruyosak, EntityFramework konseptinde de birbiriyle ilişkili sütunları Navigation Property olarak adlandırılan bu özellikler üzerinden kurarız.

Bunu bir köprü gibi düşünebilirsiniz.

Sizler de, bir tablonuzu sınıf olarak tanıtırken eğer başka bir tabloyla ilişkisi varsa yukarıdaki özellikte Product yazan yerde o tablonun sınıf adını yazıp yanındaki Products property'sini de o tabloya uygun bir isim olarak verebilirsiniz.

NOT: biz henüz Table_Product tablosuna karşılık gelen Product sınıfını yazmadığımız (oluşturmadığımız) için bu kod şimdilik hata verebilir ama önemli değil. Product sınıfını tanımlayınca bu hata gidicektir.

Table_Category için Category Sınıfı

NOT: using System bloklarına, görseldeki kırmızı alanda yer alan kodlar siz de yoksa bunları yazmalısınız.

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Otomasyon_Projesi
{
    [Table("Table_Category")]
    1 başvuru
    internal class Category
    {
        [Key]
        0 başvuru
        public int Category_Id { get; set; }

        [Required, MaxLength(50)]
        0 başvuru
        public string Category_Name { get; set; }

        0 başvuru
        public virtual ICollection<Product> Products { get; set; }
    }
}
```

Table_Product için Product sınıfı

Table_Product tablosunun SQL tarafındaki sütun ve veri tipleri aşağıdaki gibiydi. C# tarafında Product sınıfı tanımlarken (Ekle->Sınıf dedikten sonra Product ismini veriyoruz sınıfımıza) bu bilgilere ihtiyacımız olacak.

	Product_Id	int	☐
	Product_Name	nvarchar(50)	☐
	Product_Price	decimal(18, 0)	☐
	Category_Id	int	☐

Table_Product için Product sınıfı

C# tarafında Table_Product tablosu için eklediğimiz Product sınıfının kodları da aşağıdaki gibi olmalıdır:

```
[Table("Table_Product")]
internal class Product
{
    [Key]
    public int Product_Id { get; set; }

    [Required, MaxLength(50)]
    public string Product_Name { get; set; }

    [Required]
    public decimal Product_Price { get; set; }

    [ForeignKey("Category")]
    public int Category_Id { get; set; }

    public virtual Category Category { get; set; }
    public virtual ICollection<CustomerProduct> CustomerProducts { get; set; }
}
```


Table_Product için Product sınıfı

Product sınıfı Table_Product tablosuna karşılık geliyor. Table_Product tablosunun 2 tabloyla ilişkisi vardı. Bunlar: Table_Category ve Table_CustomerProduct.

Dolayısıyla Navigation Property olarak biz de 2 tane property ekledik. Bunlar sayesinde C# tarafında Product sınıfımızın Category ve birazdan oluşturacağımız CustomerProduct sınıflarıyla ilişkisini tanıtmış olduk. Tüm bunları, aşağıdaki 2 satırlık kod ile yaptık:

```
public virtual Category Category { get; set; }  
public virtual ICollection<CustomerProduct> CustomerProducts { get; set; }
```

Table_Product için Product sınıfı

Burada akıllara şu soru gelebilir: Neden Category sınıfında Product sınıfı ile ilişkisini kurarken

```
public virtual ICollection<Product> Products { get; set; }
```

Kodunu yazdık da Product sınıfında Category ile ilişkisini kurarken

```
public virtual Category Category { get; set; }
```

Kodunu yazdık?

Bu durum ilişki türüyle ilgilidir. Yani Product ve Category arasında one-to-many ilişkisi vardır. Bir ürünün bir kategorisi olabilir ama bir kategoriye ait birden fazla ürün olabilir. Bundan dolayı ürünün bir kategorisi olduğu için biz bir ICollection tipinde tanımlamadık bunu direkt Category nesnesi olarak tanımladık.

Gerçek hayatta bazı durumlarda bir ürünün de birden fazla kategorisi olabilir. Eğer siz böyle bir ilişki kurmak istiyorsanız o zaman ICollection<Category> olarak tanımlarsınız. Ancak, bunun için bir de CategoryProduct sınıfı yazmalısınız ve bunun için SQL tarafında da buna göre bir tablo oluşturmalısınız.

Biz bu senaryoda bir ürünün bir kategorisi olabilir ilişkisini tercih ediyoruz.

Table_Product için Product sınıfı

```
using System.ComponentModel.DataAnnotations;  
using System.ComponentModel.DataAnnotations.Schema;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
  
namespace Otomasyon_Projesi  
{  
    [Table("Table_Product")]  
    2 başvuru  
    internal class Product  
    {  
        [Key]  
        0 başvuru  
        public int Product_Id { get; set; }  
  
        [Required, MaxLength(50)]  
        0 başvuru  
        public string Product_Name { get; set; }  
  
        [Required]  
        0 başvuru  
        public decimal Product_Price { get; set; }  
  
        [ForeignKey("Category")]  
        0 başvuru  
        public int Category_Id { get; set; }  
  
        0 başvuru  
        public virtual Category Category { get; set; }  
        0 başvuru  
        public virtual ICollection<CustomerProduct> CustomerProducts { get; set; }  
    }  
}
```

Table_CustomerProduct Tablosu için CustomerProduct Sınıfı

Hangi müşterinin hangi ürünü aldığı bilgisini tutan Table_CustomerProduct tablosu için projemize sağ tıklayıp Ekle->Sınıf adımlarını takip edip CustomerProduct ismini veriyoruz. SQL tarafında bu tablonun sütunları ve bu sütunların veri tipleri aşağıda gösterilmektedir.

	CusPro_Id	int	☐
	Customer_Id	int	☐
	Product_Id	int	☐

Table_CustomerProduct Tablosu için CustomerProduct Sınıfı

Table_CustomerProduct tablosunun Table_Customer ve Table_Product ile ilişkisi bulunmaktadır. Bundan dolayı, CustomerProduct sınıfını oluştururken bu ilişkileri tanımlamamız gerekir ki nasıl ki SQL üzerinden bunların ilişkisi varsa C# tarafında da bu ilişkiyi kurmalıyız.

CustomerProduct sınıfının kodları aşağıdaki gibidir.

```
[Table("Table_CustomerProduct")]
internal class CustomerProduct
{
    [Key]
    public int CusPro_Id { get; set; }

    [ForeignKey("Customer")]
    public int Customer_Id { get; set; }

    [ForeignKey("Product")]
    public int Product_Id { get; set; }

    public virtual Customer Customer { get; set; }
    public virtual Product Product { get; set; }
}
```

Table_CustomerProduct Tablosu için CustomerProduct Sınıfı

Dikkat edilecek olursa Customer ve Product sınıflarını birer ICollection olarak tanımlamadık. Çünkü bu tablo zaten her iki tablonun da bulunduğu ilişkinin kendisidir. Burada ara tablo yoktur. Bundan dolayı, bu şekilde yazılmalıdır.

```
using System.ComponentModel.DataAnnotations.Schema;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Otomasyon_Projesi
{
    [Table("Table_CustomerProduct")]
    1 başvuru
    internal class CustomerProduct
    {
        [Key]
        0 başvuru
        public int CusPro_Id { get; set; }

        [ForeignKey("Customer")]
        0 başvuru
        public int Customer_Id { get; set; }

        [ForeignKey("Product")]
        0 başvuru
        public int Product_Id { get; set; }

        0 başvuru
        public virtual Customer Customer { get; set; }
        0 başvuru
        public virtual Product Product { get; set; }
    }
}
```

Table_CustomerProduct Tablosu için CustomerProduct Sınıfı

Eğer Icollection ile normal bir sınıf nesnesi tanımlama arasındaki farkı anlamadıysanız aşağıdaki görsel sizin için faydalı olabilir:

Özellik	<code>Customer Customer</code>	<code>ICollection<Customer> Customers</code>
Kaç adet?	1 müşteri	Birden çok müşteri
İlişki türü	Tekil (one-to-one veya many-to-one)	Çoklu (one-to-many veya many-to-many)
Navigation amacı	"Ben bu müşteriyle ilişkiliyim"	"Bana bağlı birçok müşteri var"

Customer Sınıfının Düzeltilmesi

Hatırlarsanız ilk tablomuz olan Table_Customer için yazdığımız ilk sınıfın (Customer) kodlarını tamamlamıştık ve 4 temel veri tabanı işlemini de halletmiştik. Ancak, projemizi genişlettik ve yeni işlemler-ilişkiler ekledik. Bunun için Table_Customer tablosunun da Table_CustomerProduct ile ilişkisi mevcuttur.

Yani, C# tarafında Customer sınıfında CustomerProduct sınıfı için bir ilişki eklememiz lazım.

Customer Sınıfının Düzeltilmesi

Mevcut Customer sınıfının kodları aşağıdaki gibidir:

```
[Table("Table_Customer")]
internal class Customer
{
    [Key]
    public int Customer_Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string Customer_Name { get; set; }

    [Required]
    [MaxLength(50)]
    public string Customer_Surname { get; set; }

    [Required]
    [MaxLength(50)]
    public string Customer_email { get; set; }

    [Required]
    [MaxLength(50)]
    public string Customer_telephone { get; set; }
}
```

Customer Sınıfının Düzeltilmesi

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Otomasyon_Projesi
{
    [Table("Table_Customer")]
    6 başvuru
    internal class Customer
    {
        [Key]
        0 başvuru
        public int Customer_Id { get; set; }

        [Required]
        [MaxLength(50)]
        2 başvuru
        public string Customer_Name { get; set; }

        [Required]
        [MaxLength(50)]
        2 başvuru
        public string Customer_Surname { get; set; }

        [Required]
        [MaxLength(50)]
        2 başvuru
        public string Customer_email { get; set; }

        [Required]
        [MaxLength(50)]
        2 başvuru
        public string Customer_telephone { get; set; }
    }
}
```

Customer Sınıfının Düzeltilmesi

Şimdi Customer sınıfı içerisine aşağıdaki kodu ekliyoruz:

```
public virtual ICollection<CustomerProduct> CustomerProducts { get; set; }
```

Customer sınıfının birden fazla CustomerProduct ile işi olabileceği için ICollection olarak ekledik.

Customer Sınıfının Düzeltilmesi

```
namespace Otomasyon_Projesi
{
    [Table("Table_Customer")]
    6 başvuru
    internal class Customer
    {
        [Key]
        0 başvuru
        public int Customer_Id { get; set; }

        [Required]
        [MaxLength(50)]
        2 başvuru
        public string Customer_Name { get; set; }

        [Required]
        [MaxLength(50)]
        2 başvuru
        public string Customer_Surname { get; set; }

        [Required]
        [MaxLength(50)]
        2 başvuru
        public string Customer_email { get; set; }

        [Required]
        [MaxLength(50)]
        2 başvuru
        public string Customer_telephone { get; set; }

        0 başvuru
        public virtual ICollection<CustomerProduct> CustomerProducts { get; set; }
    }
}
```

CustomerDbContext Sınıfının Düzenlenmesi

Nasıl ki Customer sınıfı için CustomerDbContext içerisinde

`public DbSet<Customer> Customers { get; set; }` kodunu tanımladıysak.

Aynı şekilde diğer sınıflar için de sırasıyla aşağıdaki kodları tanımlıyoruz:

```
public DbSet<Product> Products { get; set; }
```

```
public DbSet<Category> Categories { get; set; }
```

```
public DbSet<CustomerProduct> CustomerProducts { get; set; }
```

CustomerDbContext Sınıfının Düzenlenmesi

CustomerDbContext'in son hali aşağıdaki gibidir:

```
internal class CustomerDbContext:DbContext
{
    public CustomerDbContext() : base("name=CustomerDbContext")
    {

    }

    public DbSet<Customer> Customers { get; set; }
    public DbSet<Product> Products { get; set; }
    public DbSet<Category> Categories { get; set; }
    public DbSet<CustomerProduct> CustomerProducts { get; set; }

}
```

CustomerDbContext Sınıfının Düzenlenmesi

Her bir sınıf için bir DbContext sınıfı yazmaya gerek yoktur. Bu durum, EntityFramework konseptine aykırıdır. Çünkü her bir sınıf bir tabloya karşılık geliyor ve bu tabloların tamamı da aynı veri tabanında yer alıyor. Dolayısıyla aynı veri tabanındaki tüm tablolar için bir tane DbContext sınıfı yazılması yeterlidir. Nitekim `base(name=CustomerDbContext)` kısmı `App.config`'deki `connectionstring` bilgisinin adı (name) `CustomerDbContext` olan bağlantıyı baz alıyordu.

CustomerDbContext Sınıfının Düzenlenmesi

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Data.Entity;

namespace Otomasyon_Projesi
{
    3 başvuru
    internal class CustomerDbContext:DbContext
    {
        1 başvuru
        public CustomerDbContext() : base("name=CustomerDbContext")
        {
        }

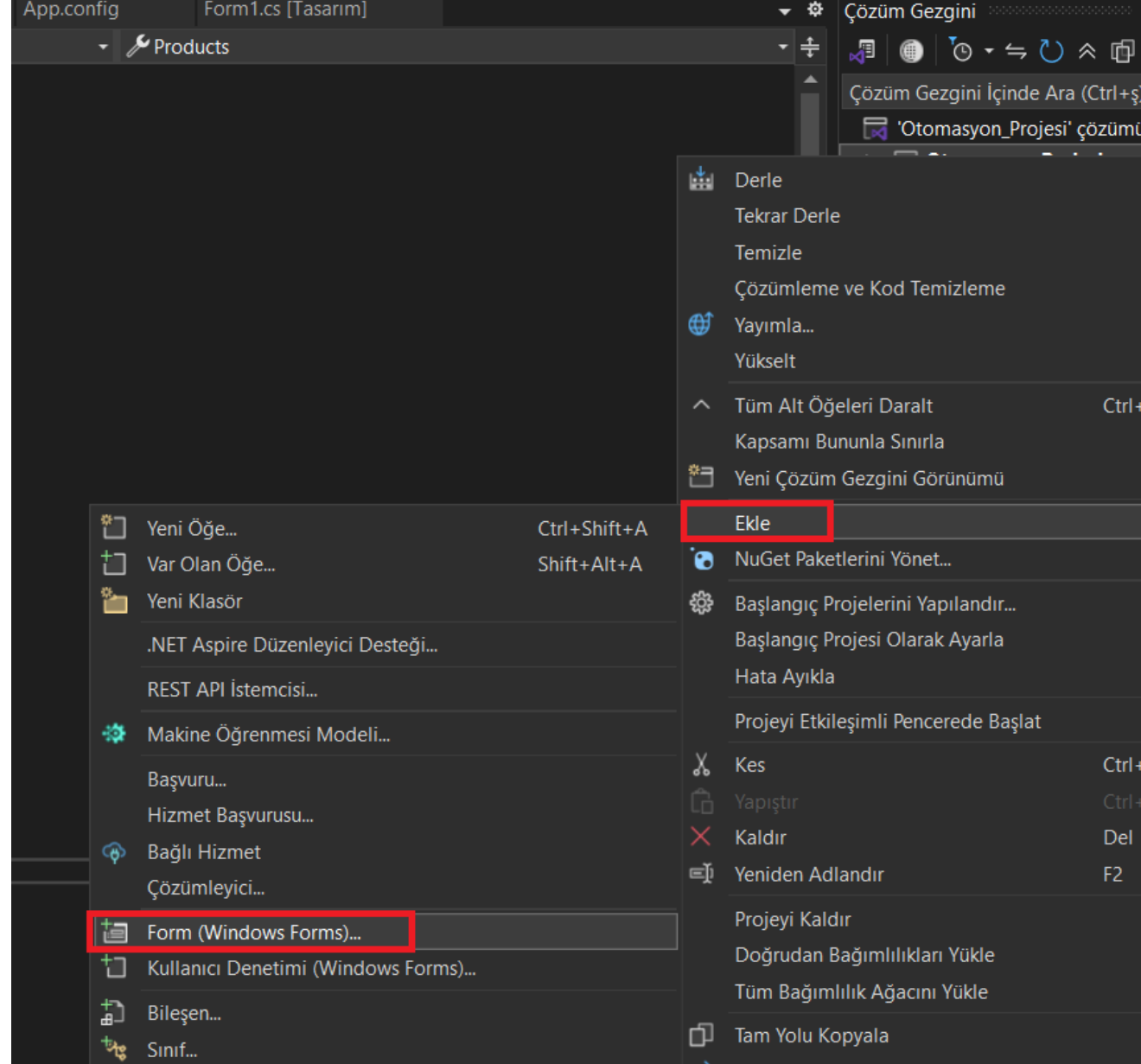
        5 başvuru
        public DbSet<Customer> Customers { get; set; }
        0 başvuru
        public DbSet<Product> Products { get; set; }
        0 başvuru
        public DbSet<Category> Categories { get; set; }
        0 başvuru
        public DbSet<CustomerProduct> CustomerProducts { get; set; }
    }
}
```


CustomerProduct Sınıfı için Form Oluşturulması

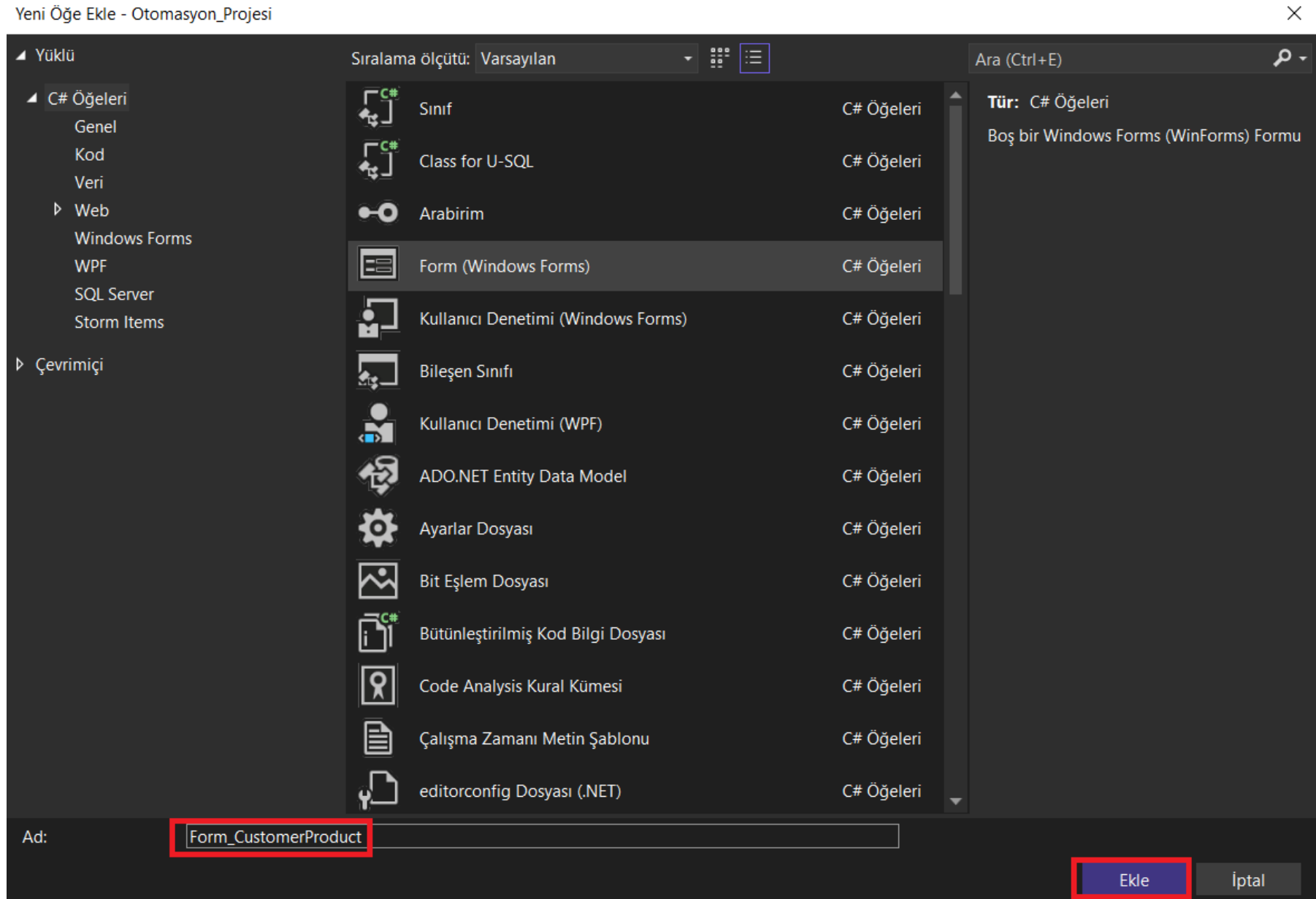
Bütün ayarlamalarımızı yaptıktan sonra mevcut formumuza bir buton ekleyeceğiz. Bu buton bizi CustomerProduct sınıfı için oluşturacağımız form'a yönlendirecek. Bunun için ilk olarak projemize yeni bir form eklememiz lazım.

Projemize sağ tıklayıp Ekle->Form (Windows Forms) seçeneğini tıklayıp formumuza Form_CustomerProduct adını veriyoruz.

CustomerProduct Sınıfı için Form Oluşturulması



CustomerProduct Sınıfı için Form Oluşturulması



CustomerProduct Sınıfı için Form Oluşturulması

Şimdi, ilk formumuza dönüp yeni bir buton ekleyip bunun text bilgisine Form CustomerProduct ismini verip id bilgisine de btn_form_cp adını veriyoruz.

Form'un görsel bütünlüğü açısından yeni bir groupbox ekleyip, başka formlara geçişi sağlayacak olan butonları burada toplayabiliriz.

İlk formumuzun son hali sonraki sayfada yer almaktadır.

CustomerProduct Sınıfı için Form Oluřturulması

Form1

— □ ×

LİSTELE

SİL

KAYIT EKLEME VE GÜNCELLEME

Ad:

Soyad:

E-Mail:

Telefon:

EKLE

GÜNCELLE

FORMLARA GEÇİŞ

FORM
CUSTOMERPRODUCT

CustomerProduct Sınıfı için Form Oluşturulması

Şimdi, son eklediğimiz butona çift tıklayıp bunun click olayı için Form_CustomerProduct formuna geçişi için aşağıdaki kodu yazıyoruz:

```
private void btn_form_cp_Click(object sender, EventArgs e)
{
    Form_CustomerProduct form2_cp = new Form_CustomerProduct();
    this.Hide();
    form2_cp.ShowDialog();
    this.Close();
}
```

CustomerProduct Sınıfı için Form Oluşturulması

```
private void btn_form_cp_Click(object sender, EventArgs e)
{
    Form_CustomerProduct form2_cp = new Form_CustomerProduct();
    this.Hide();
    form2_cp.ShowDialog();
    this.Close();
}
```

CustomerProduct Sınıfı için Form Oluşturulması

Kodunuzu çalıştırıp butona tıkladığınızda yeni eklediğiniz boş form'a geçiş yaptığını göreceksiniz.

Şimdi ise ikinci formumuzu doldurmamız lazım.

Yine bir adet datagridview ekleyeceğiz.

Verileri listelemek, silmek ve eklemek için 3 adet buton ekleyeceğiz.

2 adet label ve 2 adet combobox ekleyeceğiz.

Diğer formdan farklı olarak bu form'da güncelleme yapmamıza gerek yok. Çünkü CustomerProduct sınıfı, Customer ve Product sınıflarını içerdiği için burada yapılacak bir güncelleme aslında diğerinde yapılabilir.

Biz burada sadece hangi müşterinin hangi ürünü satın aldığıyla ilgileniyoruz.

Textbox yerine combobox kullanmamızın sebebi de yine aynı şekilde buradaki Customer bilgilerinin Customer sınıfından gelmesi ve Product bilgilerinin de Product sınıfından gelmesi. Dolayısıyla biz sadece var olan müşteriler ve var olan ürünlerle ilgileneceğiz.

CustomerProduct Sınıfı için Form Oluşturulması

Ekleyeceğimiz bileşenlerin id bilgileri şöyle olacak:

Combobox'lar için combo_customer, combo_product

Butonlar için btn_listele, btn_ekle, btn_sil

Bir adet buton daha ekleyeceğiz. Bu buton da önceki form'a geçiş yapmak için kullanılacak. Ona da btn_form_customer diyeceğiz.

Formumuzun görünümü sonraki sayfada yer almaktadır.

CustomerProduct Sınıfı için Form Oluřturulması

Form_CustomerProduct

— □ ×

LISTELE

SİL

groupBox1

MÜŐTERİ

ORON

EKLE

ANA FORM

CustomerProduct Sınıfı için Form Oluşturulması

Formumuzu oluşturduktan sonra, neden Customer ve Product bilgilerini tek bir tabloda tek bir sınıfta birleştirdiğimizi anlamalıyız. Şimdi, SQL tarafına gidip Table_CustomerProduct sınıfına bir select * from sorgusu yazacağız.

CustomerProduct Sınıfı için Form Oluşturulması

Aşağıda Table_CustomerProduct tablosundaki mevcut kayıtları görüyoruz. Burada:

CusPro_Id sütunu Table_CustomerProduct tablosunun primary key sütunudur (kayıtların sıra numarasını tutar)

Customer_Id sütunu, Table_Customer tablosundaki ilgili müşterinin tüm bilgilerini tutan foreign key sütunudur

Product_Id sütunu, Table_Product tablosundaki ilgili ürünün tüm bilgilerini tutan foreign key sütunudur

Ancak bu şekilde bakınca oldukça anlamsız gelebilir. Çünkü herhangi bir bilgi içermiyor.

	CusPro_Id	Customer_Id	Product_Id
1	1	1	3
2	2	2	5
3	3	4	2

CustomerProduct Sınıfı için Form Oluşturulması

Şimdi, Table_CustomerProduct tablosundaki ilk kaydı inceleyelim.

	CusPro_Id	Customer_Id	Product_Id
1	1	1	3
2	2	2	5
3	3	4	2

Table_Customer tablosundaki 1. kayıttaki müşteriyi bulalım önce. Aşağıda, Table_Customer tablosundaki ilk kayıt gösterilmektedir.

	Customer_Id	Customer_Name	Customer_Surname	Customer_Email	Customer_Telephone
1	1	Furkan	ATLAN	furkanatlan@msn.com	5329864512
2	2	Ahmet	TEKOK	ahmetekok@yahoo.com	5308536214
3	3	Mehmet	VEYSEL	mveysel@gmail.com	5391397241
4	4	Enes	GÜMÜŞ	enesg@hotmail.com	5063021086
5	5	Leyla	ADA	leylada@hotmail.com	55544443332
6	6	Tuğçe	HAYAT	tugcehayat@msn.com	5389756432
7	7	Elif	ŞEREFOĞLU	eserefoglu@gmail.com	5063217847
8	8	Tuğba	İZCİ	tubaizci@msn.com	5387962154
9	1009	Süleyman	ASLAN	saslan@msn.com	5458563214

CustomerProduct Sınıfı için Form Oluşturulması

Demek ki burada 1 numara ile kast edilen Aslında Furkan ATLAN müşterisiymiş. Şimdi de aldığı ürünün ID bilgisini hatırlatalım:

	CusPro_Id	Customer_Id	Product_Id
1	1	1	3
2	2	2	5
3	3	4	2

1 numaralı müşteri (Furkan ATLAN), 3 numaralı ürünü almış. Şimdi Table_Product tablosundaki 3. kayıta yer alan ürüne bakalım:

	Product_Id	Product_Name	Product_Price	Category_Id
1	1	Samsung Galaxy S24	35000	5
2	2	Iphone 16 Promax	80000	5
3	3	Keten Kumaş	750	1
4	4	Blender	850	4
5	5	Uzaktan Kumandalı Araba	4280	6

CustomerProduct Sınıfı için Form Oluşturulması

Şimdi bu bilgilere göre şunu söyleyebiliriz. Furkan ATLAN isimli müşteri, 750 TL tutarında bir Keten Kumaş pantolon almış.

İşte Table_CustomerProduct tablosu ve C# tarafında buna karşılık gelen CustomerProduct sınıfı bu iş için oluşturuldu.

Çünkü Table_Customer tablosu tek başına sadece müşteri bilgilerini tutuyor. Table_Product tablosu da tek başına sadece ürünlerin bilgilerini tutuyor ama Table_CustomerProduct tablosu, hangi müşteri hangi ürünü almış, bir müşterinin aldığı toplam ürün sayısı kaç, bir müşterinin aldığı ürünlere ödediği toplam tutar ne kadar? Gibi soruların yanıtları için oluşturuldu.

CustomerProduct Sınıfı için Form Oluşturulması

Şimdi ilk olarak, Table_CustomerProduct tablosu için listeleme sorgusu yazacağız. Ancak, normalde bu sorguda sadece müşteri ve ürünün id bilgisi gelecek. Biz böyle bir şey istemiyoruz. Dolayısıyla bir JOIN işlemi yapacağız. ID bilgileri yerine o id bilgilerinin tuttuğu diğer bilgileri (müşteri isim, müşteri soyisim, ürün isim, ürün fiyatı, ürünün kategorisi). Ürün kategorisini de yine id olarak değil name olarak getireceğiz.

Şimdi ilk iş olarak LİSTELE text bilgisine ve btn_listele name bilgisine sahip olan butona çift tıklayıp try-catch blokları açıp kodlarımızı yazmaya başlayalım.

CustomerProduct Sınıfı için Form Oluşturulması

Şimdi öncelikle hatırlarsanız bizim tüm tablolarımızın 4 temel işlemi için CustomerDbContext sınıfını oluşturmuştuk ve tüm tablolarımızı da buraya bağlamıştık.

Dolayısıyla Table_CustomerProduct tablosu için sorgu yazarken de CustomerDbContext sınıfını kullanacağız.

İlk form'da db adında bir nesne türetmiştik. Aynısını 2. formda da yine public alanda yapacağız ki tüm tablolar ile ilgili 4 temel işlemi bu db nesnesi üzerinden gerçekleştirebilelim.

Bir tablodaki tüm sütunları değil de sadece istediğimiz sütunları yazmak için SQL'de join işlemi yaptığımızı hatırlayın.

C# tarafında bu tarz özel sorgulara LINQ sorguları denir. LINQ'in açılımı: Language Integrated Query'dir. LINQ, C# tarafında SQL, XML gibi veri kaynakları için yazılan özel sorgunun adıdır.

CustomerProduct Sınıfı için Form Oluşturulması

İlk olarak 2. formumuzun kod sayfasına gidip public alanda yine CustomerDbContext sınıfından türeyen ve db adında bir instance türeteceğiz.

```
CustomerDbContext db = new CustomerDbContext();
```

Ayrıca using bloklarının olduğu kısma şu kodu yazacağız:

```
using System.Xml.Linq;
```

CustomerProduct Sınıfı için Form Oluşturulması

```
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Xml.Linq;

namespace Otomasyon_Projesi
{
    4 başvuru
    public partial class Form_CustomerProduct : Form
    {
        1 başvuru
        public Form_CustomerProduct()
        {
            InitializeComponent();

            CustomerDbContext db = new CustomerDbContext();

            1 başvuru
            private void btn_listele_Click(object sender, EventArgs e)
            {
            }
        }
    }
}
```

CustomerProduct Sınıfı için LİSTELEME İşlemi

Şimdi btn_listele'nin click olayı içerisine listeleme işlemi için kodumuzu yazacağız. Ancak, bu defa uzun bir sorgu olacağı için bu sorguyu bir değişkene atayacağız. Bu, derleyicinin (compiler) sorgu türünü otomatik olarak çözmesi için kullanışlı bir yöntemdir. NOT using bloklarının olduğu kısma şu kodu kopyalayın:

```
using System.Data.Entity;
```

CustomerProduct Sınıfı için LİSTELEME İşlemi

Aşağıdaki kodu da btn_listele'nin içine yazın

```
var list = db.CustomerProducts
    .Include(cp => cp.Customer)
    .Include(cp => cp.Product.Category)
    .Select(cp => new
    {
        cp.CusPro_Id,
        MüşteriAdı = cp.Customer.Customer_Name,
        MüşteriSoyadı = cp.Customer.Customer_Surname,
        ÜrünAdı = cp.Product.Product_Name,
        ÜrünFiyatı = cp.Product.Product_Price,
        Kategori = cp.Product.Category.Category_Name
    })
    .ToList();
```

CustomerProduct Sınıfı için LİSTELEME İşlemi

Şimdi bu sorguyu adım adım işleyelim.

db nesnesi CustomerDbContext sınıfından türetilmişti. Table_CustomerProduct tablosu ile ilgili işlemler de CustomerDbContext sınıfı içerisinde yer alıyordu:

```
internal class CustomerDbContext : DbContext
{
    2 başvuru
    public CustomerDbContext() : base("name=CustomerDbContext")
    {
        ...
    }

    5 başvuru
    public DbSet<Customer> Customers { get; set; }
    0 başvuru
    public DbSet<Product> Products { get; set; }
    0 başvuru
    public DbSet<Category> Categories { get; set; }
    0 başvuru
    public DbSet<CustomerProduct> CustomerProducts { get; set; }
}
```

CustomerProduct Sınıfı için LİSTELEME İşlemi

cp anahtar kelimesi bir lambda kısaltmasıdır. Lambda C#'taki sorgularda kullanılan bir yapıdır.

Biz her defasında CustomerProduct demek yerine bunu cp adıyla kısalttık (Tıpkı SQL'deki AS ifadesi gibi)

Sonra da

```
.Include(cp => cp.Customer)
```

```
.Include(cp => cp.Product.Category)
```

 komutlarıyla Table_CustomerProduct üzerinde yer alan Table_Customer tablosuna Customer_Id üzerinden ve yine Table_CustomerProduct üzerinde yer alan Table_Product tablosuna da Product_Id üzerinden eriştik bu iki satırlık komutla.

Yani Table_CustomerProduct tablosunda foreign key olarak kullanılan anahtarları kullanarak bu anahtarların primary key olarak tanımlandığı tablolara eriştik.

NOT: Product tablosundaki foreign key sayesinde de Category tablosuna eriştik ki Category_Id yerine Category_Name bilgisini çekebilelim.

Sizler de tüm tablolarınız için bir tane DbContext sınıfı oluşturup hepsini orda tanımlarsanız tüm tablolarınıza erişebilirsiniz.

CustomerProduct Sınıfı için LİSTELEME İşlemi

```
.Select(cp => new  
{  
    cp.CusPro_Id,  
    MüşteriAdı = cp.Customer.Customer_Name,  
    MüşteriSoyadı = cp.Customer.Customer_Surname,  
    ÜrünAdı = cp.Product.Product_Name,  
    ÜrünFiyatı = cp.Product.Product_Price,  
    Kategori = cp.Product.Category.Category_Name  
})
```

.ToList(); sorgusu ile customerproduct ögesi içerisinde sadece istediğimiz alanları seçmek için select komutunu özelleştirdik. new anahtar kelimesi normalde bir sınıftan bir nesne türetildiğinde kullanılır. Ama buradaki amacı bu değil, hem sadece istediğimiz sütunları getirmek için hem de bu sütunlara AS (Alias) özelliğiyle orijinal sütun isimleriyle değil de kendi istediğimiz sütun isimleriyle gelsin diye kullandık. Son olarak tüm bu sorguyu .ToList() metoduyla bir listeleme işlemine dönüştürüyoruz. (Önceki formda da listeleme işlemi için .ToList() metodunu kullanmıştık)

CustomerProduct Sınıfı için LİSTELEME İşlemi

NOT: tüm kodu try-catch blokları arasına yazmayı unutmayın:

```
private void btn_listele_Click(object sender, EventArgs e)
{
    try
    {
        var list = db.CustomerProducts
            .Include(cp=>cp.Customer).Include(cp=>cp.Product.Category)
            .Select(cp => new
            {
                cp.CusPro_Id,
                MüşteriAdı = cp.Customer.Customer_Name,
                MüşteriSoyadı = cp.Customer.Customer_Surname,
                ÜrünAdı = cp.Product.Product_Name,
                ÜrünFiyatı = cp.Product.Product_Price,
                Kategori = cp.Product.Category.Category_Name
            })
            .ToList();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

CustomerProduct Sınıfı için LİSTELEME İşlemi

Daha sonra bu yazdığımız sorguyu datagridview'in DataSource özelliğine atıyoruz ki artık butona tıklandığında datagridview içerisine bilgiler yüklensin.

- `dataGridView1.DataSource = list;`

Sonra da datagridview'in sütunları tüm ekranı eşit bir şekilde kaplasın diye aşağıdaki sorguyu yazıyoruz.

- `dataGridView1.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;`

CustomerProduct Sınıfı için LİSTELEME İşlemi

```
private void btn_listele_Click(object sender, EventArgs e)
{
    try
    {
        var list = db.CustomerProducts
            .Include(cp=>cp.Customer).Include(cp=>cp.Product.Category)
            .Select(cp => new
            {
                cp.CusPro_Id,
                MüşteriAdı = cp.Customer.Customer_Name,
                MüşteriSoyadı = cp.Customer.Customer_Surname,
                ÜrünAdı = cp.Product.Product_Name,
                ÜrünFiyatı = cp.Product.Product_Price,
                Kategori = cp.Product.Category.Category_Name
            })
            .ToList();

        dataGridView1.DataSource = list;
        dataGridView1.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

CustomerProduct Sınıfı için LİSTELEME İşlemi

Şimdi programımızı çalıştırıp ikinci form'a geçip, listeleme tuşuna basınca sonuçları görüyoruz:

Form_CustomerProduct

	CusPro_Id	MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
▶	1	Furkan	ATLAN	Keten Kumaş	750	Giyim
	2	Ahmet	TEKOK	Uzaktan Kumandalı Araba	4280	Oyuncak
	3	Enes	GÜMÜŞ	İphone 16 Promax	80000	Elektrik-Elektronik

LİSTELE SİL

groupBox1

MÜŞTERİ

ÜRÜN

ANA FORM

CustomerProduct Sınıfı için LİSTELEME İşlemi

Görüldüğü gibi her şey istediğimiz gibi geldi. Ancak, Table_CustomerProduct tablosunun CusPro_Id sütunu orijinal ismiyle geldi. Bizim bu sütundaki bilgilere hiç ihtiyacımız yok. Dolayısıyla bu sütunu kaldırmak için kodlarımıza aşağıdaki kodu da ekliyoruz:

```
dataGridView1.Columns["CusPro_Id"].Visible = false;
```

CustomerProduct Sınıfı için LİSTELEME İşlemi

```
private void btn_listele_Click(object sender, EventArgs e)
{
    try
    {
        var list = db.CustomerProducts
            .Include(cp=>cp.Customer).Include(cp=>cp.Product.Category)
            .Select(cp => new
            {
                cp.CusPro_Id,
                MüşteriAdı = cp.Customer.Customer_Name,
                MüşteriSoyadı = cp.Customer.Customer_Surname,
                ÜrünAdı = cp.Product.Product_Name,
                ÜrünFiyatı = cp.Product.Product_Price,
                Kategori = cp.Product.Category.Category_Name
            })
            .ToList();
        dataGridView1.DataSource = list;
        dataGridView1.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
        dataGridView1.Columns["CusPro_Id"].Visible = false;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

CustomerProduct Sınıfı için LİSTELEME İşlemi

Şimdi tekrar kodu çalıştırıp ikinci form'a geçiş yapıp listeleme işlemini gerçekleştiriyoruz. Görüldüğü gibi istemediğimiz sütunumuz artık görünmüyor.

Form_CustomerProduct

	MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
	Furkan	ATLAN	Keten Kumaş	750	Giyim
	Ahmet	TEKOK	Uzaktan Kumandalı Araba	4280	Oyuncak
	Enes	GÜMÜŞ	İphone 16 Promax	80000	Elektrik-Elektronik

CustomerProduct Sınıfı için Form Oluşturulması

Şimdi sıra geldi, combobox'lara. Bu combobox'lar form ilk yüklendiğinde yani daha hiçbir butona tıklanmamışken ve hiçbir olay gerçekleşmemişken Start butonuna tıklanıp kodlar ilk çalıştırıldığında bu combobox'lar boş gözükmesin ve veriler o an içerisine yüklensin istiyoruz. Bunun için form üzerinde herhangi bir bileşene tıklamadan form'un üzerine çift tıklayınca sizi Form_Load olayının içerisine götürecektir (Form_load metodunun içine götürecektir)

Değerlerin Combobox'a Yüklenmesi

Projeyi çalıştırmadan henüz form'un tasarım sekmesindeyken şekildeki kırmızı ile seçili alan gibi boş bir yere çift tıklarsanız sizi ilgili metodun içerisine yönlendirecektir.

The screenshot shows a Windows Forms application in design mode. The title bar displays several open files: `Form_CustomerProduct.cs`, `Form_Customer....cs [Tasarım]`, `CustomerProduct.cs`, `Form1.cs`, `CustomerDbContext.cs`, `Customer.cs`, and `Form1.cs [Tasarım]`. The main design area is a large gray rectangle. Below it, there are two buttons: `LİSTELE` and `SİL`. At the bottom, there is a group box labeled `groupBox1` containing a label `MÜŞTERİ` and a combobox. A red rectangle is drawn on the right side of the form, indicating a target area for a double-click event.

Değerlerin Combobox'a Yüklmesi

Aşağıdaki gibi bir load metodunun içerisine yönlendirileceksiniz:

Form_load metoduna yazdığımız metotlar ve açıklamaları sonraki sayfada verilmiştir.

```
private void Form_CustomerProduct_Load(object sender, EventArgs e)
{
}
}
```

Değerlerin Combobox'a Yükleneşi

```
try
{
    combo_customer.DataSource = db.Customers // Table_Customers tablosunu seçiyoruz
    .OrderBy(c => c.Customer_Name) // OrderBy metodu ile müşterinin adına göre alfabetik şekilde sıralıyoruz
    .Select(c => new
    {
        c.Customer_Id, // combobox'ta gösterilecek değerler arka planda bu ID bilgisine göre seçilecek
        FullName = c.Customer_Name + " " + c.Customer_Surname // combobox'ta değerler bu şekilde listelenecek
    })
    .ToList();
    combo_customer.DisplayMember = "FullName"; // Combobox'ta bize gösterilecek isimler FullName'den seçiliyor (yukarıdaki
    değişken)
    combo_customer.ValueMember = "Customer_Id"; // Combobox'ta gösterilen değerler arka planda ID bilgisinden çekiliyor
    // aynı şeyler aşağıdakiler için de geçerli
    combo_product.DataSource = db.Products
        .Include(p => p.Category)
        .OrderBy(p => p.Product_Name)
        .Select(p => new
        {
            p.Product_Id,
            ProductInfo = p.Product_Name + " - " + p.Product_Price + "₺ (" + p.Category.Category_Name + ")"
        })
        .ToList();
    combo_product.DisplayMember = "ProductInfo";
    combo_product.ValueMember = "Product_Id";
    btn_listele.PerformClick();
}
catch (Exception ex)
{
    MessageBox.Show($"Hata = {ex.Message}");}
```

Değerlerin Combobox'a Yüklenmesi

```
private void Form_CustomerProduct_Load(object sender, EventArgs e)
{
    try
    {
        combo_customer.DataSource = db.Customers // Table_Customers tablosunu seçiyoruz
        .OrderBy(c => c.Customer_Name) // OrderBy metodu ile müşterinin adına göre alfabetik şekilde sıralıyoruz
        .Select(c => new
        {
            c.Customer_Id, // combobox'ta gösterilecek değerler arka planda bu ID bilgisine göre seçilecek
            FullName = c.Customer_Name + " " + c.Customer_Surname // combobox'ta değerler bu şekilde listelenecek
        })
        .ToList();
        combo_customer.DisplayMember = "FullName"; // Combobox'ta bize gösterilecek isimler FullName'den seçiliyor (yukarıdaki değişken)
        combo_customer.ValueMember = "Customer_Id"; // Combobox'ta gösterilen değerler arka planda ID bilgisinden çekiliyor

        // aynı şeyler aşağıdakiler için de geçerli
        combo_product.DataSource = db.Products
        .Include(p => p.Category)
        .OrderBy(p => p.Product_Name)
        .Select(p => new
        {
            p.Product_Id,
            ProductInfo = p.Product_Name + " - " + p.Product_Price + "₺ (" + p.Category.Category_Name + ")"
        })
        .ToList();
        combo_product.DisplayMember = "ProductInfo";
        combo_product.ValueMember = "Product_Id";

        btn_listele.PerformClick();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

Değerlerin Combobox'a Yüklmesi

Şimdi programımızı çalıştırıyoruz ve bu defa combobox'ların dolu bir şekilde geldiğini görebiliriz.

MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
Furkan	ATLAN	Keten Kumaş	750	Giyim
Ahmet	TEKOK	Uzaktan Kumandalı Araba	4280	Oyuncak
Enes	GÜMÜŞ	Iphone 16 Promax	80000	Elektrik-Elektronik

LİSTELE

SİL

groupBox1

MÜŞTERİ

Ahmet TEKOK



ÜRÜN

Blender - 850tl (Ev Ürünleri)



ANA FORM

EKLE

EKLEME İŞLEMİ (Add Operation)

Şu ana kadar listeleme ve verilerin başlangıçta combobox'lara yüklenmesi işlemlerini yerine getirdik. Şimdi sıra geldi ekleme işlemine. Combobox'lardaki seçili verileri EKLE butonuna tıklayınca ekleyecek ve hemen yeni verileri datagridview'da listeleyecek.

Bunun için EKLE yazan butona çift tıklayıp onun click olayı içerisine kodlarımızı yazacağız.

EKLEME İŞLEMİ (Add Operation)

Her zamanki gibi try-catch bloklarımızı açıyoruz.

```
private void btn_ekle_Click(object sender, EventArgs e)
{
    try
    {
        // ...
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

EKLEME İŞLEMİ (Add Operation)

Aşağıdaki kodlarla ekleme işlemini yapıyoruz. btn_ekle'nin içerisine bu kodları yazıyoruz:

```
try
{
    // ComboBox'lardan seçilen ID'lerini alıyoruz
    int selectedCustomerId = (int)combo_customer.SelectedValue;
    int selectedProductId = (int)combo_product.SelectedValue;

    // Yeni CustomerProduct nesnesi oluşturuyoruz
    var newRecord = new CustomerProduct
    {
        Customer_Id = selectedCustomerId,
        Product_Id = selectedProductId
    };
    // CustomerProduct tablosuna yeni kaydımızı ekliyoruz
    db.CustomerProducts.Add(newRecord);
    db.SaveChanges();

    MessageBox.Show("Kayıt başarıyla eklendi.");

    // Listeleme işleminin tanımlı olduğu btn_listele'nin kodlarını çalıştırıyoruz
    btn_listele.PerformClick();
}
catch (Exception ex)
{
    MessageBox.Show($"Hata = {ex.Message}");}
```


EKLEME İŞLEMİ (Add Operation)

```
private void btn_ekle_Click(object sender, EventArgs e)
{
    try
    {
        // ComboBox'lardan seçilen ID'lerini alıyoruz
        int selectedCustomerId = (int)combo_customer.SelectedValue;
        int selectedProductId = (int)combo_product.SelectedValue;

        // Yeni CustomerProduct nesnesi oluşturuyoruz
        var newRecord = new CustomerProduct
        {
            Customer_Id = selectedCustomerId,
            Product_Id = selectedProductId
        };

        // CustomerProduct tablosuna yeni kaydımızı ekliyoruz
        db.CustomerProducts.Add(newRecord);
        db.SaveChanges();

        MessageBox.Show("Kayıt başarıyla eklendi.");

        // Listeleme işleminin tanımlı olduğu btn_listele'nin kodlarını çalıştırıyoruz
        btn_listele.PerformClick();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

EKLEME İŞLEMİ (Add Operation)

Şimdi kodumuzu çalıştırıp ikinci form'a geçiş yapıp combobox'larda istediğimiz verileri seçip ekle butonuna tıklıyoruz

	MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
▶	Furkan	ATLAN	Keten Kumaş	750	Giyim
	Ahmet	TEKOK	Uzaktan Kumandalı Araba	4280	Oyuncak
	Enes	GÜMÜŞ	Iphone 16 Promax	80000	Elektrik-Elektronik

LISTELE

SİL

groupBox1

MÜŞTERİ

Tuğçe HAYAT

ÜRÜN

Uzaktan Kumandalı Araba - 4280 (Oyuncak)

EKLE

ANA FORM

EKLEME İŞLEMİ (Add Operation)

İşlemimiz başarılı olduğu için MessageBox ile uyarısını verdi. Bunu kapatınca, datagridview'daki liste güncellenecek ve son kaydımız görünecek.

	MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
▶	Furkan	ATLAN	Keten Kumaş	750	Giyim
	Ahmet	TEKOK	Uzaktan Kumandalı Araba	4280	Oyuncak
	Enes	GÜMÜŞ	Iphone 16 Promax	80000	Elektrik-Elektronik

LİSTELE

Kayıt başarıyla eklendi.

Tamam

groupBox1

MÜŞTERİTuğçe HAYAT

ÜRÜNUzaktan Kumandalı Araba - 42800 (Oyuncak)

EKLE

ANA FORM

EKLEME İŞLEMİ (Add Operation)

	MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
►	Furkan	ATLAN	Keten Kumaş	750	Giyim
	Ahmet	TEKOK	Uzaktan Kumandalı Araba	4280	Oyuncak
	Enes	GÜMÜŞ	Iphone 16 Promax	80000	Elektrik-Elektronik
	Tuğçe	HAYAT	Uzaktan Kumandalı Araba	4280	Oyuncak

LİSTELE

SİL

groupBox1

MÜŞTERİ

Tuğçe HAYAT



ÜRÜN

Uzaktan Kumandalı Araba - 4280₺ (Oyuncak)



ANA FORM

EKLE

SİLME İŞLEMİ (Delete Operation)

Son olarak silme işlemi için SİL yazan butonumuza çift tıklayıp try-catch bloklarını açıp aşağıdaki kodu yazıyoruz:

NOT: ilk olarak kullanıcıya soru soracak. Bu kaydı silmek istiyoruz sorusuna kullanıcı Evet yazan butonu tıklarsa silme işlemi gerçekleşecek. Bundan dolayı ilk olarak soru sorma ve dönen cevabın evet olduğu durum için if blokları açacağız:

```
try
{
    DialogResult sonuc = MessageBox.Show(
        "Silmek istediğinize emin misiniz?",
        "Silme Onayı",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question);
    if (sonuc == DialogResult.Yes)
    {

    }
}
catch (Exception ex)
{
    MessageBox.Show($"Hata = {ex.Message}");
}
```

SİLME İŞLEMİ (Delete Operation)

```
private void btn_sil_Click(object sender, EventArgs e)
{
    try
    {
        DialogResult sonuc = MessageBox.Show(
            "Silmek istediğinize emin misiniz?",
            "Silme Onayı",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question);
        if (sonuc == DialogResult.Yes)
        {
            // Silme işlemi buraya yazılacak
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

NOT: Sonraki sayfadaki kodları buradaki if bloğunun içerisine yazıyoruz!

SİLME İŞLEMİ (Delete Operation)

```
// DataGridView'da bir satır seçili mi?
if (dataGridView1.CurrentRow != null)
{
    // Seçilen satırdaki CusPro_Id değerini alıyoruz
    int selectedId = Convert.ToInt32(dataGridView1.CurrentRow.Cells["CusPro_Id"].Value);

    // İlgili veritabanı kaydını buluyoruz
    var recordToDelete = db.CustomerProducts.Find(selectedId);

    if (recordToDelete != null)
    {
        // Kaydı siliyoruz
        db.CustomerProducts.Remove(recordToDelete);
        db.SaveChanges();

        MessageBox.Show("Kayıt başarıyla silindi.");

        // Datagridview'daki listeyi güncelliyoruz
        btn_listele.PerformClick();
    }
    else
    {
        MessageBox.Show("Kayıt bulunamadı.");
    }
}
```

SİLME İŞLEMİ (Delete Operation)

```
try
{
    DialogResult sonuc = MessageBox.Show(
        "Silmek istediğinize emin misiniz?",
        "Silme Onayı",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question);
    if (sonuc == DialogResult.Yes)
    {
        // DataGridView'da bir satır seçili mi?
        if (dataGridView1.CurrentRow != null)
        {
            // Seçilen satırdaki CusPro_Id değerini alıyoruz
            int selectedId = Convert.ToInt32(dataGridView1.CurrentRow.Cells["CusPro_Id"].Value);

            // İlgili veritabanı kaydını buluyoruz
            var recordToDelete = db.CustomerProducts.Find(selectedId);

            if (recordToDelete != null)
            {
                // Kaydı siliyoruz
                db.CustomerProducts.Remove(recordToDelete);
                db.SaveChanges();

                MessageBox.Show("Kayıt başarıyla silindi.");

                // Datagridview'daki listeyi güncelliyoruz
                btn_listele.PerformClick();
            }
            else
            {
                MessageBox.Show("Kayıt bulunamadı.");
            }
        }
    }
}
```


SİLME İŞLEMİ (Delete Operation)

Şimdi kodumuzu çalıştırıp, ikinci form'a geçiş yapıp seçtiğimiz bir kaydı silmeye çalışalım. Bu örnekte ikinci kaydı silmek istiyoruz. Görüldüğü gibi ilk olarak silme işlemiyle ilgili uyarı verdi:

MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
Furkan	ATLAN	Keten Kumaş	750	Giyim
Ahmet	TEKOK	Uzaktan Kumandalı Araba	4280	Oyuncak
Enes	GÜMÜŞ	Iphone 16 Promax	80000	Elektrik-Elektronik
Tuğçe	HAYAT	Uzaktan Kumandalı Araba	4280	Oyuncak

Silme Onayı

?

Silmek istediğinize emin misiniz?

Evet

Hayır

groupBox1

MÜŞTERİ

Ahmet TEKOK

ORUN

Blender - 850t (Ev Ürünleri)

EKLE

ANA FORM

SİLME İŞLEMİ (Delete Operation)

Evet dedikten sonra silme işlemi başarılı olduğu için bize uyarı veriyor:

MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
Furkan	ATLAN	Keten Kumaş	750	Giyim
Ahmet	TEKOK	Uzaktan Kumandalı Araba	4280	Oyuncak
Enes	GÜMÜŞ	Iphone 16 Promax	80000	Elektrik-Elektronik
Tuğçe	HAYAT	Uzaktan Kumandalı Araba	4280	Oyuncak

LISTELE

Kayıt başarıyla silindi.
Tamam

groupBox1

MÜŞTERİAhmet TEKOK

ÜRÜNBlender - 850k (Ev Ürünleri)

EKLE

ANA FORM

SİLME İŞLEMİ (Delete Operation)

Uyarıyı kapattıktan sonra datagridview'in güncellendiğini ve kaydın artık olmadığını görebiliriz:

	MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
▶	Furkan	ATLAN	Keten Kumaş	750	Giyim
	Enes	GÜMÜŞ	İphone 16 Promax	80000	Elektrik-Elektronik
	Tuğçe	HAYAT	Uzaktan Kumandalı Araba	4280	Oyuncak

LİSTELE

SİL

groupBox1

MÜŞTERİ

Ahmet TEKOK

ÜRÜN

Blender - 850€ (Ev Ürünleri)

EKLE

ANA FORM

EK İŞLEMLER-LINQ Sorguları

Bu işlemle birlikte veri tabanımızdaki temel işlemlerin tamamını bitirmiş olduk.

Şimdi, ikinci formumuza bir groupbox ve bu groupbox'ın içerisine de iki tane RadioButton ekliyoruz. Burada amacımız, ilk RadioButton'a tıklandığında bize en çok ürün alan müşterinin adını ve aldığı ürün sayısını getirsin istiyoruz.

İkinci RadioButton'a da tıklandığında bize aldığı ürünlerin toplam fiyatı en yüksek müşteriye ve ürünlerin toplam fiyat bilgisini getirsin istiyoruz.

Bunları LINQ sorgusu ile yapacağız.

Bunun için 2 adet label bileşenine de ihtiyacımız var.

Bu sorgunun cevaplarını bu label değişkenlerine yazdıracağız çünkü

Bileşenlerin name (id) bilgileri şöyle olacak:

RadioButton'lar için rd_1, rd_2 (sorgu1, sorgu2'ye karşılık gelecekler)

lbl_1, lbl_2 (yanıt1 ve yanıt2'ye karşılık gelecekler)

EK İŞLEMLER-LINQ Sorguları

Ekleme işlemlerinden sonra formumuzun son hali aşağıdaki gibidir:

	MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
▶	Furkan	ATLAN	Keten Kumaş	750	Giyim
	Enes	GÖMÜŞ	İphone 16 Promax	80000	Elektrik-Elektronik
	Tuğçe	HAYAT	Uzaktan Kumandalı Araba	4280	Oyuncak

LISTELE

SİL

ANA FORM

groupBox1

MÜŞTERİ

Ahmet TEKOK

ÜRÜN

Blender - 850tl (Ev Ürünleri)

EKLE

İSTATİSTİKLER

radioButton1

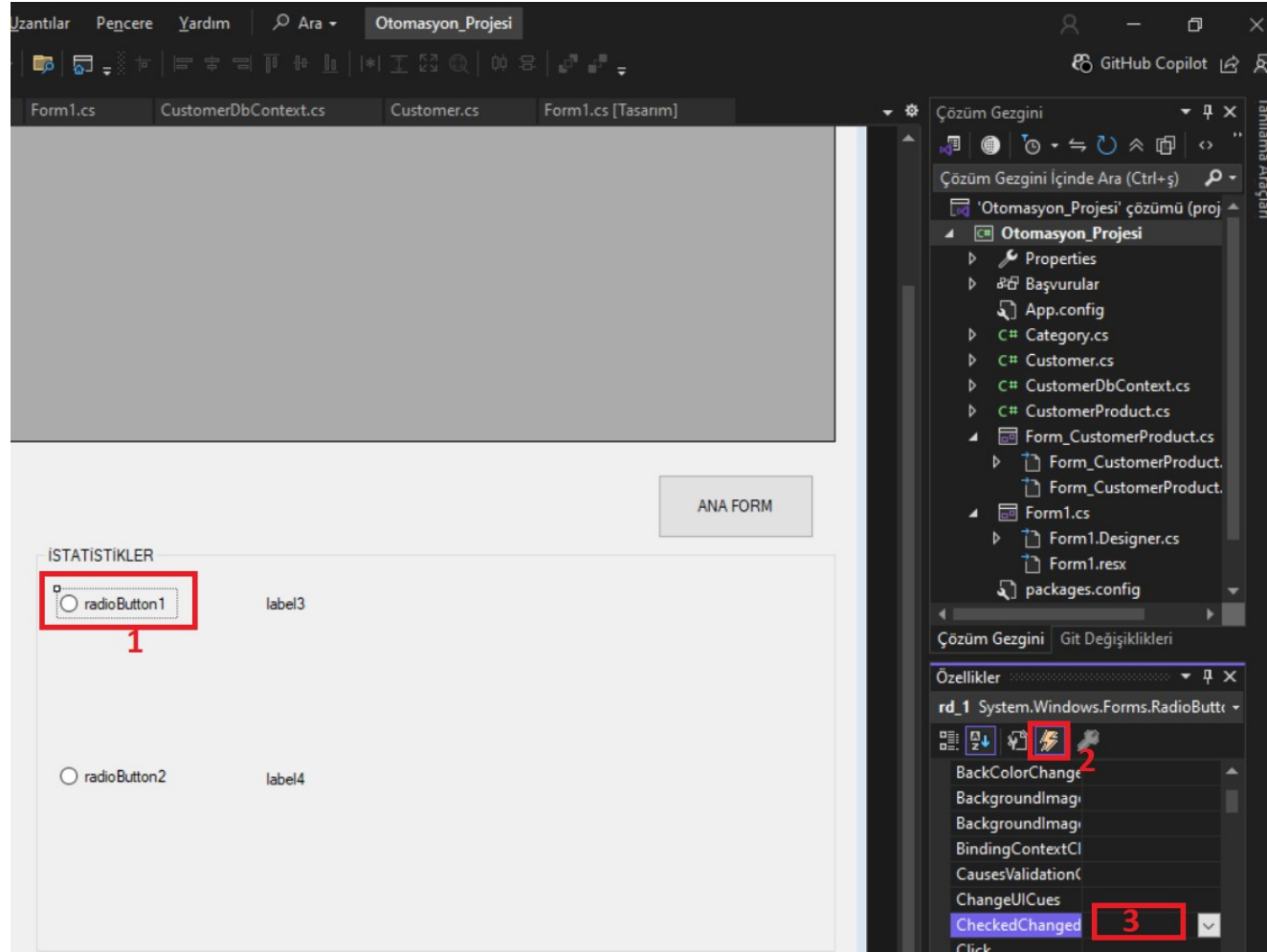
label3

radioButton2

label4

EK İŞLEMLER-LINQ Sorguları

RadioButton'lara tıklandığı zaman bu işlemler yapılsın istediğimiz için RadioButton'ların CheckedChanged olayına (event) bu kodu yazacağız. Şimdi ilk olarak form üzerindeki 1. radiobutton'umuza bir defa sol tıklayıp sağ alt taraftaki Özellikler (Properties) penceresinden şimşek simgesine tıklayıp CheckedChanged yazan yerin yanındaki boş alana çift tıklayınız.



EK İŞLEMLER-LINQ Sorguları

Her zamanki gibi önce try-catch bloklarını açıyoruz:

```
private void rd_1_CheckedChanged(object sender, EventArgs e)
{
    try
    {

    }
    catch (Exception ex)
    {

        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

Daha sonra try blokları içerisine diğer sayfadaki kodları yazıyoruz

EK İŞLEMLER-LINQ Sorguları

```
// Eğer radioButton1 seçiliyse işlem yap
if (rd_1.Checked)
{
    // CustomerProduct tablosunu müşteriye göre grupla
    var result = db.CustomerProducts
        .GroupBy(cp => new
        {
            cp.Customer.Customer_Id,
            cp.Customer.Customer_Name,
            cp.Customer.Customer_Surname
        })
        // Her grup için müşteri adı, soyadı ve toplam ürün sayısını al
        .Select(g => new
        {
            FullName = g.Key.Customer_Name + " " + g.Key.Customer_Surname,
            TotalCount = g.Count()
        })
        // Ürün sayısına göre azalan sırala (en çok ürünü alan en üstte)
        .OrderByDescending(x => x.TotalCount)
        // En çok ürün alan müşteriyi al
        .FirstOrDefault();
    // Eğer sonuç varsa label1'e yaz
    if (result != null)
    {
        lbl_1.Text = $"{result.FullName} {result.TotalCount} ürün almış";
    }
    else
    {
        lbl_1.Text = "Kayıt bulunamadı.";
    }
}
```


EK İŞLEMLER-LINQ Sorguları

```
try
{
    // Eğer radioButton1 seçiliyse işlem yap
    if (rd_1.Checked)
    {
        // CustomerProduct tablosunu müşteriye göre grupta
        var result = db.CustomerProducts
            .GroupBy(cp => new
            {
                cp.Customer.Customer_Id,
                cp.Customer.Customer_Name,
                cp.Customer.Customer_Surname
            })
            // Her grup için müşteri adı, soyadı ve toplam ürün sayısını al
            .Select(g => new
            {
                FullName = g.Key.Customer_Name + " " + g.Key.Customer_Surname,
                TotalCount = g.Count()
            })
            // Ürün sayısına göre azalan sırala (en çok ürünü alan en üstte)
            .OrderByDescending(x => x.TotalCount)
            // En çok ürün alan müşteriyi al
            .FirstOrDefault();

        // Eğer sonuç varsa labell'e yaz
        if (result != null)
        {
            lbl_1.Text = $"{result.FullName} {result.TotalCount} ürün almış";
        }
        else
        {
            lbl_1.Text = "Kayıt bulunamadı.";
        }
    }
}
```

EK İŞLEMLER-LINQ Sorguları

Bu işlemi yapmadan önce bir kullanıcıya bir ürün daha aldıralım ki kodumuz sağlıklı bir sonuç versin. Çünkü şu ana kadar 3 müşteri 1 ürün almıştı.

Furkan müşterisine bir ürün daha aldırıyoruz.

MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
Furkan	ATLAN	Keten Kumaş	750	Giyim
Enes	GÜMÜŞ	Iphone 16 Promax	80000	Elektrik-Elektronik
Tuğçe	HAYAT	Uzaktan Kumandalı Araba	4280	Oyuncak
Furkan	ATLAN	Samsung Galaxy S24	35000	Elektrik-Elektronik

LISTELE

SİL

ANA FORM

groupBox1

MÜŞTERİ

Furkan ATLAN

ÜRÜN

Samsung Galaxy S24 - 35000 (Elektrik-Elektronik)

EKLE

İSTATİSTİKLER

radioButton1

label3

radioButton2

label4

EK İŞLEMLER-LINQ Sorguları

Şimdi kodumuzu çalıştırıp ikinci forma geçiş yapıp ilk radiobutton'a tıklıyoruz:

	MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
►	Furkan	ATLAN	Keten Kumaş	750	Giyim
	Enes	GÜMÜŞ	İphone 16 Promax	80000	Elektrik-Elektronik
	Tuğçe	HAYAT	Uzaktan Kumandalı Araba	4280	Oyuncak
	Furkan	ATLAN	Samsung Galaxy S24	35000	Elektrik-Elektronik

LISTELE

SİL

ANA FORM

groupBox1

MÜŞTERİ

Furkan ATLAN

ÜRÜN

Samsung Galaxy S24 - 35000₺ (Elektrik-Elektronik)

EKLE

İSTATİSTİKLER

☒ radioButton1

label3

☐ radioButton2

label4

EK İŞLEMLER-LINQ Sorguları

Görüldüğü gibi kodumuz başarılı bir şekilde çalıştı ve doğru sonucu verdi:

	MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
►	Furkan	ATLAN	Keten Kumaş	750	Giyim
	Enes	GÜMÜŞ	İphone 16 Promax	80000	Elektrik-Elektronik
	Tuğçe	HAYAT	Uzaktan Kumandalı Araba	4280	Oyuncak
	Furkan	ATLAN	Samsung Galaxy S24	35000	Elektrik-Elektronik

LİSTELE

SİL

ANA FORM

groupBox1

MÜŞTERİ

Furkan ATLAN

ÜRÜN

Samsung Galaxy S24 - 35000₺ (Elektrik-Elektronik)

EKLE

İSTATİSTİKLER

☒ radioButton1

Furkan ATLAN 2 ürün almış

☐ radioButton2

label4

EK İŞLEMLER-LINQ Sorguları

Şimdi sıra geldi aynı şekilde ikinci radiobutton'un CheckedChanged olayına kod yazmaya. Aşağıdaki gibi try-catch blokları açıp sonra da diğer sayfadaki kodları try blokunun içine yazacağız:

```
private void rd_2_CheckedChanged(object sender, EventArgs e)
{
    try
    {

    }
    catch (Exception ex)
    {

        MessageBox.Show($"Hata = {ex.Message}");

    }
}
```

EK İŞLEMLER-LINQ Sorguları

```
private void rd_2_CheckedChanged(object sender, EventArgs e)
{
    try
    {
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Hata = {ex.Message}");
    }
}
```

EK İŞLEMLER-LINQ Sorguları

```
// Eğer radioButton2 seçiliyse işlem yap
if (rd_2.Checked)
{
    // CustomerProduct tablosunu, ilişkili ürünleri de dahil ederek sorgula
    var result = db.CustomerProducts
        .Include(cp => cp.Product) // Ürün fiyat bilgisine erişebilmek için
        .GroupBy(cp => new
        {
            cp.Customer.Customer_Id,
            cp.Customer.Customer_Name,
            cp.Customer.Customer_Surname
        })
        // Her grup için müşteri adı, soyadı ve aldığı ürünlerin toplam fiyatını al
        .Select(g => new
        {
            FullName = g.Key.Customer_Name + " " + g.Key.Customer_Surname,
            TotalPrice = g.Sum(cp => cp.Product.Product_Price)
        })
        // Toplam fiyata göre azalan sırala (en pahalı alışveriş en üstte)
        .OrderByDescending(x => x.TotalPrice)
        // En pahalı alışveriş yapan müşteriyi al
        .FirstOrDefault();
    // Eğer sonuç varsa label2'ye yaz
    if (result != null)
    {
        lbl_2.Text = $"{result.FullName} {result.TotalPrice} TL";
    }
    else
    {
        lbl_2.Text = "Kayıt bulunamadı.";
    }
}
```

EK İŞLEMLER-LINQ Sorguları

```
try
{
    // Eğer radioButton2 seçiliyse işlem yap
    if (rd_2.Checked)
    {
        // CustomerProduct tablosunu, ilişkili ürünleri de dahil ederek sorgula
        var result = db.CustomerProducts
            .Include(cp => cp.Product) // Ürün fiyat bilgisine erişebilmek için
            .GroupBy(cp => new
            {
                cp.Customer.Customer_Id,
                cp.Customer.Customer_Name,
                cp.Customer.Customer_Surname
            })
            // Her grup için müşteri adı, soyadı ve aldığı ürünlerin toplam fiyatını al
            .Select(g => new
            {
                FullName = g.Key.Customer_Name + " " + g.Key.Customer_Surname,
                TotalPrice = g.Sum(cp => cp.Product.Product_Price)
            })
            // Toplam fiyata göre azalan sırala (en pahalı alışveriş en üstte)
            .OrderByDescending(x => x.TotalPrice)
            // En pahalı alışveriş yapan müşteriyi al
            .FirstOrDefault();

        // Eğer sonuç varsa label2'ye yaz
        if (result != null)
        {
            lbl_2.Text = $"{result.FullName} {result.TotalPrice} TL";
        }
        else
        {
            lbl_2.Text = "Kayıt bulunamadı.";
        }
    }
}
```


EK İŞLEMLER-LINQ Sorguları

Şimdi programımızı çalıştırıp ikinci forma geçiş yapıyoruz ve ikinci radiobutton'a tıklıyoruz.

	MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
►	Furkan	ATLAN	Keten Kumaş	750	Giyim
	Enes	GÜMÜŞ	İphone 16 Promax	80000	Elektrik-Elektronik
	Tuğçe	HAYAT	Uzaktan Kumandalı Araba	4280	Oyuncak
	Furkan	ATLAN	Samsung Galaxy S24	35000	Elektrik-Elektronik

LİSTELE

SİL

ANA FORM

groupBox1

MÜŞTERİ

Ahmet TEKOK

ORÜN

Blender - 8500 (Ev Ürünleri)

EKLE

İSTATİSTİKLER

radioButton1

label3

radioButton2

label4

EK İŞLEMLER-LINQ Sorguları

Sonucu ilgili label bileşenine doğru bir skorla yazdırıldığını görebiliyoruz:

	MüşteriAdı	MüşteriSoyadı	ÜrünAdı	ÜrünFiyatı	Kategori
►	Furkan	ATLAN	Keten Kumaş	750	Giyim
	Enes	GÜMÜŞ	İphone 16 Promax	80000	Elektrik-Elektronik
	Tuğçe	HAYAT	Uzaktan Kumandalı Araba	4280	Oyuncak
	Furkan	ATLAN	Samsung Galaxy S24	35000	Elektrik-Elektronik

LISTELE

SİL

ANA FORM

groupBox1

MÜŞTERİ

Ahmet TEKOK

ÜRÜN

Blender - 850tl (Ev Ürünleri)

EKLE

İSTATİSTİKLER

radioButton1

label3

radioButton2

Enes GÜMÜŞ 80000 TL

EK İŞLEMLER-UML CLASS DIAGRAMS

Bu işlemle birlikte LINQ sorgularını da tamamlamış oluyoruz.

Tüm slaytı ayrıntılı bir şekilde inceleyip, özellikle tablo ve bileşen isimlerine dikkatle yaklaşın. Bu kodları, kendi projenize uyarlayıp EntityFramework konseptinde kendi otomasyonunuzu oluşturun.

